P2300R10

`std::execution`

Published Proposal, 2024-06-28

Authors:

Michał Dominiak

Georgy Evtushenko

Lewis Baker

Lucian Radu Teodorescu

Lee Howes

Kirk Shoop

Michael Garland

Eric Niebler

Bryce Adelstein Lelbach

Source:

GitHub

Issue Tracking:

GitHub

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Audience:

SG1, LEWG

Table of Contents

1 Introduction

1.1 Motivation

1.2 Priorities

1.3 Examples: End User

1.3.1 Hello world

1.3.2 Asynchronous inclusive scan

1.3.3 Asynchronous dynamically-sized read

1.4 Asynchronous Windows socket `recv`

1.4.1 More end-user examples

1.4.1.1 Sudoku solver

1.4.1.2 File copy
1.4.1.3 Echo server
1.5 Examples: Algorithms
1.5.1 `then`
1.5.2 `retry`
1.6 Examples: Schedulers
1.6.1 Inline scheduler
1.6.2 Single thread scheduler
1.7 Examples: Server theme
1.7.1 Composability with `execution::let_*`
1.7.2 Moving between execution resources with `execution::starts_on` and `execution::continues_on`
1.8 Design changes from P0443
1.9 Prior art
1.9.1 Futures
1.9.2 Coroutines
1.9.3 Callbacks
1.10 Field experience
1.10.1 libunifex
1.10.2 stdexec
1.10.3 Other implementations
1.10.4 Inspirations

2 Revision history
2.1 R10
2.2 R9
2.3 R8
2.4 R7
2.5 R6
2.5.1 Environments and attributes
2.6 R5
2.7 R4
2.7.1 Dependently-typed senders
2.8 R3
2.9 R2
2.10 R1
2.11 R0

3 Design - introduction

3.1 Conventions
3.2 Queries and algorithms

4 Design - user side
4.1 Execution resources describe the place of execution
4.2 Schedulers represent execution resources
4.3 Senders describe work
4.4 Senders are composable through sender algorithms
4.5 Senders can propagate completion schedulers
4.5.1 `execution::get_completion_scheduler`
4.6 Execution resource transitions are explicit
4.7 Senders can be either multi-shot or single-shot
4.8 Senders are forkable
4.9 Senders support cancellation
4.9.1 Cancellation design summary
4.9.2 Support for cancellation is optional
4.9.3 Cancellation is inherently racy
4.9.4 Cancellation design status
4.10 Sender factories and adaptors are lazy
4.10.1 Eager execution leads to detached work or worse
4.10.2 Eager senders complicate algorithm implementations
4.10.3 Eager senders incur cancellation-related overhead
4.10.4 Eager senders cannot access execution resource from the receiver
4.11 Schedulers advertise their forward progress guarantees
4.12 Most sender adaptors are pipeable
4.13 A range of senders represents an async sequence of data
4.14 Senders can represent partial success
4.15 All awaitables are senders
4.16 Many senders can be trivially made awaitable
4.17 Cancellation of a sender can unwind a stack of coroutines
4.18 Composition with parallel algorithms
4.19 User-facing sender factories
4.19.1 `execution::schedule`
4.19.2 `execution::just`
4.19.3 `execution::just_error`
4.19.4 `execution::just_stopped`
4.19.5 `execution::read_env`
4.20 User-facing sender adaptors
4.20.1 `execution::continues_on`

4.20.2 `execution::then`
4.20.3 `execution::upon_*`
4.20.4 `execution::let_*`
4.20.5 `execution::starts_on`
4.20.6 `execution::into_variant`
4.20.7 `execution::stopped_as_optional`
4.20.8 `execution::stopped_as_error`
4.20.9 `execution::bulk`
4.20.10 `execution::split`
4.20.11 `execution::when_all`
4.21 User-facing sender consumers
4.21.1 `this_thread::sync_wait`

5 Design - implementer side
5.1 Receivers serve as glue between senders
5.2 Operation states represent work
5.3 `execution::connect`
5.4 Sender algorithms are customizable
5.5 Sender adaptors are lazy
5.6 Lazy senders provide optimization opportunities
5.7 Execution resource transitions are two-step
5.8 All senders are typed
5.9 Customization points

6 Specification

14 Exception handling [except]
14.6 Special functions **[except.special]**
14.6.2 The `std::terminate` function **[except.terminate]**

16 Library introduction [library]

17 Language support library [cpp]
17.3 Implementation properties **[support.limits]**
17.3.2 Header `<version>` synopsis **[version.syn]**

22 General utilities library [utilities]
22.10 Function objects **[function.objects]**
22.10.2 Header `<functional>` synopsis **[functional.syn]**

33 Concurrency support library [thread]

33.3 Stop tokens **[thread.stoptoken]**
33.3.1 Introduction **[thread.stoptoken.intro]**
33.3.2 Header `<stop_token>` synopsis **[thread.stoptoken.syn]**
33.3.3 Stop token concepts **[stoptoken.concepts]**
33.3.4 Class `stop_token` **[stoptoken]**
33.3.4.1 General **[stoptoken.general]**
33.3.4.2 Constructors, copy, and assignment **[stoptoken.cons]**
33.3.4.3 Member functions **[stoptoken.mem]**
33.3.4.4 Non-member functions **[stoptoken.nonmembers]**
33.3.5 Class `stop_source` **[stopsource]**
33.3.5.1 General **[stopsource.general]**
33.3.5.2 Constructors, copy, and assignment **[stopsource.cons]**
33.3.5.3 Member functions **[stopsource.mem]**
33.3.5.4 Non-member functions **[stopsource.nonmembers]**
33.3.6 Class template `stop_callback` **[stopcallback]**
33.3.6.1 General **[stopcallback.general]**
33.3.6.2 Constructors and destructor **[stopcallback.cons]**
33.3.7 Class `never_stop_token` **[stoptoken.never]**
33.3.7.1 General **[stoptoken.never.general]**
33.3.8 Class `inplace_stop_token` **[stoptoken.inplace]**
33.3.8.1 General **[stoptoken.inplace.general]**
33.3.8.2 Member functions **[stoptoken.inplace.members]**
33.3.9 Class `inplace_stop_source` **[stopsource.inplace]**
33.3.9.1 General **[stopsource.inplace.general]**
33.3.9.2 Constructors, copy, and assignment **[stopsource.inplace.cons]**
33.3.9.3 Members **[stopsource.inplace.mem]**
33.3.10 Class template `inplace_stop_callback` **[stopcallback.inplace]**
33.3.10.1 General **[stopcallback.inplace.general]**
33.3.10.2 Constructors and destructor **[stopcallback.inplace.cons]**

34 Execution control library [exec]
34.1 General **[exec.general]**
34.2 Queries and queryables **[exec.queryable]**
34.2.1 General **[exec.queryable.general]**
34.2.2 *`queryable`* concept **[exec.queryable.concept]**
34.3 Asynchronous operations **[async.ops]**
34.4 Header `<execution>` synopsis **[exec.syn]**
34.5 Queries **[exec.queries]**
34.5.1 `forwarding_query` **[exec.fwd.env]**

34.5.2 `get_allocator` **[exec.get.allocator]**
34.5.3 `get_stop_token` **[exec.get.stop.token]**
34.5.4 `execution::get_env` **[exec.get.env]**
34.5.5 `execution::get_domain` **[exec.get.domain]**
34.5.6 `execution::get_scheduler` **[exec.get.scheduler]**
34.5.7 `execution::get_delegation_scheduler` **[exec.get.delegation.scheduler]**
34.5.8 `execution::get_forward_progress_guarantee` **[exec.get.forward.progress.guarantee]**
34.5.9 `execution::get_completion_scheduler` **[exec.completion.scheduler]**
34.6 Schedulers **[exec.sched]**
34.7 Receivers **[exec.recv]**
34.7.1 Receiver concepts **[exec.recv.concepts]**
34.7.2 `execution::set_value` **[exec.set.value]**
34.7.3 `execution::set_error` **[exec.set.error]**
34.7.4 `execution::set_stopped` **[exec.set.stopped]**
34.8 Operation states **[exec.opstate]**
34.8.1 `execution::start` **[exec.opstate.start]**
34.9 Senders **[exec.snd]**
34.9.1 General **[exec.snd.general]**
34.9.2 Sender concepts **[exec.snd.concepts]**
34.9.3 Awaitable helpers **[exec.awaitables]**
34.9.4 `execution::default_domain` **[exec.domain.default]**
34.9.4.1 Static members **[exec.domain.default.statics]**
34.9.5 `execution::transform_sender` **[exec.snd.transform]**
34.9.6 `execution::transform_env` **[exec.snd.transform.env]**
34.9.7 `execution::apply_sender` **[exec.snd.apply]**
34.9.8 `execution::get_completion_signatures` **[exec.getcomplsigs]**
34.9.9 `execution::connect` **[exec.connect]**
34.9.10 Sender factories **[exec.factories]**
34.9.10.1 `execution::schedule` **[exec.schedule]**
34.9.10.2 `execution::just`, `execution::just_error`, `execution::just_stopped` **[exec.just]**
34.9.10.3 `execution::read_env` **[exec.read.env]**
34.9.11 Sender adaptors **[exec.adapt]**
34.9.11.1 General **[exec.adapt.general]**
34.9.11.2 Sender adaptor closure objects **[exec.adapt.objects]**
34.9.11.3 `execution::starts_on` **[exec.starts.on]**
34.9.11.4 `execution::continues_on` **[exec.continues.on]**
34.9.11.5 `execution::schedule_from` **[exec.schedule.from]**
34.9.11.6 `execution::on` **[exec.on]**

34.9.11.7 `execution::then`, `execution::upon_error`, `execution::upon_stopped` **[exec.then]**
34.9.11.8 `execution::let_value`, `execution::let_error`, `execution::let_stopped`, **[exec.let]**
34.9.11.9 `execution::bulk` **[exec.bulk]**
34.9.11.10 `execution::split` **[exec.split]**
34.9.11.11 `execution::when_all` **[exec.when.all]**
34.9.11.12 `execution::into_variant` **[exec.into.variant]**
34.9.11.13 `execution::stopped_as_optional` **[exec.stopped.as.optional]**
34.9.11.14 `execution::stopped_as_error` **[exec.stopped.as.error]**
34.9.12 Sender consumers **[exec.consumers]**
34.9.12.1 `this_thread::sync_wait` **[exec.sync.wait]**
34.10 Sender/receiver utilities **[exec.utils]**
34.10.1 `execution::completion_signatures` **[exec.utils.cmplsigs]**
34.10.2 `execution::transform_completion_signatures` **[exec.utils.tfxcmplsigs]**
34.11 Execution contexts **[exec.ctx]**
34.11.1 `execution::run_loop` **[exec.run.loop]**
34.11.1.1 Associated types **[exec.run.loop.types]**
34.11.1.2 Constructor and destructor **[exec.run.loop.ctor]**
34.11.1.3 Member functions **[exec.run.loop.members]**
34.12 Coroutine utilities **[exec.coro.utils]**
34.12.1 `execution::as_awaitable` **[exec.as.awaitable]**
34.12.2 `execution::with_awaitable_senders` **[exec.with.awaitable.senders]**

Index

Terms defined by this specification

References

Informative References

§ 1. Introduction

This paper proposes a self-contained design for a Standard C++ framework for managing asynchronous execution on generic execution resources. It is based on the ideas in *A Unified Executors Proposal for C++* and its companion papers.

§ 1.1. Motivation

Today, C++ software is increasingly asynchronous and parallel, a trend that is likely to only continue going forward. Asynchrony and parallelism appears everywhere, from processor hardware interfaces, to networking, to file I/O, to GUIs, to accelerators. Every C++ domain and every platform needs to deal with asynchrony and

parallelism, from scientific computing to video games to financial services, from the smallest mobile devices to your laptop to GPUs in the world's fastest supercomputer.

While the C++ Standard Library has a rich set of concurrency primitives (`std::atomic`, `std::mutex`, `std::counting_semaphore`, etc) and lower level building blocks (`std::thread`, etc), we lack a Standard vocabulary and framework for asynchrony and parallelism that C++ programmers desperately need. `std::async`/`std::future`/`std::promise`, C++11's intended exposure for asynchrony, is inefficient, hard to use correctly, and severely lacking in genericity, making it unusable in many contexts. We introduced parallel algorithms to the C++ Standard Library in C++17, and while they are an excellent start, they are all inherently synchronous and not composable.

This paper proposes a Standard C++ model for asynchrony based around three key abstractions: schedulers, senders, and receivers, and a set of customizable asynchronous algorithms.

§ 1.2. Priorities

— Be composable and generic, allowing users to write code that can be used with many different types of execution resources.

— Encapsulate common asynchronous patterns in customizable and reusable algorithms, so users don't have to invent things themselves.

— Make it easy to be correct by construction.

— Support the diversity of execution resources and execution agents, because not all execution agents are created equal; some are less capable than others, but not less important.

— Allow everything to be customized by an execution resource, including transfer to other execution resources, but don't require that execution resources customize everything.

— Care about all reasonable use cases, domains and platforms.

— Errors must be propagated, but error handling must not present a burden.

— Support cancellation, which is not an error.

— Have clear and concise answers for where things execute.

— Be able to manage and terminate the lifetimes of objects asynchronously.

§ 1.3. Examples: End User

In this section we demonstrate the end-user experience of asynchronous programming directly with the sender algorithms presented in this paper. See § 4.19 User-facing sender factories, § 4.20 User-facing sender adaptors, and § 4.21 User-facing sender consumers for short explanations of the algorithms used in these code examples.

§ 1.3.1. Hello world

```
using namespace std::execution;

scheduler auto sch = thread_pool.scheduler();                                   // 1

sender auto begin = schedule(sch);                                              // 2
sender auto hi = then(begin, []{                                                // 3
    std::cout << "Hello world! Have an int.";                                   // 3
    return 13;                                                                  // 3
});                                                                             // 3
sender auto add_42 = then(hi, [](int arg) { return arg + 42; });                // 4

auto [i] = this_thread::sync_wait(add_42).value();                              // 5
```

This example demonstrates the basics of schedulers, senders, and receivers:

1. First we need to get a scheduler from somewhere, such as a thread pool. A scheduler is a lightweight handle to an execution resource.
2. To start a chain of work on a scheduler, we call § 4.19.1 execution::schedule, which returns a sender that completes on the scheduler. A sender describes asynchronous work and sends a signal (value, error, or stopped) to some recipient(s) when that work completes.
3. We use sender algorithms to produce senders and compose asynchronous work. § 4.20.2 execution::then is a sender adaptor that takes an input sender and a `std::invocable`, and calls the `std::invocable` on the signal sent by the input sender. The sender returned by `then` sends the result of that invocation. In this case, the input sender came from `schedule`, so its `void`, meaning it won't send us a value, so our `std::invocable` takes no parameters. But we return an `int`, which will be sent to the next recipient.
4. Now, we add another operation to the chain, again using § 4.20.2 execution::then. This time, we get sent a value - the `int` from the previous step. We add `42` to it, and then return the result.
5. Finally, we're ready to submit the entire asynchronous pipeline and wait for its completion. Everything up until this point has been completely asynchronous; the work may not have even started yet. To ensure the work has started and then block pending its completion, we use § 4.21.1 this_thread::sync_wait, which will either return a `std::optional<std::tuple<...>>` with the value sent by the last sender, or an empty `std::optional` if the last sender sent a stopped signal, or it throws an exception if the last sender sent an error.

§ 1.3.2. Asynchronous inclusive scan

```
using namespace std::execution;

sender auto async_inclusive_scan(scheduler auto sch,                            // 2
                                 std::span<const double> input,                 // 1
                                 std::span<double> output,                      // 1
                                 double init,                                   // 1
```

```
                                std::size_t tile_count)                    // 3
{
  std::size_t const tile_size = (input.size() + tile_count - 1) / tile_count;

  std::vector<double> partials(tile_count + 1);                            // 4
  partials[0] = init;                                                      // 4

  return just(std::move(partials))                                         // 5
       | continues_on(sch)
       | bulk(tile_count,                                                  // 6
           [ = ](std::size_t i, std::vector<double>& partials) {           // 7
             auto start = i * tile_size;                                   // 8
             auto end   = std::min(input.size(), (i + 1) * tile_size);     // 8
             partials[i + 1] = *--std::inclusive_scan(begin(input) + start,   // 9
                                                      begin(input) + end,     // 9
                                                      begin(output) + start); // 9
           })                                                              // 10
       | then(                                                             // 11
           [](std::vector<double>&& partials) {
             std::inclusive_scan(begin(partials), end(partials),           // 12
                                 begin(partials));                         // 12
             return std::move(partials);                                   // 13
           })
       | bulk(tile_count,                                                  // 14
           [ = ](std::size_t i, std::vector<double>& partials) {           // 14
             auto start = i * tile_size;                                   // 14
             auto end   = std::min(input.size(), (i + 1) * tile_size);     // 14
             std::for_each(begin(output) + start, begin(output) + end,     // 14
               [&] (double& e) { e = partials[i] + e; }                    // 14
             );
           })
       | then(                                                             // 15
           [ = ](std::vector<double>&& partials) {                         // 15
             return output;                                                // 15
           });                                                             // 15
}
```

This example builds an asynchronous computation of an inclusive scan:

1. It scans a sequence of `double`s (represented as the `std::span<const double> input`) and stores the result in another sequence of `double`s (represented as `std::span<double> output`).
2. It takes a scheduler, which specifies what execution resource the scan should be launched on.
3. It also takes a `tile_count` parameter that controls the number of execution agents that will be spawned.
4. First we need to allocate temporary storage needed for the algorithm, which we'll do with a `std::vector`, `partials`. We need one `double` of temporary storage for each execution agent we create.

5. Next we'll create our initial sender with § 4.19.2 execution::just and § 4.20.1 execution::continues_on. These senders will send the temporary storage, which we've moved into the sender. The sender has a completion scheduler of `sch`, which means the next item in the chain will use `sch`.
6. Senders and sender adaptors support composition via `operator|`, similar to C++ ranges. We'll use `operator|` to attach the next piece of work, which will spawn `tile_count` execution agents using § 4.20.9 execution::bulk (see § 4.12 Most sender adaptors are pipeable for details).
7. Each agent will call a `std::invocable`, passing it two arguments. The first is the agent's index (`i`) in the § 4.20.9 execution::bulk operation, in this case a unique integer in `[0, tile_count)`. The second argument is what the input sender sent - the temporary storage.
8. We start by computing the start and end of the range of input and output elements that this agent is responsible for, based on our agent index.
9. Then we do a sequential `std::inclusive_scan` over our elements. We store the scan result for our last element, which is the sum of all of our elements, in our temporary storage `partials`.
10. After all computation in that initial § 4.20.9 execution::bulk pass has completed, every one of the spawned execution agents will have written the sum of its elements into its slot in `partials`.
11. Now we need to scan all of the values in `partials`. We'll do that with a single execution agent which will execute after the § 4.20.9 execution::bulk completes. We create that execution agent with § 4.20.2 execution::then.
12. § 4.20.2 execution::then takes an input sender and an `std::invocable` and calls the `std::invocable` with the value sent by the input sender. Inside our `std::invocable`, we call `std::inclusive_scan` on `partials`, which the input senders will send to us.
13. Then we return `partials`, which the next phase will need.
14. Finally we do another § 4.20.9 execution::bulk of the same shape as before. In this § 4.20.9 execution::bulk, we will use the scanned values in `partials` to integrate the sums from other tiles into our elements, completing the inclusive scan.
15. `async_inclusive_scan` returns a sender that sends the output `std::span<double>`. A consumer of the algorithm can chain additional work that uses the scan result. At the point at which `async_inclusive_scan` returns, the computation may not have completed. In fact, it may not have even started.

§ 1.3.3. Asynchronous dynamically-sized read

```
using namespace std::execution;

sender_of<std::size_t> auto async_read(                                  // 1
    sender_of<std::span<std::byte>> auto buffer,                         // 1
    auto handle);                                                        // 1

struct dynamic_buffer {                                                  // 3
```

```
  std::unique_ptr<std::byte[]> data;                                    // 3
  std::size_t size;                                                     // 3
};                                                                      // 3

sender_of<dynamic_buffer> auto async_read_array(auto handle) {          // 2
  return just(dynamic_buffer{})                                         // 4
       | let_value([handle] (dynamic_buffer& buf) {                     // 5
           return just(std::as_writeable_bytes(std::span(&buf.size, 1)))  // 6
                | async_read(handle)                                    // 7
                | then(                                                 // 8
                    [&buf] (std::size_t bytes_read) {                   // 9
                      assert(bytes_read == sizeof(buf.size));           // 10
                      buf.data = std::make_unique<std::byte[]>(buf.size);  // 11
                      return std::span(buf.data.get(), buf.size);       // 12
                    })
                | async_read(handle)                                    // 13
                | then(
                    [&buf] (std::size_t bytes_read) {
                      assert(bytes_read == buf.size);                   // 14
                      return std::move(buf);                            // 15
                    });
       });
}
```

This example demonstrates a common asynchronous I/O pattern - reading a payload of a dynamic size by first reading the size, then reading the number of bytes specified by the size:

1. `async_read` is a pipeable sender adaptor. It's a customization point object, but this is what it's call signature looks like. It takes a sender parameter which must send an input buffer in the form of a `std::span<std::byte>`, and a handle to an I/O context. It will asynchronously read into the input buffer, up to the size of the `std::span`. It returns a sender which will send the number of bytes read once the read completes.
2. `async_read_array` takes an I/O handle and reads a size from it, and then a buffer of that many bytes. It returns a sender that sends a `dynamic_buffer` object that owns the data that was sent.
3. `dynamic_buffer` is an aggregate struct that contains a `std::unique_ptr<std::byte[]>` and a size.
4. The first thing we do inside of `async_read_array` is create a sender that will send a new, empty `dynamic_array` object using § 4.19.2 execution::just. We can attach more work to the pipeline using `operator|` composition (see § 4.12 Most sender adaptors are pipeable for details).
5. We need the lifetime of this `dynamic_array` object to last for the entire pipeline. So, we use `let_value`, which takes an input sender and a `std::invocable` that must return a sender itself (see § 4.20.4 execution::let_* for details). `let_value` sends the value from the input sender to the `std::invocable`. Critically, the lifetime of the sent object will last until the sender returned by the `std::invocable` completes.
6. Inside of the `let_value` `std::invocable`, we have the rest of our logic. First, we want to initiate an

`async_read` of the buffer size. To do that, we need to send a `std::span` pointing to `buf.size`. We can do that with § 4.19.2 execution::just.

7. We chain the `async_read` onto the § 4.19.2 execution::just sender with `operator|`.
8. Next, we pipe a `std::invocable` that will be invoked after the `async_read` completes using § 4.20.2 execution::then.
9. That `std::invocable` gets sent the number of bytes read.
10. We need to check that the number of bytes read is what we expected.
11. Now that we have read the size of the data, we can allocate storage for it.
12. We return a `std::span<std::byte>` to the storage for the data from the `std::invocable`. This will be sent to the next recipient in the pipeline.
13. And that recipient will be another `async_read`, which will read the data.
14. Once the data has been read, in another § 4.20.2 execution::then, we confirm that we read the right number of bytes.
15. Finally, we move out of and return our `dynamic_buffer` object. It will get sent by the sender returned by `async_read_array`. We can attach more things to that sender to use the data in the buffer.

§ 1.4. Asynchronous Windows socket `recv`

To get a better feel for how this interface might be used by low-level operations see this example implementation of a cancellable `async_recv()` operation for a Windows Socket.

```
struct operation_base : WSAOVERALAPPED {
    using completion_fn = void(operation_base* op, DWORD bytesTransferred, int errorCode

    // Assume IOCP event loop will call this when this OVERLAPPED structure is dequeued.
    completion_fn* completed;
};

template<class Receiver>
struct recv_op : operation_base {
    using operation_state_concept = std::execution::operation_state_t;

    recv_op(SOCKET s, void* data, size_t len, Receiver r)
    : receiver(std::move(r))
    , sock(s) {
        this->Internal = 0;
        this->InternalHigh = 0;
        this->Offset = 0;
        this->OffsetHigh = 0;
        this->hEvent = NULL;
        this->completed = &recv_op::on_complete;
```

```
        buffer.len = len;
        buffer.buf = static_cast<CHAR*>(data);
    }

    void start() & noexcept {
        // Avoid even calling WSARecv() if operation already cancelled
        auto st = std::execution::get_stop_token(
          std::execution::get_env(receiver));
        if (st.stop_requested()) {
            std::execution::set_stopped(std::move(receiver));
            return;
        }

        // Store and cache result here in case it changes during execution
        const bool stopPossible = st.stop_possible();
        if (!stopPossible) {
            ready.store(true, std::memory_order_relaxed);
        }

        // Launch the operation
        DWORD bytesTransferred = 0;
        DWORD flags = 0;
        int result = WSARecv(sock, &buffer, 1, &bytesTransferred, &flags,
                             static_cast<WSAOVERLAPPED*>(this), NULL);
        if (result == SOCKET_ERROR) {
            int errorCode = WSAGetLastError();
            if (errorCode != WSA_IO_PENDING) {
                if (errorCode == WSA_OPERATION_ABORTED) {
                    std::execution::set_stopped(std::move(receiver));
                } else {
                    std::execution::set_error(std::move(receiver),
                                              std::error_code(errorCode, std::system_cat
                }
                return;
            }
        } else {
            // Completed synchronously (assuming FILE_SKIP_COMPLETION_PORT_ON_SUCCESS has
            execution::set_value(std::move(receiver), bytesTransferred);
            return;
        }

        // If we get here then operation has launched successfully and will complete asyn
        // May be completing concurrently on another thread already.
        if (stopPossible) {
            // Register the stop callback
            stopCallback.emplace(std::move(st), cancel_cb{*this});

            // Mark as 'completed'
```

```
        if (ready.load(std::memory_order_acquire) ||
            ready.exchange(true, std::memory_order_acq_rel)) {
            // Already completed on another thread
            stopCallback.reset();

            BOOL ok = WSAGetOverlappedResult(sock, (WSAOVERLAPPED*)this, &bytesTrans
            if (ok) {
                std::execution::set_value(std::move(receiver), bytesTransferred);
            } else {
                int errorCode = WSAGetLastError();
                std::execution::set_error(std::move(receiver),
                                          std::error_code(errorCode, std::system_cat
            }
        }
    }
}

struct cancel_cb {
    recv_op& op;

    void operator()() noexcept {
        CancelIoEx((HANDLE)op.sock, (OVERLAPPED*)(WSAOVERLAPPED*)&op);
    }
};

static void on_complete(operation_base* op, DWORD bytesTransferred, int errorCode) n
    recv_op& self = *static_cast<recv_op*>(op);

    if (self.ready.load(std::memory_order_acquire) ||
        self.ready.exchange(true, std::memory_order_acq_rel)) {
        // Unsubscribe any stop callback so we know that CancelIoEx() is not accessi
        // any more
        self.stopCallback.reset();

        if (errorCode == 0) {
            std::execution::set_value(std::move(self.receiver), bytesTransferred);
        } else {
            std::execution::set_error(std::move(self.receiver),
                                      std::error_code(errorCode, std::system_categor
        }
    }
}

using stop_callback_t = stop_callback_of_t<stop_token_of_t<env_of_t<Receiver>>, canc

Receiver receiver;
SOCKET sock;
WSABUF buffer;
```

```
    std::optional<stop_callback_t> stopCallback;
    std::atomic<bool> ready{false};
};

struct recv_sender {
    using sender_concept = std::execution::sender_t;
    SOCKET sock;
    void* data;
    size_t len;

    template<class Receiver>
    recv_op<Receiver> connect(Receiver r) const {
        return recv_op<Receiver>{sock, data, len, std::move(r)};
    }
};

recv_sender async_recv(SOCKET s, void* data, size_t len) {
    return recv_sender{s, data, len};
}
```

1.4.1. More end-user examples

1.4.1.1. Sudoku solver

This example comes from Kirk Shoop, who ported an example from TBB's documentation to sender/receiver in his fork of the libunifex repo. It is a Sudoku solver that uses a configurable number of threads to explore the search space for solutions.

The sender/receiver-based Sudoku solver can be found here. Some things that are worth noting about Kirk's solution:

1. Although it schedules asynchronous work onto a thread pool, and each unit of work will schedule more work, its use of structured concurrency patterns make reference counting unnecessary. The solution does not make use of `shared_ptr`.
2. In addition to eliminating the need for reference counting, the use of structured concurrency makes it easy to ensure that resources are cleaned up on all code paths. In contrast, the TBB example that inspired this one leaks memory.

For comparison, the TBB-based Sudoku solver can be found here.

1.4.1.2. File copy

This example also comes from Kirk Shoop which uses sender/receiver to recursively copy the files a directory

tree. It demonstrates how sender/receiver can be used to do IO, using a scheduler that schedules work on Linux's io_uring.

As with the Sudoku example, this example obviates the need for reference counting by employing structured concurrency. It uses iteration with an upper limit to avoid having too many open file handles.

You can find the example here.

§ 1.4.1.3. Echo server

Dietmar Kuehl has proposed networking APIs that use the sender/receiver abstraction (see P2762). He has implemented an echo server as a demo. His echo server code can be found here.

Below, I show the part of the echo server code. This code is executed for each client that connects to the echo server. In a loop, it reads input from a socket and echos the input back to the same socket. All of this, including the loop, is implemented with generic async algorithms.

```
outstanding.start(
    EX::repeat_effect_until(
          EX::let_value(
              NN::async_read_some(ptr->d_socket,
                                  context.scheduler(),
                                  NN::buffer(ptr->d_buffer))
        | EX::then([ptr](::std::size_t n){
            ::std::cout << "read='" << ::std::string_view(ptr->d_buffer, n) << "'\n";
            ptr->d_done = n == 0;
            return n;
        }),
          [&context, ptr](::std::size_t n){
            return NN::async_write_some(ptr->d_socket,
                                        context.scheduler(),
                                        NN::buffer(ptr->d_buffer, n));
          })
        | EX::then([](auto&&...){})
        , [owner = ::std::move(owner)]{ return owner->d_done; }
    )
);
```

In this code, `NN::async_read_some` and `NN::async_write_some` are asynchronous socket-based networking APIs that return senders. `EX::repeat_effect_until`, `EX::let_value`, and `EX::then` are fully generic sender adaptor algorithms that accept and return senders.

This is a good example of seamless composition of async IO functions with non-IO operations. And by composing the senders in this structured way, all the state for the composite operation -- the `repeat_effect_until` expression and all its child operations -- is stored altogether in a single object.

§ 1.5. Examples: Algorithms

In this section we show a few simple sender/receiver-based algorithm implementations.

§ 1.5.1. then

```
namespace stdexec = std::execution;

template <class R, class F>
class _then_receiver : public R {
  F f_;

 public:
  _then_receiver(R r, F f) : R(std::move(r)), f_(std::move(f)) {}

  // Customize set_value by invoking the callable and passing the result to
  // the inner receiver
  template <class... As>
    requires std::invocable<F, As...>
  void set_value(As&&... as) && noexcept {
    try {
      stdexec::set_value(std::move(*this).base(), std::invoke((F&&) f_, (As&&) as...));
    } catch(...) {
      stdexec::set_error(std::move(*this).base(), std::current_exception());
    }
  }
};

template <stdexec::sender S, class F>
struct _then_sender {
  using sender_concept = stdexec::sender_t;
  S s_;
  F f_;

  template <class... Args>
    using _set_value_t = stdexec::completion_signatures<
      stdexec::set_value_t(std::invoke_result_t<F, Args...>)>;

  using _except_ptr_sig =
    stdexec::completion_signatures<stdexec::set_error_t(std::exception_ptr)>;

  // Compute the completion signatures
  template <class Env>
  auto get_completion_signatures(Env&& env) && noexcept
    -> stdexec::transform_completion_signatures_of<
        S, Env, _except_ptr_sig, _set_value_t> {
```

```
    return {};
  }

  // Connect:
  template <stdexec::receiver R>
  auto connect(R r) && -> stdexec::connect_result_t<S, _then_receiver<R, F>> {
    return stdexec::connect(
      (S&&) s_, _then_receiver{(R&&) r, (F&&) f_});
  }

  decltype(auto) get_env() const noexcept {
    return get_env(s_);
  }
};

template <stdexec::sender S, class F>
stdexec::sender auto then(S s, F f) {
  return _then_sender<S, F>{(S&&) s, (F&&) f};
}
```

This code builds a `then` algorithm that transforms the value(s) from the input sender with a transformation function. The result of the transformation becomes the new value. The other receiver functions (`set_error` and `set_stopped`), as well as all receiver queries, are passed through unchanged.

In detail, it does the following:

1. Defines a receiver in terms of receiver and an invocable that:

   — Defines a constrained `set_value` member function for transforming the value channel.

   — Delegates `set_error` and `set_stopped` to the inner receiver.

2. Defines a sender that aggregates another sender and the invocable, which defines a `connect` member function that wraps the incoming receiver in the receiver from (1) and passes it and the incoming sender to `std::execution::connect`, returning the result. It also defines a `get_completion_signatures` member function that declares the sender's completion signatures when executed within a particular environment.

§ 1.5.2. `retry`

```
using namespace std;
namespace stdexec = execution;

template<class From, class To>
concept _decays_to = same_as<decay_t<From>, To>;
```

```
// _conv needed so we can emplace construct non-movable types into
// a std::optional.
template<invocable F>
struct _conv {
  F f_;

  static_assert(is_nothrow_move_constructible_v<F>);
  explicit _conv(F f) noexcept : f_((F&&) f) {}

  operator invoke_result_t<F>() && {
    return ((F&&) f_)();
  }
};

template<class S, class R>
struct _retry_op;

// pass through all customizations except set_error, which retries
// the operation.
template<class S, class R>
struct _retry_receiver {
  _retry_op<S, R>* o_;

  void set_value(auto&&... as) && noexcept {
    stdexec::set_value(std::move(o_->r_), (decltype(as)&&) as...);
  }

  void set_error(auto&&) && noexcept {
    o_->_retry(); // This causes the op to be retried
  }

  void set_stopped() && noexcept {
    stdexec::set_stopped(std::move(o_->r_));
  }

  decltype(auto) get_env() const noexcept {
    return get_env(o_->r_);
  }
};

// Hold the nested operation state in an optional so we can
// re-construct and re-start it if the operation fails.
template<class S, class R>
struct _retry_op {
  using operation_state_concept = stdexec::operation_state_t;
  using _child_op_t =
    stdexec::connect_result_t<S&, _retry_receiver<S, R>>;
```

```
  S s_;
  R r_;
  optional<_child_op_t> o_;

  _op(_op&&) = delete;
  _op(S s, R r)
    : s_(std::move(s)), r_(std::move(r)), o_{_connect()} {}

  auto _connect() noexcept {
    return _conv{[this] {
      return stdexec::connect(s_, _retry_receiver<S, R>{this});
    }};
  }

  void _retry() noexcept {
    try {
      o_.emplace(_connect()); // potentially-throwing
      stdexec::start(*o_);
    } catch(...) {
      stdexec::set_error(std::move(r_), std::current_exception());
    }
  }

  void start() & noexcept {
    stdexec::start(*o_);
  }
};

// Helpers for computing the <code data-opaque bs-autolink-syntax='`then`'>then</code> se
template <class... Ts>
  using _value_t =
    stdexec::completion_signatures<stdexec::set_value_t(Ts...)>;

template <class>
  using _error_t = stdexec::completion_signatures<>;

using _except_sig =
  stdexec::completion_signatures<stdexec::set_error_t(std::exception_ptr)>;

template<class S>
struct _retry_sender {
  using sender_concept = stdexec::sender_t;
  S s_;
  explicit _retry_sender(S s) : s_(std::move(s)) {}

  // Declare the signatures with which this sender can complete
  template <class Env>
    using _compl_sigs =
```

```
      stdexec::transform_completion_signatures_of<
        S&, Env, _except_sig, _value_t, _error_t>;

  template <class Env>
  auto get_completion_signatures(Env&&) const noexcept -> _compl_sigs<Env> {
    return {};
  }

  template <stdexec::receiver R>
    requires stdexec::sender_to<S&, _retry_receiver<S, R>>
  _retry_op<S, R> connect(R r) && {
    return {std::move(s_), std::move(r)};
  }

  decltype(auto) get_env() const noexcept {
    return get_env(s_);
  }
};

template <stdexec::sender S>
stdexec::sender auto retry(S s) {
  return _retry_sender{std::move(s)};
}
```

The `retry` algorithm takes a multi-shot sender and causes it to repeat on error, passing through values and stopped signals. Each time the input sender is restarted, a new receiver is connected and the resulting operation state is stored in an `optional`, which allows us to reinitialize it multiple times.

This example does the following:

1. Defines a `_conv` utility that takes advantage of C++17's guaranteed copy elision to emplace a non-movable type in a `std::optional`.
2. Defines a `_retry_receiver` that holds a pointer back to the operation state. It passes all customizations through unmodified to the inner receiver owned by the operation state except for `set_error`, which causes a `_retry()` function to be called instead.
3. Defines an operation state that aggregates the input sender and receiver, and declares storage for the nested operation state in an `optional`. Constructing the operation state constructs a `_retry_receiver` with a pointer to the (under construction) operation state and uses it to connect to the input sender.
4. Starting the operation state dispatches to `start` on the inner operation state.
5. The `_retry()` function reinitializes the inner operation state by connecting the sender to a new receiver, holding a pointer back to the outer operation state as before.
6. After reinitializing the inner operation state, `_retry()` calls `start` on it, causing the failed operation to be rescheduled.
7. Defines a `_retry_sender` that implements a `connect` member function to return an operation state con-

structed from the passed-in sender and receiver.

8. `_retry_sender` also implements a `get_completion_signatures` member function to describe the ways this sender may complete when executed in a particular execution resource.

§ 1.6. Examples: Schedulers

In this section we look at some schedulers of varying complexity.

§ 1.6.1. Inline scheduler

```
namespace stdexec = std::execution;

class inline_scheduler {
  template <class R>
  struct _op {
    using operation_state_concept = operation_state_t;
    R rec_;

    void start() & noexcept {
      stdexec::set_value(std::move(rec_));
    }
  };

  struct _env {
    template <class Tag>
    inline_scheduler query(stdexec::get_completion_scheduler_t<Tag>) const noexcept {
      return {};
    }
  };

  struct _sender {
    using sender_concept = stdexec::sender_t;
    using _compl_sigs = stdexec::completion_signatures<stdexec::set_value_t()>;
    using completion_signatures = _compl_sigs;

    template <stdexec::receiver_of<_compl_sigs> R>
    _op<R> connect(R rec) noexcept(std::is_nothrow_move_constructible_v<R>) {
      return {std::move(rec)};
    }

    _env get_env() const noexcept {
      return {};
    }
  };
```

```
 public:
  inline_scheduler() = default;

  _sender schedule() const noexcept {
    return {};
  }

  bool operator==(const inline_scheduler&) const noexcept = default;
};
```

The inline scheduler is a trivial scheduler that completes immediately and synchronously on the thread that calls `std::execution::start` on the operation state produced by its sender. In other words, `start(connect(schedule(inline_scheduler()), receiver))` is just a fancy way of saying `set_value(receiver)`, with the exception of the fact that `start` wants to be passed an lvalue.

Although not a particularly useful scheduler, it serves to illustrate the basics of implementing one. The `inline_scheduler`:

1. Customizes `execution::schedule` to return an instance of the sender type `_sender`.
2. The `_sender` type models the `sender` concept and provides the metadata needed to describe it as a sender of no values and that never calls `set_error` or `set_stopped`. This metadata is provided with the help of the `execution::completion_signatures` utility.
3. The `_sender` type customizes `execution::connect` to accept a receiver of no values. It returns an instance of type `_op` that holds the receiver by value.
4. The operation state customizes `std::execution::start` to call `std::execution::set_value` on the receiver.

§ 1.6.2. Single thread scheduler

This example shows how to create a scheduler for an execution resource that consists of a single thread. It is implemented in terms of a lower-level execution resource called `std::execution::run_loop`.

```
class single_thread_context {
  std::execution::run_loop loop_;
  std::thread thread_;

public:
  single_thread_context()
    : loop_()
    , thread_([this] { loop_.run(); })
  {}
  single_thread_context(single_thread_context&&) = delete;
```

```
  ~single_thread_context() {
    loop_.finish();
    thread_.join();
  }

  auto get_scheduler() noexcept {
    return loop_.get_scheduler();
  }

  std::thread::id get_thread_id() const noexcept {
    return thread_.get_id();
  }
};
```

The `single_thread_context` owns an event loop and a thread to drive it. In the destructor, it tells the event loop to finish up what it's doing and then joins the thread, blocking for the event loop to drain.

The interesting bits are in the `execution::run_loop` context implementation. It is slightly too long to include here, so we only provide a reference to it, but there is one noteworthy detail about its implementation: It uses space in its operation states to build an intrusive linked list of work items. In structured concurrency patterns, the operation states of nested operations compose statically, and in an algorithm like `this_thread::sync_wait`, the composite operation state lives on the stack for the duration of the operation. The end result is that work can be scheduled onto this thread with zero allocations.

§ 1.7. Examples: Server theme

In this section we look at some examples of how one would use senders to implement an HTTP server. The examples ignore the low-level details of the HTTP server and looks at how senders can be combined to achieve the goals of the project.

General application context:

- server application that processes images
- execution resources:
  - 1 dedicated thread for network I/O
  - N worker threads used for CPU-intensive work
  - M threads for auxiliary I/O
  - optional GPU context that may be used on some types of servers
- all parts of the applications can be asynchronous
- no locks shall be used in user code

§ 1.7.1. Composability with `execution::let_*`

Example context:

— we are looking at the flow of processing an HTTP request and sending back the response.

— show how one can break the (slightly complex) flow into steps with `execution::let_*` functions.

— different phases of processing HTTP requests are broken down into separate concerns.

— each part of the processing might use different execution resources (details not shown in this example).

— error handling is generic, regardless which component fails; we always send the right response to the clients.

Goals:

— show how one can break more complex flows into steps with let_* functions.

— exemplify the use of `let_value`, `let_error`, `let_stopped`, and `just` algorithms.

```
namespace stdexec = std::execution;

// Returns a sender that yields an http_request object for an incoming request
stdexec::sender auto schedule_request_start(read_requests_ctx ctx) {...}

// Sends a response back to the client; yields a void signal on success
stdexec::sender auto send_response(const http_response& resp) {...}

// Validate that the HTTP request is well-formed; forwards the request on success
stdexec::sender auto validate_request(const http_request& req) {...}

// Handle the request; main application logic
stdexec::sender auto handle_request(const http_request& req) {
  //...
  return stdexec::just(http_response{200, result_body});
}

// Transforms server errors into responses to be sent to the client
stdexec::sender auto error_to_response(std::exception_ptr err) {
  try {
    std::rethrow_exception(err);
  } catch (const std::invalid_argument& e) {
    return stdexec::just(http_response{404, e.what()});
  } catch (const std::exception& e) {
    return stdexec::just(http_response{500, e.what()});
  } catch (...) {
    return stdexec::just(http_response{500, "Unknown server error"});
  }
}
```

```
// Transforms cancellation of the server into responses to be sent to the client
stdexec::sender auto stopped_to_response() {
  return stdexec::just(http_response{503, "Service temporarily unavailable"});
}

//...

// The whole flow for transforming incoming requests into responses
stdexec::sender auto snd =
    // get a sender when a new request comes
    schedule_request_start(the_read_requests_ctx)
    // make sure the request is valid; throw if not
    | stdexec::let_value(validate_request)
    // process the request in a function that may be using a different execution resource
    | stdexec::let_value(handle_request)
    // If there are errors transform them into proper responses
    | stdexec::let_error(error_to_response)
    // If the flow is cancelled, send back a proper response
    | stdexec::let_stopped(stopped_to_response)
    // write the result back to the client
    | stdexec::let_value(send_response)
    // done
    ;

// execute the whole flow asynchronously
stdexec::start_detached(std::move(snd));
```

The example shows how one can separate out the concerns for interpreting requests, validating requests, running the main logic for handling the request, generating error responses, handling cancellation and sending the response back to the client. They are all different phases in the application, and can be joined together with the `let_*` functions.

All our functions return `execution::sender` objects, so that they can all generate success, failure and cancellation paths. For example, regardless where an error is generated (reading request, validating request or handling the response), we would have one common block to handle the error, and following error flows is easy.

Also, because of using `execution::sender` objects at any step, we might expect any of these steps to be completely asynchronous; the overall flow doesn't care. Regardless of the execution resource in which the steps, or part of the steps are executed in, the flow is still the same.

§ 1.7.2. Moving between execution resources with `execution::starts_on` and `execution::continues_on`

Example context:

— reading data from the socket before processing the request

— reading of the data is done on the I/O context

— no processing of the data needs to be done on the I/O context

Goals:

— show how one can change the execution resource

— exemplify the use of `starts_on` and `continues_on` algorithms

```
namespace stdexec = std::execution;

size_t legacy_read_from_socket(int sock, char* buffer, size_t buffer_len);
void process_read_data(const char* read_data, size_t read_len);
//...

// A sender that just calls the legacy read function
auto snd_read = stdexec::just(sock, buf, buf_len)
              | stdexec::then(legacy_read_from_socket);

// The entire flow
auto snd =
    // start by reading data on the I/O thread
    stdexec::starts_on(io_sched, std::move(snd_read))
    // do the processing on the worker threads pool
    | stdexec::continues_on(work_sched)
    // process the incoming data (on worker threads)
    | stdexec::then([buf](int read_len) { process_read_data(buf, read_len); })
    // done
    ;

// execute the whole flow asynchronously
stdexec::start_detached(std::move(snd));
```

The example assume that we need to wrap some legacy code of reading sockets, and handle execution resource switching. (This style of reading from socket may not be the most efficient one, but it's working for our purposes.) For performance reasons, the reading from the socket needs to be done on the I/O thread, and all the processing needs to happen on a work-specific execution resource (i.e., thread pool).

Calling `execution::starts_on` will ensure that the given sender will be started on the given scheduler. In our example, `snd_read` is going to be started on the I/O scheduler. This sender will just call the legacy code.

The completion-signal will be issued in the I/O execution resource, so we have to move it to the work thread pool. This is achieved with the help of the `execution::continues_on` algorithm. The rest of the processing (in our case, the last call to `then`) will happen in the work thread pool.

The reader should notice the difference between `execution::starts_on` and `execution::continues_on`. The `execution::starts_on` algorithm will ensure that the given sender will start in the specified context, and

doesn't care where the completion-signal for that sender is sent. The `execution::continues_on` algorithm will not care where the given sender is going to be started, but will ensure that the completion-signal of will be transferred to the given context.

§ 1.8. Design changes from P0443

1. The `executor` concept has been removed and all of its proposed functionality is now based on schedulers and senders, as per SG1 direction.
2. Properties are not included in this paper. We see them as a possible future extension, if the committee gets more comfortable with them.
3. Senders now advertise what scheduler, if any, their evaluation will complete on.
4. The places of execution of user code in P0443 weren't precisely defined, whereas they are in this paper. See § 4.5 Senders can propagate completion schedulers.
5. P0443 did not propose a suite of sender algorithms necessary for writing sender code; this paper does. See § 4.19 User-facing sender factories, § 4.20 User-facing sender adaptors, and § 4.21 User-facing sender consumers.
6. P0443 did not specify the semantics of variously qualified `connect` overloads; this paper does. See § 4.7 Senders can be either multi-shot or single-shot.
7. This paper extends the sender traits/typed sender design to support typed senders whose value/error types depend on type information provided late via the receiver.
8. Support for untyped senders is dropped; the `typed_sender` concept is renamed `sender`; `sender_traits` is replaced with `completion_signatures_of_t`.
9. Specific type erasure facilities are omitted, as per LEWG direction. Type erasure facilities can be built on top of this proposal, as discussed in § 5.9 Customization points.
10. A specific thread pool implementation is omitted, as per LEWG direction.
11. Some additional utilities are added:

    — **`run_loop`**: An execution resource that provides a multi-producer, single-consumer, first-in-first-out work queue.

    — **`completion_signatures`** and **`transform_completion_signatures`**: Utilities for describing the ways in which a sender can complete in a declarative syntax.

§ 1.9. Prior art

This proposal builds upon and learns from years of prior art with asynchronous and parallel programming frameworks in C++. In this section, we discuss async abstractions that have previously been suggested as a possible basis for asynchronous algorithms and why they fall short.

§ 1.9.1. Futures

A future is a handle to work that has already been scheduled for execution. It is one end of a communication channel; the other end is a promise, used to receive the result from the concurrent operation and to communicate it to the future.

Futures, as traditionally realized, require the dynamic allocation and management of a shared state, synchronization, and typically type-erasure of work and continuation. Many of these costs are inherent in the nature of "future" as a handle to work that is already scheduled for execution. These expenses rule out the future abstraction for many uses and makes it a poor choice for a basis of a generic mechanism.

§ 1.9.2. Coroutines

C++20 coroutines are frequently suggested as a basis for asynchronous algorithms. It's fair to ask why, if we added coroutines to C++, are we suggesting the addition of a library-based abstraction for asynchrony. Certainly, coroutines come with huge syntactic and semantic advantages over the alternatives.

Although coroutines are lighter weight than futures, coroutines suffer many of the same problems. Since they typically start suspended, they can avoid synchronizing the chaining of dependent work. However in many cases, coroutine frames require an unavoidable dynamic allocation and indirect function calls. This is done to hide the layout of the coroutine frame from the C++ type system, which in turn makes possible the separate compilation of coroutines and certain compiler optimizations, such as optimization of the coroutine frame size.

Those advantages come at a cost, though. Because of the dynamic allocation of coroutine frames, coroutines in embedded or heterogeneous environments, which often lack support for dynamic allocation, require great attention to detail. And the allocations and indirections tend to complicate the job of the inliner, often resulting in sub-optimal codegen.

The coroutine language feature mitigates these shortcomings somewhat with the HALO optimization *Halo: coroutine Heap Allocation eLision Optimization: the joint response*, which leverages existing compiler optimizations such as allocation elision and devirtualization to inline the coroutine, completely eliminating the runtime overhead. However, HALO requires a sophisiticated compiler, and a fair number of stars need to align for the optimization to kick in. In our experience, more often than not in real-world code today's compilers are not able to inline the coroutine, resulting in allocations and indirections in the generated code.

In a suite of generic async algorithms that are expected to be callable from hot code paths, the extra allocations and indirections are a deal-breaker. It is for these reasons that we consider coroutines a poor choice for a basis of all standard async.

§ 1.9.3. Callbacks

Callbacks are the oldest, simplest, most powerful, and most efficient mechanism for creating chains of work, but suffer problems of their own. Callbacks must propagate either errors or values. This simple requirement yields many different interface possibilities. The lack of a standard callback shape obstructs generic design.

Additionally, few of these possibilities accommodate cancellation signals when the user requests upstream work to stop and clean up.

§ 1.10. Field experience

§ 1.10.1. libunifex

This proposal draws heavily from our field experience with libunifex. Libunifex implements all of the concepts and customization points defined in this paper (with slight variations -- the design of P2300 has evolved due to LEWG feedback), many of this paper's algorithms (some under different names), and much more besides.

Libunifex has several concrete schedulers in addition to the `run_loop` suggested here (where it is called `manual_event_loop`). It has schedulers that dispatch efficiently to epoll and io_uring on Linux and the Windows Thread Pool on Windows.

In addition to the proposed interfaces and the additional schedulers, it has several important extensions to the facilities described in this paper, which demonstrate directions in which these abstractions may be evolved over time, including:

— Timed schedulers, which permit scheduling work on an execution resource at a particular time or after a particular duration has elapsed. In addition, it provides time-based algorithms.

— File I/O schedulers, which permit filesystem I/O to be scheduled.

— Two complementary abstractions for streams (asynchronous ranges), and a set of stream-based algorithms.

Libunifex has seen heavy production use at Meta. An employee summarizes it as follows:

> As of June, 2023, Unifex is still used in production at Meta. It's used to express the asynchrony in rsys, and is therefore serving video calling to billions of people every month on Meta's social networking apps on iOS, Android, Windows, and macOS. It's also serving the Virtual Desktop experience on Oculus Quest devices, and some internal uses that run on Linux.
>
> One team at Meta has migrated from `folly::Future` to `unifex::task` and seen significant developer efficiency improvements. Coroutines are easier to understand than chained futures so the team was able to meet requirements for certain constrained environments that would have been too complicated to maintain with futures.
>
> In all the cases mentioned above, developers mix-and-match between the sender algorithms in Unifex and Unifex's coroutine type, `unifex::task`. We also rely on `unifex::task`'s scheduler affinity to minimize surprise when programming with coroutines.

§ 1.10.2. stdexec

stdexec is the reference implementation of this proposal. It is a complete implementation, written from the specification in this paper, and is current with \R8.

The original purpose of stdexec was to help find specification bugs and to harden the wording of the proposal, but it has since become one of NVIDIA's core C++ libraries for high-performance computing. In addition to the facilities proposed in this paper, stdexec has schedulers for CUDA, Intel TBB, and MacOS. Like libunifex, its scope has also expanded to include a streaming abstraction and stream algorithms, and time-based schedulers and algorithms.

The stdexec project has seen lots of community interest and contributions. At the time of writing (March, 2024), the GitHub repository has 1.2k stars, 130 forks, and 50 contributors.

stdexec is fit for broad use and for ultimate contribution to libc++.

§ 1.10.3. Other implementations

The authors are aware of a number of other implementations of sender/receiver from this paper. These are presented here in perceived order of maturity and field experience.

— ***HPX - The C++ Standard Library for Parallelism and Concurrency***

HPX is a general purpose C++ runtime system for parallel and distributed applications that has been under active development since 2007. HPX exposes a uniform, standards-oriented API, and keeps abreast of the latest standards and proposals. It is used in a wide variety of high-performance applications.

The sender/receiver implementation in HPX has been under active development since May 2020. It is used to erase the overhead of futures and to make it possible to write efficient generic asynchronous algorithms that are agnostic to their execution resource. In HPX, algorithms can migrate execution between execution resources, even to GPUs and back, using a uniform standard interface with sender/receiver.

Far and away, the HPX team has the greatest usage experience outside Facebook. Mikael Simberg summarizes the experience as follows:

> Summarizing, for us the major benefits of sender/receiver compared to the old model are:
>
> 1. Proper hooks for transitioning between execution resources.
> 2. The adaptors. Things like `let_value` are really nice additions.
> 3. Separation of the error channel from the value channel (also cancellation, but we don't have much use for it at the moment). Even from a teaching perspective having to explain that the future `f2` in the continuation will always be ready here `f1.then([](future<T> f2) {...})` is enough of a reason to separate the channels. All the other obvious reasons apply as well of course.
> 4. For futures we have a thing called `hpx::dataflow` which is an optimized version of `when_all(...).then(...)` which avoids intermediate allocations. With the sender/receiver `when_all(...) | then(...)` we get that "for free".

— **kuhllib by Dietmar Kuehl**

This is a prototype Standard Template Library with an implementation of sender/receiver that has been under development since May, 2021. It is significant mostly for its support for sender/receiver-based networking interfaces.

Here, Dietmar Kuehl speaks about the perceived complexity of sender/receiver:

> ... and, also similar to STL: as I had tried to do things in that space before I recognize sender/receivers as being maybe complicated in one way but a huge simplification in another one: like with STL I think those who use it will benefit - if not from the algorithm from the clarity of abstraction: the separation of concerns of STL (the algorithm being detached from the details of the sequence representation) is a major leap. Here it is rather similar: the separation of the asynchronous algorithm from the details of execution. Sure, there is some glue to tie things back together but each of them is simpler than the combined result.

Elsewhere, he said:

> ... to me it feels like sender/receivers are like iterators when STL emerged: they are different from what everybody did in that space. However, everything people are already doing in that space isn't right.

Kuehl also has experience teaching sender/receiver at Bloomberg. About that experience he says:

> When I asked [my students] specifically about how complex they consider the sender/receiver stuff the feedback was quite unanimous that the sender/receiver parts aren't trivial but not what contributes to the complexity.

— **C++ Bare Metal Senders and Receivers from Intel**

This is a prototype implementation of sender/receiver by Intel that has been under development since August, 2023. It is significant mostly for its support for bare metal (no operating system) and embedded systems, a domain for which senders are particularly well-suited due to their very low dynamic memory requirements.

§ 1.10.4. Inspirations

This proposal also draws heavily from our experience with Thrust and Agency. It is also inspired by the needs of countless other C++ frameworks for asynchrony, parallelism, and concurrency, including:

— HPX

— Folly

— stlab

§ 2. Revision history

§ 2.1. R10

The changes since R9 are as follows:

Fixes:

— Fixed `connect` and `get_completion_signatures` to use `transform_sender`, as "*Sender Algorithm Customization*" proposed (but failed) to do. See "*Fixing Lazy Sender Algorithm Customization*" for details.

— `ensure_started`, `start_detached`, `execute`, and `execute_may_block_caller` are removed from the proposal. They are to be replaced with safer and more structured APIs by "*async_scope — Creating scopes for non-sequential concurrency*". See "*remove ensure_started and start_detached from P2300*" for details.

— Fixed a logic error in the specification of `split` that could have caused a receiver to be completed twice in some cases.

— Fixed `stopped_as_optional` to handle the case where the child sender completes with more than one value, in which case the `stopped_as_optional` sender completes with an `optional` of a `tuple` of the values.

— The `queryable`, `stoppable_source`, and `stoppable_callback_for` concepts have been made exposition-only.

Enhancements:

— The `operation_state` concept no longer requires that operation states model `queryable`.

— The `get_delegatee_scheduler` query has been renamed to `get_delegation_scheduler`.

— The `read` environment has been renamed to `read_env`.

— The nullary forms of the queries which returned instances of the `read_env` sender have been removed. That is, `get_scheduler()` is no longer another way to spell `read_env(get_scheduler)`. Same for the other queries.

— A feature test macro has been added: `__cpp_lib_senders`.

— `transfer` has been renamed to `continues_on`. `on` has been renamed to `starts_on`. A new `on` algorithm has been added that is a combination of `starts_on` and `continues_on` for performing work on a different context and automatically transitioning back to the starting one. See "*Reconsidering the std::execution::on algorithm*" for details.

— An exposition-only *`simple-allocator`* concept is added to the Library introduction ([library]), and the specification of the `get_allocator` query is expressed in terms of it.

— An exposition-only *`write-env`* sender adaptor has been added for use in the implementation of the new `on` algorithm.

§ 2.2. R9

The changes since R8 are as follows:

Fixes:

— The `tag_invoke` mechanism has been replaced with member functions for customizations as per "*Member customization points for Senders and Receivers*".

— Per guidance from LWG and LEWG, `receiver_adaptor` has been removed.

— The `receiver` concept is tweaked to require that receiver types are not `final`. Without `receiver_adaptor` and `tag_invoke`, receiver adaptors are easily written using implementation inheritance.

— `std::tag_t` is made exposition-only.

— The types `in_place_stop_token`, `in_place_stop_source`, and `in_place_stop_callback` are renamed to `inplace_stop_token`, `inplace_stop_source`, and `inplace_stop_callback`, respectively.

Enhancements:

— The specification of the `sync_wait` algorithm has been updated for clarity.

— The specification of all the stop token, source, and callback types have been re-expressed in terms of shared concepts.

— Declarations are shown in their proper namespaces.

— Editorial changes have been made to clarify what text is added, what is removed, and what is an editorial note.

— The section numbers of the proposed wording now match the section numbers in the working draft of the

C++ standard.

§ 2.3. R8

The changes since R7 are as follows:

Fixes:

— `get_env(obj)` is required to be nothrow.

— `get_env` and the associated environment utilities are moved back into `std::execution` from `std::`.

— `make_completion_signatures` is renamed `transform_completion_signatures_of` and is expressed in terms of the new `transform_completion_signatures`, which takes an input set of completion signatures instead of a sender and an environment.

— Add a requirement on queryable objects that if `tag_invoke(query, env, args...)` is well-formed, then `query(env, args...)` is expression-equivalent to it. This is necessary to properly specify how to join two environments in the presence of queries that have defaults.

— The `sender_in<Sndr, Env>` concept requires that `E` satisfies `queryable`.

— Senders of more than one value are now `co_await`-able in coroutines, the result of which is a `std::tuple` of the values (which is suitable as the initializer of a structured binding).

Enhancements:

— The exposition-only class template *`basic-sender`* is greatly enhanced, and the sender algorithms are respecified in term of it.

— `enable_sender` and `enable_receiver` traits now have default implementations that look for nested `sender_concept` and `receiver_concept` types, respectively.

§ 2.4. R7

The changes since R6 are as follows:

Fixes:

— Make it valid to pass non-variadic templates to the exposition-only alias template *`gather-signatures`*, fixing the definitions of `value_types_of_t`, `error_types_of_t`, and the exposition-only alias template *`sync-wait-result-type`*.

— Removed the query forwarding from `receiver_adaptor` that was inadvertantly left over from a previous edit.

— When adapting a sender to an awaitable with `as_awaitable`, the sender's value result datum is decayed before being stored in the exposition-only `variant`.

— Correctly specify the completion signatures of the `schedule_from` algorithm.

— The `sender_of` concept no longer distinguishes between a sender of a type `T` and a sender of a type `T&&`.

— The `just` and `just_error` sender factories now reject C-style arrays instead of silently decaying them to pointers.

Enhancements:

— The `sender` and `receiver` concepts get explicit opt-in traits called `enable_sender` and `enable_receiver`, respectively. The traits have default implementations that look for nested `is_sender` and `is_receiver` types, respectively.

— `get_attrs` is removed and `get_env` is used in its place.

— The exposition-only type *`empty-env`* is made normative and is renamed `empty_env`.

— `get_env` gets a fall-back implementation that simply returns `empty_env{}` if a `tag_invoke` overload is not found.

— `get_env` is required to be insensitive to the cvref-qualification of its argument.

— `get_env`, `empty_env`, and `env_of_t` are moved into the `std::` namespace.

— Add a new subclause describing the async programming model of senders in abstract terms. See § 34.3 Asynchronous operations [async.ops].

§ 2.5. R6

The changes since R5 are as follows:

Fixes:

— Fix typo in the specification of `in_place_stop_source` about the relative lifetimes of the tokens and the source that produced them.

— `get_completion_signatures` tests for awaitability with a promise type similar to the one used by `connect` for the sake of consistency.

— A coroutine promise type is an environment provider (that is, it implements `get_env()`) rather than being directly queryable. The previous draft was inconsistent about that.

Enhancements:

— Sender queries are moved into a separate queryable "attributes" object that is accessed by passing the sender to `get_attrs()` (see below). The `sender` concept is reexpressed to require `get_attrs()` and separated from a new `sender_in<Snd, Env>` concept for checking whether a type is a sender within a particular execution environment.

— The placeholder types `no_env` and `dependent_completion_signatures<>` are no longer needed and are dropped.

— `ensure_started` and `split` are changed to persist the result of calling `get_attrs()` on the input sender.

— Reorder constraints of the `scheduler` and `receiver` concepts to avoid constraint recursion when used in tandem with poorly-constrained, implicitly convertible types.

— Re-express the `sender_of` concept to be more ergonomic and general.

— Make the specification of the alias templates `value_types_of_t` and `error_types_of_t`, and the variable template `sends_done` more concise by expressing them in terms of a new exposition-only alias template *`gather-signatures`*.

§ 2.5.1. Environments and attributes

In earlier revisions, receivers, senders, and schedulers all were directly queryable. In R4, receiver queries were moved into a separate "environment" object, obtainable from a receiver with a `get_env` accessor. In R6, the sender queries are given similar treatment, relocating to a "attributes" object obtainable from a sender with a `get_attrs` accessor. This was done to solve a number of design problems with the `split` and `ensure_started` algorithms; e.g., see NVIDIA/stdexec#466.

Schedulers, however, remain directly queryable. As lightweight handles that are required to be movable and copyable, there is little reason to want to dispose of a scheduler and yet persist the scheduler's queries.

This revision also makes operation states directly queryable, even though there isn't yet a use for such. Some early prototypes of cooperative bulk parallel sender algorithms done at NVIDIA suggest the utility of forwardable operation state queries. The authors chose to make opstates directly queryable since the opstate object is itself required to be kept alive for the duration of asynchronous operation.

§ 2.6. R5

The changes since R4 are as follows:

Fixes:

— `start_detached` requires its argument to be a `void` sender (sends no values to `set_value`).

Enhancements:

— Receiver concepts refactored to no longer require an error channel for `exception_ptr` or a stopped channel.

— `sender_of` concept and `connect` customization point additionally require that the receiver is capable of receiving all of the sender's possible completions.

— `get_completion_signatures` is now required to return an instance of either `completion_signatures` or `dependent_completion_signatures`.

— `make_completion_signatures` made more general.

— `receiver_adaptor` handles `get_env` as it does the `set_*` members; that is, `receiver_adaptor` will look for a member named `get_env()` in the derived class, and if found dispatch the `get_env_t` tag invoke customization to it.

— `just`, `just_error`, `just_stopped`, and `into_variant` have been respecified as customization point objects instead of functions, following LEWG guidance.

§ 2.7. R4

The changes since R3 are as follows:

Fixes:

— Fix specification of `get_completion_scheduler` on the `transfer`, `schedule_from` and `transfer_when_all` algorithms; the completion scheduler cannot be guaranteed for `set_error`.

— The value of `sends_stopped` for the default sender traits of types that are generally awaitable was changed from `false` to `true` to acknowledge the fact that some coroutine types are generally awaitable and may implement the `unhandled_stopped()` protocol in their promise types.

— Fix the incorrect use of inline namespaces in the `<execution>` header.

— Shorten the stable names for the sections.

— `sync_wait` now handles `std::error_code` specially by throwing a `std::system_error` on failure.

— Fix how ADL isolation from class template arguments is specified so it doesn't constrain implmentations.

— Properly expose the tag types in the header `<execution>` synopsis.

Enhancements:

— Support for "dependently-typed" senders, where the completion signatures -- and thus the sender metadata -- depend on the type of the receiver connected to it. See the section dependently-typed senders below for more information.

— Add a `read(query)` sender factory for issuing a query against a receiver and sending the result through the value channel. (This is a useful instance of a dependently-typed sender.)

— Add `completion_signatures` utility for declaratively defining a typed sender's metadata.

— Add `make_completion_signatures` utility for specifying a sender's completion signatures by adapting those of another sender.

— Drop support for untyped senders and rename `typed_sender` to `sender`.

— `set_done` is renamed to `set_stopped`. All occurances of "`done`" in indentifiers replaced with "`stopped`"

— Add customization points for controlling the forwarding of scheduler, sender, receiver, and environment queries through layers of adaptors; specify the behavior of the standard adaptors in terms of the new cus-

tomization points.

— Add `get_delegatee_scheduler` query to forward a scheduler that can be used by algorithms or by the scheduler to delegate work and forward progress.

— Add `schedule_result_t` alias template.

— More precisely specify the sender algorithms, including precisely what their completion signatures are.

— `stopped_as_error` respecified as a customization point object.

— `tag_invoke` respecified to improve diagnostics.

§ 2.7.1. Dependently-typed senders

Background:

In the sender/receiver model, as with coroutines, contextual information about the current execution is most naturally propagated from the consumer to the producer. In coroutines, that means information like stop tokens, allocators and schedulers are propagated from the calling coroutine to the callee. In sender/receiver, that means that that contextual information is associated with the receiver and is queried by the sender and/or operation state after the sender and the receiver are `connect`-ed.

Problem:

The implication of the above is that the sender alone does not have all the information about the async computation it will ultimately initiate; some of that information is provided late via the receiver. However, the `sender_traits` mechanism, by which an algorithm can introspect the value and error types the sender will propagate, *only* accepts a sender parameter. It does not take into consideration the type information that will come in late via the receiver. The effect of this is that some senders cannot be typed senders when they otherwise could be.

Example:

To get concrete, consider the case of the "`get_scheduler()`" sender: when `connect`-ed and `start`-ed, it queries the receiver for its associated scheduler and passes it back to the receiver through the value channel. That sender's "value type" is the type of the *receiver's* scheduler. What then should `sender_traits<get_scheduler_sender>::value_types` report for the `get_scheduler()`'s value type? It can't answer because it doesn't know.

This causes knock-on problems since some important algorithms require a typed sender, such as `sync_wait`. To illustrate the problem, consider the following code:

```
namespace ex = std::execution;

ex::sender auto task =
  ex::let_value(
    ex::get_scheduler(), // Fetches scheduler from receiver.
```

```
    [](auto current_sched) {
      // Lauch some nested work on the current scheduler:
      return ex::starts_on(current_sched, nested work...);
    });

std::this_thread::sync_wait(std::move(task));
```

The code above is attempting to schedule some work onto the `sync_wait`'s `run_loop` execution resource. But `let_value` only returns a typed sender when the input sender is typed. As we explained above, `get_scheduler()` is not typed, so `task` is likewise not typed. Since `task` isn't typed, it cannot be passed to `sync_wait` which is expecting a typed sender. The above code would fail to compile.

Solution:

The solution is conceptually quite simple: extend the `sender_traits` mechanism to optionally accept a receiver in addition to the sender. The algorithms can use `sender_traits<Sender, Receiver>` to inspect the async operation's completion-signals. The `typed_sender` concept would also need to take an optional receiver parameter. This is the simplest change, and it would solve the immediate problem.

Design:

Using the receiver type to compute the sender traits turns out to have pitfalls in practice. Many receivers make use of that type information in their implementation. It is very easy to create cycles in the type system, leading to inscrutible errors. The design pursued in R4 is to give receivers an associated *environment* object -- a bag of key/value pairs -- and to move the contextual information (schedulers, etc) out of the receiver and into the environment. The `sender_traits` template and the `typed_sender` concept, rather than taking a receiver, take an environment. This is a much more robust design.

A further refinement of this design would be to separate the receiver and the environment entirely, passing then as separate arguments along with the sender to `connect`. This paper does not propose that change.

Impact:

This change, apart from increasing the expressive power of the sender/receiver abstraction, has the following impact:

— Typed senders become moderately more challenging to write. (The new `completion_signatures` and `transform_completion_signatures` utilities are added to ease this extra burden.)

— Sender adaptor algorithms that previously constrained their sender arguments to satisfy the `typed_sender` concept can no longer do so as the receiver is not available yet. This can result in type-checking that is done later, when `connect` is ultimately called on the resulting sender adaptor.

— Operation states that own receivers that add to or change the environment are typically larger by one pointer. It comes with the benefit of far fewer indirections to evaluate queries.

"Has it been implemented?"

Yes, the reference implementation, which can be found at https://github.com/NVIDIA/stdexec, has implemented this design as well as some dependently-typed senders to confirm that it works.

Implementation experience

Although this change has not yet been made in libunifex, the most widely adopted sender/receiver implementation, a similar design can be found in Folly's coroutine support library. In Folly.Coro, it is possible to await a special awaitable to obtain the current coroutine's associated scheduler (called an executor in Folly).

For instance, the following Folly code grabs the current executor, schedules a task for execution on that executor, and starts the resulting (scheduled) task by enqueueing it for execution.

```
// From Facebook's Folly open source library:
template <class T>
folly::coro::Task<void> CancellableAsyncScope::co_schedule(folly::coro::Task<T>&& task) ·
  this->add(std::move(task).scheduleOn(co_await co_current_executor));
  co_return;
}
```

Facebook relies heavily on this pattern in its coroutine code. But as described above, this pattern doesn't work with R3 of `std::execution` because of the lack of dependently-typed schedulers. The change to `sender_traits` in R4 rectifies that.

Why now?

The authors are loathe to make any changes to the design, however small, at this stage of the C++23 release cycle. But we feel that, for a relatively minor design change -- adding an extra template parameter to `sender_traits` and `typed_sender` -- the returns are large enough to justify the change. And there is no better time to make this change than as early as possible.

One might wonder why this missing feature not been added to sender/receiver before now. The designers of sender/receiver have long been aware of the need. What was missing was a clean, robust, and simple design for the change, which we now have.

Drive-by:

We took the opportunity to make an additional drive-by change: Rather than providing the sender traits via a class template for users to specialize, we changed it into a sender *query*: `get_completion_signatures(sender, env)`. That function's return type is used as the sender's traits. The authors feel this leads to a more uniform design and gives sender authors a straightforward way to make the value/error types dependent on the cv- and ref-qualification of the sender if need be.

Details:

Below are the salient parts of the new support for dependently-typed senders in R4:

— Receiver queries have been moved from the receiver into a separate environment object.

— Receivers have an associated environment. The new `get_env` CPO retrieves a receiver's environment. If a receiver doesn't implement `get_env`, it returns an unspecified "empty" environment -- an empty struct.

— `sender_traits` now takes an optional `Env` parameter that is used to determine the error/value types.

— The primary `sender_traits` template is replaced with a `completion_signatures_of_t` alias implemented in terms of a new `get_completion_signatures` CPO that dispatches with `tag_invoke`. `get_completion_signatures` takes a sender and an optional environment. A sender can customize this to specify its value/error types.

— Support for untyped senders is dropped. The `typed_sender` concept has been renamed to `sender` and now takes an optional environment.

— The environment argument to the `sender` concept and the `get_completion_signatures` CPO defaults to `no_env`. All environment queries fail (are ill-formed) when passed an instance of `no_env`.

— A type `S` is required to satisfy `sender<S>` to be considered a sender. If it doesn't know what types it will complete with independent of an environment, it returns an instance of the placeholder traits `dependent_completion_signatures`.

— If a sender satisfies both `sender<S>` and `sender<S, Env>`, then the completion signatures for the two cannot be different in any way. It is possible for an implementation to enforce this statically, but not required.

— All of the algorithms and examples have been updated to work with dependently-typed senders.

§ 2.8. R3

The changes since R2 are as follows:

Fixes:

— Fix specification of the `starts_on` algorithm to clarify lifetimes of intermediate operation states and properly scope the `get_scheduler` query.

— Fix a memory safety bug in the implementation of *`connect-awaitable`*.

— Fix recursive definition of the `scheduler` concept.

Enhancements:

— Add `run_loop` execution resource.

— Add `receiver_adaptor` utility to simplify writing receivers.

— Require a scheduler's sender to model `sender_of` and provide a completion scheduler.

— Specify the cancellation scope of the `when_all` algorithm.

— Make `as_awaitable` a customization point.

— Change `connect`'s handling of awaitables to consider those types that are awaitable owing to customization of `as_awaitable`.

— Add `value_types_of_t` and `error_types_of_t` alias templates; rename `stop_token_type_t` to `stop_token_of_t`.

— Add a design rationale for the removal of the possibly eager algorithms.

— Expand the section on field experience.

§ 2.9. R2

The changes since R1 are as follows:

— Remove the eagerly executing sender algorithms.

— Extend the `execution::connect` customization point and the `sender_traits<>` template to recognize awaitables as `typed_sender`s.

— Add utilities `as_awaitable()` and `with_awaitable_senders<>` so a coroutine type can trivially make senders awaitable with a coroutine.

— Add a section describing the design of the sender/awaitable interactions.

— Add a section describing the design of the cancellation support in sender/receiver.

— Add a section showing examples of simple sender adaptor algorithms.

— Add a section showing examples of simple schedulers.

— Add a few more examples: a sudoku solver, a parallel recursive file copy, and an echo server.

— Refined the forward progress guarantees on the `bulk` algorithm.

— Add a section describing how to use a range of senders to represent async sequences.

— Add a section showing how to use senders to represent partial success.

— Add sender factories `execution::just_error` and `execution::just_stopped`.

— Add sender adaptors `execution::stopped_as_optional` and `execution::stopped_as_error`.

— Document more production uses of sender/receiver at scale.

— Various fixes of typos and bugs.

§ 2.10. R1

The changes since R0 are as follows:

— Added a new concept, `sender_of`.

— Added a new scheduler query, `this_thread::execute_may_block_caller`.

— Added a new scheduler query, `get_forward_progress_guarantee`.

— Removed the `unschedule` adaptor.

— Various fixes of typos and bugs.

§ 2.11. R0

Initial revision.

§ 3. Design - introduction

The following three sections describe the entirety of the proposed design.

— § 3 Design - introduction describes the conventions used through the rest of the design sections, as well as an example illustrating how we envision code will be written using this proposal.

— § 4 Design - user side describes all the functionality from the perspective we intend for users: it describes the various concepts they will interact with, and what their programming model is.

— § 5 Design - implementer side describes the machinery that allows for that programming model to function, and the information contained there is necessary for people implementing senders and sender algorithms (including the standard library ones) - but is not necessary to use senders productively.

§ 3.1. Conventions

The following conventions are used throughout the design section:

1. The namespace proposed in this paper is the same as in *A Unified Executors Proposal for C++*: `std::execution`; however, for brevity, the `std::` part of this name is omitted. When you see `execution::foo`, treat that as `std::execution::foo`.
2. Universal references and explicit calls to `std::move`/`std::forward` are omitted in code samples and signatures for simplicity; assume universal references and perfect forwarding unless stated otherwise.
3. None of the names proposed here are names that we are particularly attached to; consider the names to be reasonable placeholders that can freely be changed, should the committee want to do so.

§ 3.2. Queries and algorithms

A **query** is a callable that takes some set of objects (usually one) as parameters and returns facts about those objects without modifying them. Queries are usually customization point objects, but in some cases may be functions.

An **algorithm** is a callable that takes some set of objects as parameters and causes those objects to do something. Algorithms are usually customization point objects, but in some cases may be functions.

§ 4. Design - user side

§ 4.1. Execution resources describe the place of execution

An execution resource is a resource that represents the *place* where execution will happen. This could be a concrete resource - like a specific thread pool object, or a GPU - or a more abstract one, like the current thread of execution. Execution contexts don't need to have a representation in code; they are simply a term describing certain properties of execution of a function.

§ 4.2. Schedulers represent execution resources

A scheduler is a lightweight handle that represents a strategy for scheduling work onto an execution resource. Since execution resources don't necessarily manifest in C++ code, it's not possible to program directly against their API. A scheduler is a solution to that problem: the scheduler concept is defined by a single sender algorithm, `schedule`, which returns a sender that will complete on an execution resource determined by the scheduler. Logic that you want to run on that context can be placed in the receiver's completion-signalling method.

```
execution::scheduler auto sch = thread_pool.scheduler();
execution::sender auto snd = execution::schedule(sch);
// snd is a sender (see below) describing the creation of a new execution resource
// on the execution resource associated with sch
```

Note that a particular scheduler type may provide other kinds of scheduling operations which are supported by its associated execution resource. It is not limited to scheduling purely using the `execution::schedule` API.

Future papers will propose additional scheduler concepts that extend `scheduler` to add other capabilities. For example:

- A `time_scheduler` concept that extends `scheduler` to support time-based scheduling. Such a concept might provide access to `schedule_after(sched, duration)`, `schedule_at(sched, time_point)` and `now(sched)` APIs.
- Concepts that extend `scheduler` to support opening, reading and writing files asynchronously.
- Concepts that extend `scheduler` to support connecting, sending data and receiving data over the network asynchronously.

§ 4.3. Senders describe work

A sender is an object that describes work. Senders are similar to futures in existing asynchrony designs, but unlike futures, the work that is being done to arrive at the values they will *send* is also directly described by the sender object itself. A sender is said to *send* some values if a receiver connected (see § 5.3 execution::connect) to that sender will eventually *receive* said values.

The primary defining sender algorithm is § 5.3 execution::connect; this function, however, is not a user-facing API; it is used to facilitate communication between senders and various sender algorithms, but end user code is not expected to invoke it directly.

The way user code is expected to interact with senders is by using sender algorithms. This paper proposes an initial set of such sender algorithms, which are described in § 4.4 Senders are composable through sender algorithms, § 4.19 User-facing sender factories, § 4.20 User-facing sender adaptors, and § 4.21 User-facing sender consumers. For example, here is how a user can create a new sender on a scheduler, attach a continuation to it, and then wait for execution of the continuation to complete:

```
execution::scheduler auto sch = thread_pool.scheduler();
execution::sender auto snd = execution::schedule(sch);
execution::sender auto cont = execution::then(snd, []{
    std::fstream file{ "result.txt" };
    file << compute_result;
});

this_thread::sync_wait(cont);
// at this point, cont has completed execution
```

§ 4.4. Senders are composable through sender algorithms

Asynchronous programming often departs from traditional code structure and control flow that we are familiar with. A successful asynchronous framework must provide an intuitive story for composition of asynchronous work: expressing dependencies, passing objects, managing object lifetimes, etc.

The true power and utility of senders is in their composability. With senders, users can describe generic execution pipelines and graphs, and then run them on and across a variety of different schedulers. Senders are composed using sender algorithms:

— sender factories, algorithms that take no senders and return a sender.

— sender adaptors, algorithms that take (and potentially `execution::connect`) senders and return a sender.

— sender consumers, algorithms that take (and potentially `execution::connect`) senders and do not return a sender.

§ 4.5. Senders can propagate completion schedulers

One of the goals of executors is to support a diverse set of execution resources, including traditional thread pools, task and fiber frameworks (like HPX Legion), and GPUs and other accelerators (managed by runtimes such as CUDA or SYCL). On many of these systems, not all execution agents are created equal and not all functions can be run on all execution agents. Having precise control over the execution resource used for any given function call being submitted is important on such systems, and the users of standard execution facilities

will expect to be able to express such requirements.

A Unified Executors Proposal for C++ was not always clear about the *place of execution* of any given piece of code. Precise control was present in the two-way execution API present in earlier executor designs, but it has so far been missing from the senders design. There has been a proposal (*Towards C++23 executors: A proposal for an initial set of algorithms*) to provide a number of sender algorithms that would enforce certain rules on the places of execution of the work described by a sender, but we have found those sender algorithms to be insufficient for achieving the best performance on all platforms that are of interest to us. The implementation strategies that we are aware of result in one of the following situations:

1. trying to submit work to one execution resource (such as a CPU thread pool) from another execution resource (such as a GPU or a task framework), which assumes that all execution agents are as capable as a `std::thread` (which they aren't).
2. forcibly interleaving two adjacent execution graph nodes that are both executing on one execution resource (such as a GPU) with glue code that runs on another execution resource (such as a CPU), which is prohibitively expensive for some execution resources (such as CUDA or SYCL).
3. having to customise most or all sender algorithms to support an execution resource, so that you can avoid problems described in 1. and 2, which we believe is impractical and brittle based on months of field experience attempting this in Agency.

None of these implementation strategies are acceptable for many classes of parallel runtimes, such as task frameworks (like HPX) or accelerator runtimes (like CUDA or SYCL).

Therefore, in addition to the `starts_on` sender algorithm from *Towards C++23 executors: A proposal for an initial set of algorithms*, we are proposing a way for senders to advertise what scheduler (and by extension what execution resource) they will complete on. Any given sender **may** have completion schedulers for some or all of the signals (value, error, or stopped) it completes with (for more detail on the completion-signals, see § 5.1 Receivers serve as glue between senders). When further work is attached to that sender by invoking sender algorithms, that work will also complete on an appropriate completion scheduler.

§ 4.5.1. `execution::get_completion_scheduler`

`get_completion_scheduler` is a query that retrieves the completion scheduler for a specific completion-signal from a sender's environment. For a sender that lacks a completion scheduler query for a given signal, calling `get_completion_scheduler` is ill-formed. If a sender advertises a completion scheduler for a signal in this way, that sender **must** ensure that it sends that signal on an execution agent belonging to an execution resource represented by a scheduler returned from this function. See § 4.5 Senders can propagate completion schedulers for more details.

```
execution::scheduler auto cpu_sched = new_thread_scheduler{};
execution::scheduler auto gpu_sched = cuda::scheduler();
```

```
execution::sender auto snd0 = execution::schedule(cpu_sched);
execution::scheduler auto completion_sch0 =
  execution::get_completion_scheduler<execution::set_value_t>(get_env(snd0));
// completion_sch0 is equivalent to cpu_sched

execution::sender auto snd1 = execution::then(snd0, []{
    std::cout << "I am running on cpu_sched!\n";
});
execution::scheduler auto completion_sch1 =
  execution::get_completion_scheduler<execution::set_value_t>(get_env(snd1));
// completion_sch1 is equivalent to cpu_sched

execution::sender auto snd2 = execution::continues_on(snd1, gpu_sched);
execution::sender auto snd3 = execution::then(snd2, []{
    std::cout << "I am running on gpu_sched!\n";
});
execution::scheduler auto completion_sch3 =
  execution::get_completion_scheduler<execution::set_value_t>(get_env(snd3));
// completion_sch3 is equivalent to gpu_sched
```

§ 4.6. Execution resource transitions are explicit

A Unified Executors Proposal for C++ does not contain any mechanisms for performing an execution resource transition. The only sender algorithm that can create a sender that will move execution to a *specific* execution resource is `execution::schedule`, which does not take an input sender. That means that there's no way to construct sender chains that traverse different execution resources. This is necessary to fulfill the promise of senders being able to replace two-way executors, which had this capability.

We propose that, for senders advertising their completion scheduler, all execution resource transitions **must** be explicit; running user code anywhere but where they defined it to run **must** be considered a bug.

The `execution::continues_on` sender adaptor performs a transition from one execution resource to another:

```
execution::scheduler auto sch1 = ...;
execution::scheduler auto sch2 = ...;

execution::sender auto snd1 = execution::schedule(sch1);
execution::sender auto then1 = execution::then(snd1, []{
    std::cout << "I am running on sch1!\n";
});

execution::sender auto snd2 = execution::continues_on(then1, sch2);
execution::sender auto then2 = execution::then(snd2, []{
    std::cout << "I am running on sch2!\n";
```

```
});

this_thread::sync_wait(then2);
```

§ 4.7. Senders can be either multi-shot or single-shot

Some senders may only support launching their operation a single time, while others may be repeatable and support being launched multiple times. Executing the operation may consume resources owned by the sender.

For example, a sender may contain a `std::unique_ptr` that it will be transferring ownership of to the operation-state returned by a call to `execution::connect` so that the operation has access to this resource. In such a sender, calling `execution::connect` consumes the sender such that after the call the input sender is no longer valid. Such a sender will also typically be move-only so that it can maintain unique ownership of that resource.

A *single-shot sender* can only be connected to a receiver at most once. Its implementation of `execution::connect` only has overloads for an rvalue-qualified sender. Callers must pass the sender as an rvalue to the call to `execution::connect`, indicating that the call consumes the sender.

A *multi-shot sender* can be connected to multiple receivers and can be launched multiple times. Multi-shot senders customise `execution::connect` to accept an lvalue reference to the sender. Callers can indicate that they want the sender to remain valid after the call to `execution::connect` by passing an lvalue reference to the sender to call these overloads. Multi-shot senders should also define overloads of `execution::connect` that accept rvalue-qualified senders to allow the sender to be also used in places where only a single-shot sender is required.

If the user of a sender does not require the sender to remain valid after connecting it to a receiver then it can pass an rvalue-reference to the sender to the call to `execution::connect`. Such usages should be able to accept either single-shot or multi-shot senders.

If the caller does wish for the sender to remain valid after the call then it can pass an lvalue-qualified sender to the call to `execution::connect`. Such usages will only accept multi-shot senders.

Algorithms that accept senders will typically either decay-copy an input sender and store it somewhere for later usage (for example as a data-member of the returned sender) or will immediately call `execution::connect` on the input sender, such as in `this_thread::sync_wait`.

Some multi-use sender algorithms may require that an input sender be copy-constructible but will only call `execution::connect` on an rvalue of each copy, which still results in effectively executing the operation multiple times. Other multi-use sender algorithms may require that the sender is move-constructible but will invoke `execution::connect` on an lvalue reference to the sender.

For a sender to be usable in both multi-use scenarios, it will generally be required to be both copy-constructible and lvalue-connectable.

§ 4.8. Senders are forkable

Any non-trivial program will eventually want to fork a chain of senders into independent streams of work, regardless of whether they are single-shot or multi-shot. For instance, an incoming event to a middleware system may be required to trigger events on more than one downstream system. This requires that we provide well defined mechanisms for making sure that connecting a sender multiple times is possible and correct.

The `split` sender adaptor facilitates connecting to a sender multiple times, regardless of whether it is single-shot or multi-shot:

```
auto some_algorithm(execution::sender auto&& input) {
    execution::sender auto multi_shot = split(input);
    // "multi_shot" is guaranteed to be multi-shot,
    // regardless of whether "input" was multi-shot or not

    return when_all(
      then(multi_shot, [] { std::cout << "First continuation\n"; }),
      then(multi_shot, [] { std::cout << "Second continuation\n"; })
    );
}
```

§ 4.9. Senders support cancellation

Senders are often used in scenarios where the application may be concurrently executing multiple strategies for achieving some program goal. When one of these strategies succeeds (or fails) it may not make sense to continue pursuing the other strategies as their results are no longer useful.

For example, we may want to try to simultaneously connect to multiple network servers and use whichever server responds first. Once the first server responds we no longer need to continue trying to connect to the other servers.

Ideally, in these scenarios, we would somehow be able to request that those other strategies stop executing promptly so that their resources (e.g. cpu, memory, I/O bandwidth) can be released and used for other work.

While the design of senders has support for cancelling an operation before it starts by simply destroying the sender or the operation-state returned from `execution::connect()` before calling `execution::start()`, there also needs to be a standard, generic mechanism to ask for an already-started operation to complete early.

The ability to be able to cancel in-flight operations is fundamental to supporting some kinds of generic concurrency algorithms.

For example:

— a `when_all(ops...)` algorithm should cancel other operations as soon as one operation fails

— a `first_successful(ops...)` algorithm should cancel the other operations as soon as one operation

completes successfuly

— a generic `timeout(src, duration)` algorithm needs to be able to cancel the `src` operation after the timeout duration has elapsed.

— a `stop_when(src, trigger)` algorithm should cancel `src` if `trigger` completes first and cancel `trigger` if `src` completes first

The mechanism used for communcating cancellation-requests, or stop-requests, needs to have a uniform interface so that generic algorithms that compose sender-based operations, such as the ones listed above, are able to communicate these cancellation requests to senders that they don't know anything about.

The design is intended to be composable so that cancellation of higher-level operations can propagate those cancellation requests through intermediate layers to lower-level operations that need to actually respond to the cancellation requests.

For example, we can compose the algorithms mentioned above so that child operations are cancelled when any one of the multiple cancellation conditions occurs:

```
sender auto composed_cancellation_example(auto query) {
  return stop_when(
    timeout(
      when_all(
        first_successful(
          query_server_a(query),
          query_server_b(query)),
        load_file("some_file.jpg")),
      5s),
    cancelButton.on_click());
}
```

In this example, if we take the operation returned by `query_server_b(query)`, this operation will receive a stop-request when any of the following happens:

— `first_successful` algorithm will send a stop-request if `query_server_a(query)` completes successfully

— `when_all` algorithm will send a stop-request if the `load_file("some_file.jpg")` operation completes with an error or stopped result.

— `timeout` algorithm will send a stop-request if the operation does not complete within 5 seconds.

— `stop_when` algorithm will send a stop-request if the user clicks on the "Cancel" button in the user-interface.

— The parent operation consuming the `composed_cancellation_example()` sends a stop-request

Note that within this code there is no explicit mention of cancellation, stop-tokens, callbacks, etc. yet the example fully supports and responds to the various cancellation sources.

The intent of the design is that the common usage of cancellation in sender/receiver-based code is primarily through use of concurrency algorithms that manage the detailed plumbing of cancellation for you. Much like algorithms that compose senders relieve the user from having to write their own receiver types, algorithms that introduce concurrency and provide higher-level cancellation semantics relieve the user from having to deal with low-level details of cancellation.

§ 4.9.1. Cancellation design summary

The design of cancellation described in this paper is built on top of and extends the `std::stop_token`-based cancellation facilities added in C++20, first proposed in *Composable cancellation for sender-based async operations*.

At a high-level, the facilities proposed by this paper for supporting cancellation include:

- Add a `std::stoppable_token` concept that generalises the interface of the `std::stop_token` type to allow other stop token types with different implementation strategies.
- Add `std::unstoppable_token` concept for detecting whether a `stoppable_token` can never receive a stop-request.
- Add `std::inplace_stop_token`, `std::inplace_stop_source` and `std::inplace_stop_callback<CB>` types that provide a more efficient implementation of a stop-token for use in structured concurrency situations.
- Add `std::never_stop_token` for use in places where you never want to issue a stop-request.
- Add `std::execution::get_stop_token()` CPO for querying the stop-token to use for an operation from its receiver's execution environment.
- Add `std::execution::stop_token_of_t<T>` for querying the type of a stop-token returned from `get_stop_token()`.

In addition, there are requirements added to some of the algorithms to specify what their cancellation behaviour is and what the requirements of customisations of those algorithms are with respect to cancellation.

The key component that enables generic cancellation within sender-based operations is the `execution::get_stop_token()` CPO. This CPO takes a single parameter, which is the execution environment of the receiver passed to `execution::connect`, and returns a `std::stoppable_token` that the operation can use to check for stop-requests for that operation.

As the caller of `execution::connect` typically has control over the receiver type it passes, it is able to customise the `std::execution::get_env()` CPO for that receiver to return an execution environment that hooks the `execution::get_stop_token()` CPO to return a stop-token that the receiver has control over and that it can use to communicate a stop-request to the operation once it has started.

§ 4.9.2. Support for cancellation is optional

Support for cancellation is optional, both on part of the author of the receiver and on part of the author of the sender.

If the receiver's execution environment does not customise the `execution::get_stop_token()` CPO then invoking the CPO on that receiver's environment will invoke the default implementation which returns `std::never_stop_token`. This is a special `stoppable_token` type that is statically known to always return `false` from the `stop_possible()` method.

Sender code that tries to use this stop-token will in general result in code that handles stop-requests being compiled out and having little to no run-time overhead.

If the sender doesn't call `execution::get_stop_token()`, for example because the operation does not support cancellation, then it will simply not respond to stop-requests from the caller.

Note that stop-requests are generally racy in nature as there is often a race betwen an operation completing naturally and the stop-request being made. If the operation has already completed or past the point at which it can be cancelled when the stop-request is sent then the stop-request may just be ignored. An application will typically need to be able to cope with senders that might ignore a stop-request anyway.

§ 4.9.3. Cancellation is inherently racy

Usually, an operation will attach a stop callback at some point inside the call to `execution::start()` so that a subsequent stop-request will interrupt the logic.

A stop-request can be issued concurrently from another thread. This means the implementation of `execution::start()` needs to be careful to ensure that, once a stop callback has been registered, that there are no data-races between a potentially concurrently-executing stop callback and the rest of the `execution::start()` implementation.

An implementation of `execution::start()` that supports cancellation will generally need to perform (at least) two separate steps: launch the operation, subscribe a stop callback to the receiver's stop-token. Care needs to be taken depending on the order in which these two steps are performed.

If the stop callback is subscribed first and then the operation is launched, care needs to be taken to ensure that a stop-request that invokes the stop callback on another thread after the stop callback is registered but before the operation finishes launching does not either result in a missed cancellation request or a data-race. e.g. by performing an atomic write after the launch has finished executing

If the operation is launched first and then the stop callback is subscribed, care needs to be taken to ensure that if the launched operation completes concurrently on another thread that it does not destroy the operation-state until after the stop callback has been registered. e.g. by having the `execution::start` implementation write to an atomic variable once it has finished registering the stop callback and having the concurrent completion handler check that variable and either call the completion-signalling operation or store the result and defer calling the receiver's completion-signalling operation to the `execution::start()` call (which is still executing).

For an example of an implementation strategy for solving these data-races see § 1.4 Asynchronous Windows socket recv.

§ 4.9.4. Cancellation design status

This paper currently includes the design for cancellation as proposed in *Composable cancellation for sender-based async operations* - "Composable cancellation for sender-based async operations". P2175R0 contains more details on the background motivation and prior-art and design rationale of this design.

It is important to note, however, that initial review of this design in the SG1 concurrency subgroup raised some concerns related to runtime overhead of the design in single-threaded scenarios and these concerns are still being investigated.

The design of P2175R0 has been included in this paper for now, despite its potential to change, as we believe that support for cancellation is a fundamental requirement for an async model and is required in some form to be able to talk about the semantics of some of the algorithms proposed in this paper.

This paper will be updated in the future with any changes that arise from the investigations into P2175R0.

§ 4.10. Sender factories and adaptors are lazy

In an earlier revision of this paper, some of the proposed algorithms supported executing their logic eagerly; *i.e.*, before the returned sender has been connected to a receiver and started. These algorithms were removed because eager execution has a number of negative semantic and performance implications.

We have originally included this functionality in the paper because of a long-standing belief that eager execution is a mandatory feature to be included in the standard Executors facility for that facility to be acceptable for accelerator vendors. A particular concern was that we must be able to write generic algorithms that can run either eagerly or lazily, depending on the kind of an input sender or scheduler that have been passed into them as arguments. We considered this a requirement, because the _latency_ of launching work on an accelerator can sometimes be considerable.

However, in the process of working on this paper and implementations of the features proposed within, our set of requirements has shifted, as we understood the different implementation strategies that are available for the feature set of this paper better, and, after weighing the earlier concerns against the points presented below, we have arrived at the conclusion that a purely lazy model is enough for most algorithms, and users who intend to launch work earlier may write an algorithm to achieve that goal. We have also come to deeply appreciate the fact that a purely lazy model allows both the implementation and the compiler to have a much better understanding of what the complete graph of tasks looks like, allowing them to better optimize the code - also when targetting accelerators.

§ 4.10.1. Eager execution leads to detached work or worse

One of the questions that arises with APIs that can potentially return eagerly-executing senders is "What happens when those senders are destructed without a call to `execution::connect`?" or similarly, "What happens if a call to `execution::connect` is made, but the returned operation state is destroyed before `execution::start` is called on that operation state"?

In these cases, the operation represented by the sender is potentially executing concurrently in another thread at the time that the destructor of the sender and/or operation-state is running. In the case that the operation has not completed executing by the time that the destructor is run we need to decide what the semantics of the destructor is.

There are three main strategies that can be adopted here, none of which is particularly satisfactory:

1. Make this undefined-behaviour - the caller must ensure that any eagerly-executing sender is always joined by connecting and starting that sender. This approach is generally pretty hostile to programmers, particularly in the presence of exceptions, since it complicates the ability to compose these operations.

   Eager operations typically need to acquire resources when they are first called in order to start the operation early. This makes eager algorithms prone to failure. Consider, then, what might happen in an expression such as `when_all(eager_op_1(), eager_op_2())`. Imagine `eager_op_1()` starts an asynchronous operation successfully, but then `eager_op_2()` throws. For lazy senders, that failure happens in the context of the `when_all` algorithm, which handles the failure and ensures that async work joins on all code paths. In this case though -- the eager case -- the child operation has failed even before `when_all` has been called.

   It then becomes the responsibility, not of the algorithm, but of the end user to handle the exception and ensure that `eager_op_1()` is joined before allowing the exception to propagate. If they fail to do that, they incur undefined behavior.

2. Detach from the computation - let the operation continue in the background - like an implicit call to `std::thread::detach()`. While this approach can work in some circumstances for some kinds of applications, in general it is also pretty user-hostile; it makes it difficult to reason about the safe destruction of resources used by these eager operations. In general, detached work necessitates some kind of garbage collection; e.g., `std::shared_ptr`, to ensure resources are kept alive until the operations complete, and can make clean shutdown nigh impossible.

3. Block in the destructor until the operation completes. This approach is probably the safest to use as it preserves the structured nature of the concurrent operations, but also introduces the potential for deadlocking the application if the completion of the operation depends on the current thread making forward progress.

   The risk of deadlock might occur, for example, if a thread-pool with a small number of threads is executing code that creates a sender representing an eagerly-executing operation and then calls the destructor of that sender without joining it (e.g. because an exception was thrown). If the current thread blocks waiting for that eager operation to complete and that eager operation cannot complete until some entry enqueued to the thread-pool's queue of work is run then the thread may wait for an indefinite amount of time. If all threads of the thread-pool are simultaneously performing such blocking operations then deadlock can re-

sult.

There are also minor variations on each of these choices. For example:

4. A variation of (1): Call `std::terminate` if an eager sender is destructed without joining it. This is the approach that `std::thread` destructor takes.
5. A variation of (2): Request cancellation of the operation before detaching. This reduces the chances of operations continuing to run indefinitely in the background once they have been detached but does not solve the lifetime- or shutdown-related challenges.
6. A variation of (3): Request cancellation of the operation before blocking on its completion. This is the strategy that `std::jthread` uses for its destructor. It reduces the risk of deadlock but does not eliminate it.

§ 4.10.2. Eager senders complicate algorithm implementations

Algorithms that can assume they are operating on senders with strictly lazy semantics are able to make certain optimizations that are not available if senders can be potentially eager. With lazy senders, an algorithm can safely assume that a call to `execution::start` on an operation state strictly happens before the execution of that async operation. This frees the algorithm from needing to resolve potential race conditions. For example, consider an algorithm `sequence` that puts async operations in sequence by starting an operation only after the preceding one has completed. In an expression like `sequence(a(), then(src, [] { b(); }), c())`, one may reasonably assume that `a()`, `b()` and `c()` are sequenced and therefore do not need synchronisation. Eager algorithms break that assumption.

When an algorithm needs to deal with potentially eager senders, the potential race conditions can be resolved one of two ways, neither of which is desirable:

1. Assume the worst and implement the algorithm defensively, assuming all senders are eager. This obviously has overheads both at runtime and in algorithm complexity. Resolving race conditions is hard.
2. Require senders to declare whether they are eager or not with a query. Algorithms can then implement two different implementation strategies, one for strictly lazy senders and one for potentially eager senders. This addresses the performance problem of (1) while compounding the complexity problem.

§ 4.10.3. Eager senders incur cancellation-related overhead

Another implication of the use of eager operations is with regards to cancellation. The eagerly executing operation will not have access to the caller's stop token until the sender is connected to a receiver. If we still want to be able to cancel the eager operation then it will need to create a new stop source and pass its associated stop token down to child operations. Then when the returned sender is eventually connected it will register a stop callback with the receiver's stop token that will request stop on the eager sender's stop source.

As the eager operation does not know at the time that it is launched what the type of the receiver is going to be,

and thus whether or not the stop token returned from `execution::get_stop_token` is an `std::unstoppable_token` or not, the eager operation is going to need to assume it might be later connected to a receiver with a stop token that might actually issue a stop request. Thus it needs to declare space in the operation state for a type-erased stop callback and incur the runtime overhead of supporting cancellation, even if cancellation will never be requested by the caller.

The eager operation will also need to do this to support sending a stop request to the eager operation in the case that the sender representing the eager work is destroyed before it has been joined (assuming strategy (5) or (6) listed above is chosen).

§ 4.10.4. Eager senders cannot access execution resource from the receiver

In sender/receiver, contextual information is passed from parent operations to their children by way of receivers. Information like stop tokens, allocators, current scheduler, priority, and deadline are propagated to child operations with custom receivers at the time the operation is connected. That way, each operation has the contextual information it needs before it is started.

But if the operation is started before it is connected to a receiver, then there isn't a way for a parent operation to communicate contextual information to its child operations, which may complete before a receiver is ever attached.

§ 4.11. Schedulers advertise their forward progress guarantees

To decide whether a scheduler (and its associated execution resource) is sufficient for a specific task, it may be necessary to know what kind of forward progress guarantees it provides for the execution agents it creates. The C++ Standard defines the following forward progress guarantees:

— *concurrent*, which requires that a thread makes progress *eventually*;

— *parallel*, which requires that a thread makes progress once it executes a step; and

— *weakly parallel*, which does not require that the thread makes progress.

This paper introduces a scheduler query function, `get_forward_progress_guarantee`, which returns one of the enumerators of a new `enum` type, `forward_progress_guarantee`. Each enumerator of `forward_progress_guarantee` corresponds to one of the aforementioned guarantees.

§ 4.12. Most sender adaptors are pipeable

To facilitate an intuitive syntax for composition, most sender adaptors are *pipeable*; they can be composed (*piped*) together with `operator|`. This mechanism is similar to the `operator|` composition that C++ range adaptors support and draws inspiration from piping in *nix shells. Pipeable sender adaptors take a sender as their first parameter and have no other sender parameters.

`a | b` will pass the sender `a` as the first argument to the pipeable sender adaptor `b`. Pipeable sender adaptors support partial application of the parameters after the first. For example, all of the following are equivalent:

```
execution::bulk(snd, N, [] (std::size_t i, auto d) {});
execution::bulk(N, [] (std::size_t i, auto d) {})(snd);
snd | execution::bulk(N, [] (std::size_t i, auto d) {});
```

Piping enables you to compose together senders with a linear syntax. Without it, you'd have to use either nested function call syntax, which would cause a syntactic inversion of the direction of control flow, or you'd have to introduce a temporary variable for each stage of the pipeline. Consider the following example where we want to execute first on a CPU thread pool, then on a CUDA GPU, then back on the CPU thread pool:

Syntax Style	Example

Function call (nested)	`auto snd = execution::then(`
	`  execution::continues_on(`
	`  execution::then(`
	`  execution::continues_on(`
	`  execution::then(`
	`  execution::schedule(thread_pool.scheduler())`
	`  []{ return 123; }),`
	`  cuda::new_stream_scheduler()),`
	`  [](int i){ return 123 * 5; }),`
	`  thread_pool.scheduler()),`
	`  [](int i){ return i - 5; });`
	`auto [result] = this_thread::sync_wait(snd).value();`
	`// result == 610`

Function call (named temporaries)	`auto snd0 = execution::schedule(thread_pool.scheduler());`
	`auto snd1 = execution::then(snd0, []{ return 123; });`
	`auto snd2 = execution::continues_on(snd1, cuda::new_stream_scheduler());`
	`auto snd3 = execution::then(snd2, [](int i){ return 123 * 5; })`
	`auto snd4 = execution::continues_on(snd3, thread_pool.scheduler())`
	`auto snd5 = execution::then(snd4, [](int i){ return i - 5; });`
	`auto [result] = *this_thread::sync_wait(snd4);`
	`// result == 610`

Pipe	`auto snd = execution::schedule(thread_pool.scheduler())`
	`  | execution::then([]{ return 123; })`
	`  | execution::continues_on(cuda::new_stream_scheduler())`
	`  | execution::then([](int i){ return 123 * 5; })`
	`  | execution::continues_on(thread_pool.scheduler())`
	`  | execution::then([](int i){ return i - 5; });`
	`auto [result] = this_thread::sync_wait(snd).value();`

```
// result == 610
```

Certain sender adaptors are not pipeable, because using the pipeline syntax can result in confusion of the semantics of the adaptors involved. Specifically, the following sender adaptors are not pipeable.

— `execution::when_all` and `execution::when_all_with_variant`: Since this sender adaptor takes a variadic pack of senders, a partially applied form would be ambiguous with a non partially applied form with an arity of one less.

— `execution::starts_on`: This sender adaptor changes how the sender passed to it is executed, not what happens to its result, but allowing it in a pipeline makes it read as if it performed a function more similar to `continues_on`.

Sender consumers could be made pipeable, but we have chosen to not do so. However, since these are terminal nodes in a pipeline and nothing can be piped after them, we believe a pipe syntax may be confusing as well as unnecessary, as consumers cannot be chained. We believe sender consumers read better with function call syntax.

§ 4.13. A range of senders represents an async sequence of data

Senders represent a single unit of asynchronous work. In many cases though, what is being modeled is a sequence of data arriving asynchronously, and you want computation to happen on demand, when each element arrives. This requires nothing more than what is in this paper and the range support in C++20. A range of senders would allow you to model such input as keystrikes, mouse movements, sensor readings, or network requests.

Given some expression `R` that is a range of senders, consider the following in a coroutine that returns an async generator type:

```
for (auto snd : R) {
  if (auto opt = co_await execution::stopped_as_optional(std::move(snd)))
    co_yield fn(*std::move(opt));
  else
    break;
}
```

This transforms each element of the asynchronous sequence `R` with the function `fn` on demand, as the data arrives. The result is a new asynchronous sequence of the transformed values.

Now imagine that `R` is the simple expression `views::iota(0) | views::transform(execution::just)`. This creates a lazy range of senders, each of which completes immediately with monotonically increasing integers. The above code churns through the range, generating a new infine asynchronous range of values [`fn(0)`, `fn(1)`, `fn(2)`, ...].

Far more interesting would be if `R` were a range of senders representing, say, user actions in a UI. The above code gives a simple way to respond to user actions on demand.

§ 4.14. Senders can represent partial success

Receivers have three ways they can complete: with success, failure, or cancellation. This begs the question of how they can be used to represent async operations that *partially* succeed. For example, consider an API that reads from a socket. The connection could drop after the API has filled in some of the buffer. In cases like that, it makes sense to want to report both that the connection dropped and that some data has been successfully read.

Often in the case of partial success, the error condition is not fatal nor does it mean the API has failed to satisfy its post-conditions. It is merely an extra piece of information about the nature of the completion. In those cases, "partial success" is another way of saying "success". As a result, it is sensible to pass both the error code and the result (if any) through the value channel, as shown below:

```
// Capture a buffer for read_socket_async to fill in
execution::just(array<byte, 1024>{})
  | execution::let_value([socket](array<byte, 1024>& buff) {
      // read_socket_async completes with two values: an error_code and
      // a count of bytes:
      return read_socket_async(socket, span{buff})
          // For success (partial and full), specify the next action:
        | execution::let_value([](error_code err, size_t bytes_read) {
            if (err != 0) {
              // OK, partial success. Decide how to deal with the partial results
            } else {
              // OK, full success here.
            }
          });
    })
```

In other cases, the partial success is more of a partial *failure*. That happens when the error condition indicates that in some way the function failed to satisfy its post-conditions. In those cases, sending the error through the value channel loses valuable contextual information. It's possible that bundling the error and the incomplete results into an object and passing it through the error channel makes more sense. In that way, generic algorithms will not miss the fact that a post-condition has not been met and react inappropriately.

Another possibility is for an async API to return a *range* of senders: if the API completes with full success, full error, or cancellation, the returned range contains just one sender with the result. Otherwise, if the API partially fails (doesn't satisfy its post-conditions, but some incomplete result is available), the returned range would have *two* senders: the first containing the partial result, and the second containing the error. Such an API might be used in a coroutine as follows:

```
// Declare a buffer for read_socket_async to fill in
array<byte, 1024> buff;

for (auto snd : read_socket_async(socket, span{buff})) {
  try {
    if (optional<size_t> bytes_read =
          co_await execution::stopped_as_optional(std::move(snd))) {
      // OK, we read some bytes into buff. Process them here....
    } else {
      // The socket read was cancelled and returned no data. React
      // appropriately.
    }
  } catch (...) {
    // read_socket_async failed to meet its post-conditions.
    // Do some cleanup and propagate the error...
  }
}
```

Finally, it's possible to combine these two approaches when the API can both partially succeed (meeting its post-conditions) and partially fail (not meeting its post-conditions).

§ 4.15. All awaitables are senders

Since C++20 added coroutines to the standard, we expect that coroutines and awaitables will be how a great many will choose to express their asynchronous code. However, in this paper, we are proposing to add a suite of asynchronous algorithms that accept senders, not awaitables. One might wonder whether and how these algorithms will be accessible to those who choose coroutines instead of senders.

In truth there will be no problem because all generally awaitable types automatically model the `sender` concept. The adaptation is transparent and happens in the sender customization points, which are aware of awaitables. (By "generally awaitable" we mean types that don't require custom `await_transform` trickery from a promise type to make them awaitable.)

For an example, imagine a coroutine type called `task<T>` that knows nothing about senders. It doesn't implement any of the sender customization points. Despite that fact, and despite the fact that the `this_thread::sync_wait` algorithm is constrained with the `sender` concept, the following would compile and do what the user wants:

```
task<int> doSomeAsyncWork();

int main() {
  // OK, awaitable types satisfy the requirements for senders:
  auto o = this_thread::sync_wait(doSomeAsyncWork());
}
```

Since awaitables are senders, writing a sender-based asynchronous algorithm is trivial if you have a coroutine task type: implement the algorithm as a coroutine. If you are not bothered by the possibility of allocations and indirections as a result of using coroutines, then there is no need to ever write a sender, a receiver, or an operation state.

§ 4.16. Many senders can be trivially made awaitable

If you choose to implement your sender-based algorithms as coroutines, you'll run into the issue of how to retrieve results from a passed-in sender. This is not a problem. If the coroutine type opts in to sender support -- trivial with the `execution::with_awaitable_senders` utility -- then a large class of senders are transparently awaitable from within the coroutine.

For example, consider the following trivial implementation of the sender-based `retry` algorithm:

```
template<class S>
  requires single-sender<S&> // see [exec.as.awaitable]
task<single-sender-value-type<S>> retry(S s) {
  for (;;) {
    try {
      co_return co_await s;
    } catch(...) {
    }
  }
}
```

Only *some* senders can be made awaitable directly because of the fact that callbacks are more expressive than coroutines. An awaitable expression has a single type: the result value of the async operation. In contrast, a callback can accept multiple arguments as the result of an operation. What's more, the callback can have overloaded function call signatures that take different sets of arguments. There is no way to automatically map such senders into awaitables. The `with_awaitable_senders` utility recognizes as awaitables those senders that send a single value of a single type. To await another kind of sender, a user would have to first map its value channel into a single value of a single type -- say, with the `into_variant` sender algorithm -- before `co_await`-ing that sender.

§ 4.17. Cancellation of a sender can unwind a stack of coroutines

When looking at the sender-based `retry` algorithm in the previous section, we can see that the value and error cases are correctly handled. But what about cancellation? What happens to a coroutine that is suspended awaiting a sender that completes by calling `execution::set_stopped`?

When your task type's promise inherits from `with_awaitable_senders`, what happens is this: the coroutine behaves as if an *uncatchable exception* had been thrown from the `co_await` expression. (It is not really an exception, but it's helpful to think of it that way.) Provided that the promise types of the calling coroutines also inherit from `with_awaitable_senders`, or more generally implement a member function called `unhandled_stopped`, the exception unwinds the chain of coroutines as if an exception were thrown except that it bypasses `catch(...)` clauses.

In order to "catch" this uncatchable stopped exception, one of the calling coroutines in the stack would have to await a sender that maps the stopped channel into either a value or an error. That is achievable with the `execution::let_stopped`, `execution::upon_stopped`, `execution::stopped_as_optional`, or `execution::stopped_as_error` sender adaptors. For instance, we can use `execution::stopped_as_optional` to "catch" the stopped signal and map it into an empty optional as shown below:

```
if (auto opt = co_await execution::stopped_as_optional(some_sender)) {
  // OK, some_sender completed successfully, and opt contains the result.
} else {
  // some_sender completed with a cancellation signal.
}
```

As described in the section "All awaitables are senders", the sender customization points recognize awaitables and adapt them transparently to model the sender concept. When `connect`-ing an awaitable and a receiver, the adaptation layer awaits the awaitable within a coroutine that implements `unhandled_stopped` in its promise type. The effect of this is that an "uncatchable" stopped exception propagates seamlessly out of awaitables, causing `execution::set_stopped` to be called on the receiver.

Obviously, `unhandled_stopped` is a library extension of the coroutine promise interface. Many promise types will not implement `unhandled_stopped`. When an uncatchable stopped exception tries to propagate through such a coroutine, it is treated as an unhandled exception and `terminate` is called. The solution, as described above, is to use a sender adaptor to handle the stopped exception before awaiting it. It goes without saying that any future Standard Library coroutine types ought to implement `unhandled_stopped`. The author of *Add lazy coroutine (coroutine task) type*, which proposes a standard coroutine task type, is in agreement.

§ 4.18. Composition with parallel algorithms

The C++ Standard Library provides a large number of algorithms that offer the potential for non-sequential execution via the use of execution policies. The set of algorithms with execution policy overloads are often referred to as "parallel algorithms", although additional policies are available.

Existing policies, such as `execution::par`, give the implementation permission to execute the algorithm in parallel. However, the choice of execution resources used to perform the work is left to the implementation.

We will propose a customization point for combining schedulers with policies in order to provide control over where work will execute.

```
template<class ExecutionPolicy>
unspecified executing_on(
    execution::scheduler auto scheduler,
    ExecutionPolicy && policy
);
```

This function would return an object of an unspecified type which can be used in place of an execution policy as the first argument to one of the parallel algorithms. The overload selected by that object should execute its computation as requested by `policy` while using `scheduler` to create any work to be run. The expression may be ill-formed if `scheduler` is not able to support the given policy.

The existing parallel algorithms are synchronous; all of the effects performed by the computation are complete before the algorithm returns to its caller. This remains unchanged with the `executing_on` customization point.

In the future, we expect additional papers will propose asynchronous forms of the parallel algorithms which (1) return senders rather than values or `void` and (2) where a customization point pairing a sender with an execution policy would similarly be used to obtain an object of unspecified type to be provided as the first argument to the algorithm.

§ 4.19. User-facing sender factories

A sender factory is an algorithm that takes no senders as parameters and returns a sender.

§ 4.19.1. `execution::schedule`

```
execution::sender auto schedule(
    execution::scheduler auto scheduler
);
```

Returns a sender describing the start of a task graph on the provided scheduler. See § 4.2 Schedulers represent execution resources.

```
execution::scheduler auto sch1 = get_system_thread_pool().scheduler();

execution::sender auto snd1 = execution::schedule(sch1);
// snd1 describes the creation of a new task on the system thread pool
```

§ 4.19.2. `execution::just`

```
execution::sender auto just(
    auto ...&& values
);
```

Returns a sender with no completion schedulers, which sends the provided values. The input values are decay-copied into the returned sender. When the returned sender is connected to a receiver, the values are moved into the operation state if the sender is an rvalue; otherwise, they are copied. Then xvalues referencing the values in the operation state are passed to the receiver's `set_value`.

```
execution::sender auto snd1 = execution::just(3.14);
execution::sender auto then1 = execution::then(snd1, [] (double d) {
  std::cout << d << "\n";
});

execution::sender auto snd2 = execution::just(3.14, 42);
execution::sender auto then2 = execution::then(snd2, [] (double d, int i) {
  std::cout << d << ", " << i << "\n";
});

std::vector v3{1, 2, 3, 4, 5};
execution::sender auto snd3 = execution::just(v3);
execution::sender auto then3 = execution::then(snd3, [] (std::vector<int>&& v3copy) {
  for (auto&& e : v3copy) { e *= 2; }
  return std::move(v3copy);
}
auto&& [v3copy] = this_thread::sync_wait(then3).value();
// v3 contains {1, 2, 3, 4, 5}; v3copy will contain {2, 4, 6, 8, 10}.

execution::sender auto snd4 = execution::just(std::vector{1, 2, 3, 4, 5});
execution::sender auto then4 = execution::then(std::move(snd4), [] (std::vector<int>&& v4
  for (auto&& e : v4) { e *= 2; }
  return std::move(v4);
});
auto&& [v4] = this_thread::sync_wait(std::move(then4)).value();
// v4 contains {2, 4, 6, 8, 10}. No vectors were copied in this example.
```

§ 4.19.3. `execution::just_error`

```
execution::sender auto just_error(
    auto && error
);
```

Returns a sender with no completion schedulers, which completes with the specified error. If the provided error is an lvalue reference, a copy is made inside the returned sender and a non-const lvalue reference to the copy is sent to the receiver's `set_error`. If the provided value is an rvalue reference, it is moved into the returned sender and an rvalue reference to it is sent to the receiver's `set_error`.

§ 4.19.4. `execution::just_stopped`

```
execution::sender auto just_stopped();
```

Returns a sender with no completion schedulers, which completes immediately by calling the receiver's `set_stopped`.

§ 4.19.5. `execution::read_env`

```
execution::sender auto read_env(auto tag);
```

Returns a sender that reaches into a receiver's environment and pulls out the current value associated with the customization point denoted by `Tag`. It then sends the value read back to the receiver through the value channel. For instance, `read_env(get_scheduler)` is a sender that asks the receiver for the currently suggested `scheduler` and passes it to the receiver's `set_value` completion-signal.

This can be useful when scheduling nested dependent work. The following sender pulls the current schduler into the value channel and then schedules more work onto it.

```
execution::sender auto task =
  execution::read_env(get_scheduler)
    | execution::let_value([](auto sched) {
        return execution::starts_on(sched, some nested work here);
    });

this_thread::sync_wait( std::move(task) ); // wait for it to finish
```

This code uses the fact that `sync_wait` associates a scheduler with the receiver that it connects with `task`. `read_env(get_scheduler)` reads that scheduler out of the receiver, and passes it to `let_value`'s receiver's

`set_value` function, which in turn passes it to the lambda. That lambda returns a new sender that uses the scheduler to schedule some nested work onto `sync_wait`'s scheduler.

§ 4.20. User-facing sender adaptors

A sender adaptor is an algorithm that takes one or more senders, which it may `execution::connect`, as parameters, and returns a sender, whose completion is related to the sender arguments it has received.

Sender adaptors are *lazy*, that is, they are never allowed to submit any work for execution prior to the returned sender being started later on, and are also guaranteed to not start any input senders passed into them. Sender consumers such as § 4.21.1 this_thread::sync_wait start senders.

For more implementer-centric description of starting senders, see § 5.5 Sender adaptors are lazy.

§ 4.20.1. `execution::continues_on`

```
execution::sender auto continues_on(
    execution::sender auto input,
    execution::scheduler auto scheduler
);
```

Returns a sender describing the transition from the execution agent of the input sender to the execution agent of the target scheduler. See § 4.6 Execution resource transitions are explicit.

```
execution::scheduler auto cpu_sched = get_system_thread_pool().scheduler();
execution::scheduler auto gpu_sched = cuda::scheduler();

execution::sender auto cpu_task = execution::schedule(cpu_sched);
// cpu_task describes the creation of a new task on the system thread pool

execution::sender auto gpu_task = execution::continues_on(cpu_task, gpu_sched);
// gpu_task describes the transition of the task graph described by cpu_task to the gpu
```

§ 4.20.2. `execution::then`

```
execution::sender auto then(
    execution::sender auto input,
    std::invocable<values-sent-by(input)...> function
);
```

`then` returns a sender describing the task graph described by the input sender, with an added node of invoking the provided function with the values sent by the input sender as arguments.

`then` is **guaranteed** to not begin executing `function` until the returned sender is started.

```
execution::sender auto input = get_input();
execution::sender auto snd = execution::then(input, [](auto... args) {
    std::print(args...);
});
// snd describes the work described by pred
// followed by printing all of the values sent by pred
```

This adaptor is included as it is necessary for writing any sender code that actually performs a useful function.

§ 4.20.3. `execution::upon_*`

```
execution::sender auto upon_error(
    execution::sender auto input,
    std::invocable<errors-sent-by(input)...> function
);

execution::sender auto upon_stopped(
    execution::sender auto input,
    std::invocable auto function
);
```

`upon_error` and `upon_stopped` are similar to `then`, but where `then` works with values sent by the input sender, `upon_error` works with errors, and `upon_stopped` is invoked when the "stopped" signal is sent.

§ 4.20.4. `execution::let_*`

```
execution::sender auto let_value(
    execution::sender auto input,
    std::invocable<values-sent-by(input)...> function
);

execution::sender auto let_error(
    execution::sender auto input,
    std::invocable<errors-sent-by(input)...> function
);

execution::sender auto let_stopped(
    execution::sender auto input,
```

```
    std::invocable auto function
);
```

`let_value` is very similar to `then`: when it is started, it invokes the provided function with the values sent by the input sender as arguments. However, where the sender returned from `then` sends exactly what that function ends up returning - `let_value` requires that the function return a sender, and the sender returned by `let_value` sends the values sent by the sender returned from the callback. This is similar to the notion of "future unwrapping" in future/promise-based frameworks.

`let_value` is **guaranteed** to not begin executing `function` until the returned sender is started.

`let_error` and `let_stopped` are similar to `let_value`, but where `let_value` works with values sent by the input sender, `let_error` works with errors, and `let_stopped` is invoked when the "stopped" signal is sent.

§ 4.20.5. `execution::starts_on`

```
execution::sender auto starts_on(
    execution::scheduler auto sched,
    execution::sender auto snd
);
```

Returns a sender which, when started, will start the provided sender on an execution agent belonging to the execution resource associated with the provided scheduler. This returned sender has no completion schedulers.

§ 4.20.6. `execution::into_variant`

```
execution::sender auto into_variant(
    execution::sender auto snd
);
```

Returns a sender which sends a variant of tuples of all the possible sets of types sent by the input sender. Senders can send multiple sets of values depending on runtime conditions; this is a helper function that turns them into a single variant value.

§ 4.20.7. `execution::stopped_as_optional`

```
execution::sender auto stopped_as_optional(
    single-sender auto snd
);
```

Returns a sender that maps the value channel from a `T` to an `optional<decay_t<T>>`, and maps the stopped channel to a value of an empty `optional<decay_t<T>>`.

§ 4.20.8. `execution::stopped_as_error`

```
template<move_constructible Error>
execution::sender auto stopped_as_error(
    execution::sender auto snd,
    Error err
);
```

Returns a sender that maps the stopped channel to an error of `err`.

§ 4.20.9. `execution::bulk`

```
execution::sender auto bulk(
    execution::sender auto input,
    std::integral auto shape,
    invocable<decltype(size), values-sent-by(input)...> function
);
```

Returns a sender describing the task of invoking the provided function with every index in the provided shape along with the values sent by the input sender. The returned sender completes once all invocations have completed, or an error has occurred. If it completes by sending values, they are equivalent to those sent by the input sender.

No instance of `function` will begin executing until the returned sender is started. Each invocation of `function` runs in an execution agent whose forward progress guarantees are determined by the scheduler on which they are run. All agents created by a single use of `bulk` execute with the same guarantee. The number of execution agents used by `bulk` is not specified. This allows a scheduler to execute some invocations of the `function` in parallel.

In this proposal, only integral types are used to specify the shape of the bulk section. We expect that future papers may wish to explore extensions of the interface to explore additional kinds of shapes, such as multi-dimensional grids, that are commonly used for parallel computing tasks.

§ 4.20.10. `execution::split`

```
execution::sender auto split(execution::sender auto sender);
```

If the provided sender is a multi-shot sender, returns that sender. Otherwise, returns a multi-shot sender which sends values equivalent to the values sent by the provided sender. See § 4.7 Senders can be either multi-shot or single-shot.

§ 4.20.11. `execution::when_all`

```
execution::sender auto when_all(
    execution::sender auto ...inputs
);

execution::sender auto when_all_with_variant(
    execution::sender auto ...inputs
);
```

`when_all` returns a sender that completes once all of the input senders have completed. It is constrained to only accept senders that can complete with a single set of values (_i.e._, it only calls one overload of `set_value` on its receiver). The values sent by this sender are the values sent by each of the input senders, in order of the arguments passed to `when_all`. It completes inline on the execution resource on which the last input sender completes, unless stop is requested before `when_all` is started, in which case it completes inline within the call to `start`.

`when_all_with_variant` does the same, but it adapts all the input senders using `into_variant`, and so it does not constrain the input arguments as `when_all` does.

The returned sender has no completion schedulers.

```
execution::scheduler auto sched = thread_pool.scheduler();

execution::sender auto sends_1 = ...;
execution::sender auto sends_abc = ...;

execution::sender auto both = execution::when_all(
    sends_1,
    sends_abc
);

execution::sender auto final = execution::then(both, [](auto... args){
    std::cout << std::format("the two args: {}, {}", args...);
});
// when final executes, it will print "the two args: 1, abc"
```

§ 4.21. User-facing sender consumers

A sender consumer is an algorithm that takes one or more senders, which it may `execution::connect`, as parameters, and does not return a sender.

§ 4.21.1. `this_thread::sync_wait`

```
auto sync_wait(
    execution::sender auto sender
) requires (always-sends-same-values(sender))
    -> std::optional<std::tuple<values-sent-by(sender)>>;
```

`this_thread::sync_wait` is a sender consumer that submits the work described by the provided sender for execution, blocking **the current `std::thread` or thread of `main`** until the work is completed, and returns an optional tuple of values that were sent by the provided sender on its completion of work. Where § 4.19.1 execution::schedule and § 4.19.2 execution::just are meant to *enter* the domain of senders, `sync_wait` is one way to *exit* the domain of senders, retrieving the result of the task graph.

If the provided sender sends an error instead of values, `sync_wait` throws that error as an exception, or rethrows the original exception if the error is of type `std::exception_ptr`.

If the provided sender sends the "stopped" signal instead of values, `sync_wait` returns an empty optional.

For an explanation of the `requires` clause, see § 5.8 All senders are typed. That clause also explains another sender consumer, built on top of `sync_wait`: `sync_wait_with_variant`.

> Note: This function is specified inside `std::this_thread`, and not inside `execution`. This is because `sync_wait` has to block the *current* execution agent, but determining what the current execution agent is is not reliable. Since the standard does not specify any functions on the current execution agent other than those in `std::this_thread`, this is the flavor of this function that is being proposed. If C++ ever obtains fibers, for instance, we expect that a variant of this function called `std::this_fiber::sync_wait` would be provided. We also expect that runtimes with execution agents that use different synchronization mechanisms than `std::thread`'s will provide their own flavors of `sync_wait` as well (assuming their execution agents have the means to block in a non-deadlock manner).

§ 5. Design - implementer side

§ 5.1. Receivers serve as glue between senders

A receiver is a callback that supports more than one channel. In fact, it supports three of them:

— `set_value`, which is the moral equivalent of an `operator()` or a function call, which signals successful completion of the operation its execution depends on;

— `set_error`, which signals that an error has happened during scheduling of the current work, executing

the current work, or at some earlier point in the sender chain; and

— `set_stopped`, which signals that the operation completed without succeeding (`set_value`) and without failing (`set_error`). This result is often used to indicate that the operation stopped early, typically because it was asked to do so because the result is no longer needed.

Once an async operation has been started exactly one of these functions must be invoked on a receiver before it is destroyed.

While the receiver interface may look novel, it is in fact very similar to the interface of `std::promise`, which provides the first two signals as `set_value` and `set_exception`, and it's possible to emulate the third channel with lifetime management of the promise.

Receivers are not a part of the end-user-facing API of this proposal; they are necessary to allow unrelated senders communicate with each other, but the only users who will interact with receivers directly are authors of senders.

Receivers are what is passed as the second argument to § 5.3 execution::connect.

§ 5.2. Operation states represent work

An operation state is an object that represents work. Unlike senders, it is not a chaining mechanism; instead, it is a concrete object that packages the work described by a full sender chain, ready to be executed. An operation state is neither movable nor copyable, and its interface consists of a single algorithm: `start`, which serves as the submission point of the work represented by a given operation state.

Operation states are not a part of the user-facing API of this proposal; they are necessary for implementing sender consumers like `this_thread::sync_wait`, and the knowledge of them is necessary to implement senders, so the only users who will interact with operation states directly are authors of senders and authors of sender algorithms.

The return value of § 5.3 execution::connect must satisfy the operation state concept.

§ 5.3. `execution::connect`

`execution::connect` is a customization point which connects senders with receivers, resulting in an operation state that will ensure that if `start` is called that one of the completion operations will be called on the receiver passed to `connect`.

```
execution::sender auto snd = some input sender;
execution::receiver auto rcv = some receiver;
execution::operation_state auto state = execution::connect(snd, rcv);

execution::start(state);
```

```
// at this point, it is guaranteed that the work represented by state has been submitted
// to an execution resource, and that execution resource will eventually call one of the
// completion operations on rcv

// operation states are not movable, and therefore this operation state object must be
// kept alive until the operation finishes
```

§ 5.4. Sender algorithms are customizable

Senders being able to advertise what their completion schedulers are fulfills one of the promises of senders: that of being able to customize an implementation of a sender algorithm based on what scheduler any work it depends on will complete on.

The simple way to provide customizations for functions like `then`, that is for sender adaptors and sender consumers, is to follow the customization scheme that has been adopted for C++20 ranges library; to do that, we would define the expression `execution::then(sender, invocable)` to be equivalent to:

1. `sender.then(invocable)`, if that expression is well-formed; otherwise
2. `then(sender, invocable)`, performed in a context where this call always performs ADL, if that expression is well-formed; otherwise
3. a default implementation of `then`, which returns a sender adaptor, and then define the exact semantics of said adaptor.

However, this definition is problematic. Imagine another sender adaptor, `bulk`, which is a structured abstraction for a loop over an index space. Its default implementation is just a for loop. However, for accelerator runtimes like CUDA, we would like sender algorithms like `bulk` to have specialized behavior, which invokes a kernel of more than one thread (with its size defined by the call to `bulk`); therefore, we would like to customize `bulk` for CUDA senders to achieve this. However, there's no reason for CUDA kernels to necessarily customize the `then` sender adaptor, as the generic implementation is perfectly sufficient. This creates a problem, though; consider the following snippet:

```
execution::scheduler auto cuda_sch = cuda_scheduler{};

execution::sender auto initial = execution::schedule(cuda_sch);
// the type of initial is a type defined by the cuda_scheduler
// let's call it cuda::schedule_sender<>

execution::sender auto next = execution::then(cuda_sch, []{ return 1; });
// the type of next is a standard-library unspecified sender adaptor
// that wraps the cuda sender
// let's call it execution::then_sender_adaptor<cuda::schedule_sender<>>

execution::sender auto kernel_sender = execution::bulk(next, shape, [](int i){ ... });
```

How can we specialize the `bulk` sender adaptor for our wrapped `schedule_sender`? Well, here's one possible approach, taking advantage of ADL (and the fact that the definition of "associated namespace" also recursively enumerates the associated namespaces of all template parameters of a type):

```
namespace cuda::for_adl_purposes {
template<typename... SentValues>
class schedule_sender {
    execution::operation_state auto connect(execution::receiver auto rcv);
    execution::scheduler auto get_completion_scheduler() const;
};

execution::sender auto bulk(
    execution::sender auto && input,
    execution::shape auto && shape,
    invocable%lt;sender-values(input)> auto && fn)
{
    // return a cuda sender representing a bulk kernel launch
}
} // namespace cuda::for_adl_purposes
```

However, if the input sender is not just a `then_sender_adaptor` like in the example above, but another sender that overrides `bulk` by itself, as a member function, because its author believes they know an optimization for bulk - the specialization above will no longer be selected, because a member function of the first argument is a better match than the ADL-found overload.

This means that well-meant specialization of sender algorithms that are entirely scheduler-agnostic can have negative consequences. The scheduler-specific specialization - which is essential for good performance on platforms providing specialized ways to launch certain sender algorithms - would not be selected in such cases. But it's really the scheduler that should control the behavior of sender algorithms when a non-default implementation exists, not the sender. Senders merely describe work; schedulers, however, are the handle to the runtime that will eventually execute said work, and should thus have the final say in *how* the work is going to be executed.

Therefore, we are proposing the following customization scheme: the expression `execution::<sender-algorithm>(sender, args...)`, for any given sender algorithm that accepts a sender as its first argument, should do the following:

1. Create a sender that implements the default implementation of the sender algorithm. That sender is tuple-like; it can be destructured into its constituent parts: algorithm tag, data, and child sender(s).
2. We query the child sender for its *domain*. A **domain** is a tag type associated with the scheduler that the child sender will complete on. If there are multiple child senders, we query all of them for their domains and require that they all be the same.
3. We use the domain to dispatch to a `transform_sender` customization, which accepts the sender and optionally performs a domain-specific transformation on it. This customization is expected to return a new

sender, which will be returned from `<sender-algorithm>` in place of the original sender.

§ 5.5. Sender adaptors are lazy

Contrary to early revisions of this paper, we propose to make all sender adaptors perform strictly lazy submission, unless specified otherwise.

Strictly lazy submission means that there is a guarantee that no work is submitted to an execution resource before a receiver is connected to a sender, and `execution::start` is called on the resulting operation state.

§ 5.6. Lazy senders provide optimization opportunities

Because lazy senders fundamentally *describe* work, instead of describing or representing the submission of said work to an execution resource, and thanks to the flexibility of the customization of most sender algorithms, they provide an opportunity for fusing multiple algorithms in a sender chain together, into a single function that can later be submitted for execution by an execution resource. There are two ways this can happen.

The first (and most common) way for such optimizations to happen is thanks to the structure of the implementation: because all the work is done within callbacks invoked on the completion of an earlier sender, recursively up to the original source of computation, the compiler is able to see a chain of work described using senders as a tree of tail calls, allowing for inlining and removal of most of the sender machinery. In fact, when work is not submitted to execution resources outside of the current thread of execution, compilers are capable of removing the senders abstraction entirely, while still allowing for composition of functions across different parts of a program.

The second way for this to occur is when a sender algorithm is specialized for a specific set of arguments. For instance, an implementation could recognize two subsequent § 4.20.9 execution::bulks of compatible shapes, and merge them together into a single submission of a GPU kernel.

§ 5.7. Execution resource transitions are two-step

Because `execution::continues_on` takes a sender as its first argument, it is not actually directly customizable by the target scheduler. This is by design: the target scheduler may not know how to transition *from* a scheduler such as a CUDA scheduler; transitioning away from a GPU in an efficient manner requires making runtime calls that are specific to the GPU in question, and the same is usually true for other kinds of accelerators too (or for scheduler running on remote systems). To avoid this problem, specialized schedulers like the ones mentioned here can still hook into the transition mechanism, and inject a sender which will perform a transition to the regular CPU execution resource, so that any sender can be attached to it.

This, however, is a problem: because customization of sender algorithms must be controlled by the scheduler they will run on (see § 5.4 Sender algorithms are customizable), the type of the sender returned from

`continues_on` must be controllable by the target scheduler. Besides, the target scheduler may itself represent a specialized execution resource, which requires additional work to be performed to transition *to* it. GPUs and remote node schedulers are once again good examples of such schedulers: executing code on their execution resources requires making runtime API calls for work submission, and quite possibly for the data movement of the values being sent by the input sender passed into `continues_on`.

To allow for such customization from both ends, we propose the inclusion of a secondary transitioning sender adaptor, called `schedule_from`. This adaptor is a form of `schedule`, but takes an additional, second argument: the input sender. This adaptor is not meant to be invoked manually by the end users; they are always supposed to invoke `continues_on`, to ensure that both schedulers have a say in how the transitions are made. Any scheduler that specializes `continues_on(snd, sch)` shall ensure that the return value of their customization is equivalent to `schedule_from(sch, snd2)`, where `snd2` is a successor of `snd` that sends values equivalent to those sent by `snd`.

The default implementation of `continues_on(snd, sched)` is `schedule_from(sched, snd)`.

§ 5.8. All senders are typed

All senders must advertise the types they will send when they complete. There are many sender adaptors that need this information. Even just transitioning from one execution context to another requires temporarily storing the async result data so it can be propagated in the new execution context. Doing that efficiently requires knowing the type of the data.

The mechanism a sender uses to advertise its completions is the `get_completion_signatures` customization point, which takes an environment and must return a specialization of the `execution::completion_signatures` class template. The template parameters of `execution::completion_signatures` is a list of function types that represent the completion operations of the sender. for example, the type `execution::set_value_t(size_t, const char*)` indicates that the sender can complete successfully by passing a `size_t` and a `const char*` to the receiver's `set_value` function.

This proposal includes utilities for parsing and manipulating the list of a sender's completion signatures. For instance, `values_of_t` is a template alias for accessing a sender's value completions. It takes a sender, an environment, and two variadic template template parameters: a tuple-like template and a variant-like template. You can get the value completions of `S` and `Env` with `value_types_of_t<S, Env, tuple-like, variant-like>`. For example, for a sender that can complete successfully with either `Ts...` or `Us...`, `value_types_of_t<S, Env, std::tuple, std::variant>` would name the type `std::variant<std::tuple<Ts...>, std::tuple<Us...>>`.

§ 5.9. Customization points

Earlier versions of this paper used a dispatching technique known as `tag_invoke` (see *tag_invoke: A general pattern for supporting customisable functions*) to allow for customization of basis oper-

ations and sender algorithms. This technique used private friend functions named "`tag_invoke`" that are found by argument-dependent look-up. The `tag_invoke` overloads are distinguished from each other by their first argument, which is the type of the customization point object being customized. For instance, to customize the `execution::set_value` operation, a receiver type might do the following:

```
struct my_receiver {
  friend void tag_invoke(execution::set_value_t, my_receiver&& self, int value) noexcept
    std::cout << "received value: " << value;
  }
  //...
};
```

The `tag_invoke` technique, although it had its strengths, has been replaced with a new (or rather, a very old) technique that uses explicit concept opt-ins and named member functions. For instance, the `execution::set_value` operation is now customized by defining a member function named `set_value` in the receiver type. This technique is more explicit and easier to understand than `tag_invoke`. This is what a receiver author would do to customize `execution::set_value` now:

```
struct my_receiver {
  using receiver_concept = execution::receiver_t;

  void set_value(int value) && noexcept {
    std::cout << "received value: " << value;
  }
  //...
};
```

The only exception to this is the customization of queries. There is a need to build queryable adaptors that can forward an open and unknowable set of queries to some wrapped object. This is done by defining a member function named `query` in the adaptor type that takes the query CPO object as its first (and usually only) argument. A queryable adaptor might look like this:

```
template<class Query, class Queryable, class... Args>
concept query_for =
  requires (const Queryable& o, Args&&... args) {
    o.query(Query(), (Args&&) args...);
  };

template<class Allocator = std::allocator<>,
        class Base = execution::empty_env>
struct with_allocator {
  Allocator alloc{};
  Base base{};
```

```
  // Forward unknown queries to the wrapped object:
  template<query_for<Base> Query>
  decltype(auto) query(Query q) const {
    return base.query(q);
  }

  // Specialize the query for the allocator:
  Allocator query(execution::get_allocator_t) const {
    return alloc;
  }
};
```

Customization of sender algorithms such as `execution::then` and `execution::bulk` are handled differently because they must dispatch based on where the sender is executing. See the section on § 5.4 Sender algorithms are customizable for more information.

§ 6. Specification

Much of this wording follows the wording of *A Unified Executors Proposal for C++*.

§ 22 General utilities library [utilities] is meant to be a diff relative to the wording of the **[utilities]** clause of *Working Draft, Standard for Programming Language C++*.

§ 33 Concurrency support library [thread] is meant to be a diff relative to the wording of the **[thread]** clause of *Working Draft, Standard for Programming Language C++*. This diff applies changes from *Composable cancellation for sender-based async operations*.

§ 34 Execution control library [exec] is meant to be added as a new library clause to the working draft of C++.

§ 14. Exception handling **[except]**

§ 14.6. Special functions **[except.special]**

§ **14.6.2. The `std::terminate` function [except.terminate]**

[Editorial note: At the end of the bulleted list in the Note in paragraph 1, add a new bullet as follows: — *end note]*

(1.13) — when a call to a `wait()`, `wait_until()`, or `wait_for()` function on a condition variable (33.7.4, 33.7.5) fails to meet a postcondition.

(1.14) — when a callback invocation exits via an exception when requesting stop on a `std::stop_source` or a `std::inplace_stop_source` ([stopsource.mem], [stopsource.inplace.mem]), or in the constructor of `std::stop_callback` or `std::inplace_stop_callback` ([stopcallback.cons], [stopcallback.inplace.cons]) when a callback invocation exits via an exception.

(1.15) — when a `run_loop` object is destroyed that is still in the *`running`* state ([exec.run.loop]).

(1.16) — when `unhandled_stopped()` is called on a `with_awaitable_senders<T>` object ([exec.with.awaitable.senders]) whose continuation is not a handle to a coroutine whose promise type has an `unhandled_stopped()` member function.

§ 16. Library introduction **[library]**

[Editorial note: At the end of [expos.only.entity], add the following: — *end note]*

2. The following are defined for exposition only to aid in the specification of the library:

```
namespace std {
  // ...as before...
}
```

3. An object `dst` is said to be *decay-copied from* a subexpression `src` if the type of `dst` is `decay_t<decltype((src))>`, and `dst` is copy-initialized from `src`.

§ 16.4.4.6.1. General **[allocator.requirements.general]**

[Editorial note: At the end of [allocator.requirements.general], add the following new paragraph: — *end note]*

98. [*Example 2*: The following is an allocator class template supporting the minimal interface that meets the requirements of [allocator.requirements.general]:

```
template<class T>
struct SimpleAllocator {
  using value_type = T;
  SimpleAllocator(ctor args);

  template<class U> SimpleAllocator(const SimpleAllocator<U>& other);

  T* allocate(std::size_t n);
  void deallocate(T* p, std::size_t n);
```

```
  template<class U> bool operator==(const SimpleAllocator<U>& rhs) const;
};
```

-- *end example*]

99. The following exposition-only concept defines the minimal requirements on an *Allocator* type.

```
template<class Alloc>
concept simple-allocator =
  requires(Alloc alloc, size_t n) {
    { *alloc.allocate(n) } -> same_as<typename Alloc::value_type&>;
    { alloc.deallocate(alloc.allocate(n), n) };
  } &&
  copy_constructible<Alloc> &&
  equality_comparable<Alloc>;
```

1. A type `Alloc` models *`simple-allocator`* if it meets the requirements of [allocator.requirements.general].

§ 17. Language support library **[cpp]**

§ 17.3. Implementation properties **[support.limits]**

§ **17.3.2. Header `<version>` synopsis [version.syn]**

[Editorial note: To the `<version>` synopsis, add the following: — *end note]*

```
#define __cpp_lib_semaphore       201907L         // also in <semaphore>
#define __cpp_lib_senders         2024XXL         // also in <execution>
#define __cpp_lib_shared_mutex    201505L         // also in <shared_mutex>
```

§ 22. General utilities library **[utilities]**

§ 22.10. Function objects **[function.objects]**

§ **22.10.2. Header `<functional>` synopsis [functional.syn]**

[Editorial note: At the end of this subclause, insert the following declarations into the synopsis within `namespace std`: — *end note]*

```
namespace std {
  // ...as before...

  namespace ranges {
    // 22.10.9, concept-constrained comparisons
    struct equal_to;                                    // freestanding
    struct not_equal_to;                                // freestanding
    struct greater;                                     // freestanding
    struct less;                                        // freestanding
    struct greater_equal;                               // freestanding
    struct less_equal;                                  // freestanding
  }

  template<class Fn, class... Args>
    concept callable =  // exposition only
      requires (Fn&& fn, Args&&... args) {
        std::forward<Fn>(fn)(std::forward<Args>(args)...);
      };
  template<class Fn, class... Args>
    concept nothrow-callable =   // exposition only
      callable<Fn, Args...> &&
      requires (Fn&& fn, Args&&... args) {
        { std::forward<Fn>(fn)(std::forward<Args>(args)...) } noexcept;
      };
  // exposition only:
  template<class Fn, class... Args>
    using call-result-t = decltype(declval<Fn>()(declval<Args>()...));

  template<const auto& Tag>
    using decayed-typeof = decltype(auto(Tag)); // exposition only

}
```

§ 33. Concurrency support library **[thread]**

§ 33.3. Stop tokens **[thread.stoptoken]**

§ 33.3.1. Introduction [thread.stoptoken.intro]

1. Subclause [thread.stoptoken] describes components that can be used to asynchronously request that an operation stops execution in a timely manner, typically because the result is no longer required.

Such a request is called a *stop request*.

2. ~~`stop_source`, `stop_token`, and `stop_callback` implement~~ *`stoppable-source`*, `stoppable_token`, and *`stoppable-callback-for`* are concepts that specify the required syntax and semantics of shared ~~ownership of~~ access to a *stop state*. ~~Any `stop_source`, `stop_token`, or `stop_callback` object that shares ownership of the same stop state is an ***associated*** `stop_source`, `stop_token`, or `stop_callback`, respectively.~~ Any object modeling *`stoppable-source`*, `stoppable_token`, or *`stoppable-callback-for`* that refers to the same stop state is an *associated* *`stoppable-source`*, `stoppable_token`, or *`stoppable-callback-for`*, respectively. ~~The last remaining owner of the stop state automatically releases the resources associated with the stop state.~~

3. A n object of a type that models `stoppable_token` can be passed to an operation ~~which~~ that can either

   (3.1) — actively poll the token to check if there has been a stop request, or

   (3.2) — register a callback ~~using the `stop_callback` class template which~~ that will be called in the event that a stop request is made.

   A stop request made via ~~a `stop_source`~~ an object whose type models *`stoppable-source`* will be visible to all associated `stoppable_token` and ~~`stop_source`~~ *`stoppable-source`* objects. Once a stop request has been made it cannot be withdrawn (a subsequent stop request has no effect).

4. Callbacks registered via ~~a `stop_callback` object~~ an object whose type models *`stoppable-callback-for`* are called when a stop request is first made by any associated ~~`stop_source`~~ *`stoppable-source`* object.

[Editorial note: The following paragraph is moved to the specification of the new *`stoppable-source`* concept. *— end note]*

5. ~~Calls to the functions `request_stop`, `stop_requested`, and `stop_possible` do not introduce data races. A call to `request_stop` that returns `true` synchronizes with a call to `stop_requested` on an associated `stop_token` or `stop_source` object that returns `true`. Registration of a callback synchronizes with the invocation of that callback.~~

5. The types `stop_source` and `stop_token` and the class template `stop_callback` implement the semantics of shared ownership of a stop state. The last remaining owner of the stop state automatically releases the resources associated with the stop state.

6. An object of type `inplace_stop_source` is the sole owner of its stop state. An object of type `inplace_stop_token` or of a specialization of the class template `inplace_stop_callback` does not participate in ownership of its associated stop state. *[Note:* They are for use when all uses of the associated token and callback objects are known to nest within the lifetime of the `inplace_stop_source` object. *— end note]*

§ 33.3.2. Header <`stop_token`> synopsis [thread.stoptoken.syn]

[Editorial note: In this subclause, insert the following declarations into the `<stop_token>` synopsis: *— end note]*

```
namespace std {
  // [stoptoken.concepts], stop token concepts
  template<class CallbackFn, class Token, class Initializer = CallbackFn>
    concept stoppable-callback-for = see below; // exposition only

  template<class Token>
    concept stoppable_token = see below;

  template<class Token>
    concept unstoppable_token = see below;

  template<class Source>
    concept stoppable-source = see below; // exposition only

  // 33.3.3, class stop_token
  class stop_token;

  // 33.3.4, class stop_source
  class stop_source;

  // no-shared-stop-state indicator
  struct nostopstate_t {
    explicit nostopstate_t() = default;
  };
  inline constexpr nostopstate_t nostopstate{};

  // 33.3.5, class template stop_callback
  template<class CallbackFn>
    class stop_callback;

  // [stoptoken.never], class never_stop_token
  class never_stop_token;

  // [stoptoken.inplace], class inplace_stop_token
  class inplace_stop_token;

  // [stopsource.inplace], class inplace_stop_source
  class inplace_stop_source;

  // [stopcallback.inplace], class template inplace_stop_callback
  template<class CallbackFn>
    class inplace_stop_callback;

  template<class T, class CallbackFn>
```

```
    using stop_callback_for_t = T::template callback_type<CallbackFn>;

}
```

[Editorial note: Insert the following subclause as a new subclause between Header `<stop_token>` synopsis **[thread.stoptoken.syn]** and Class `stop_token` **[stoptoken]**. — *end note]*

§ 33.3.3. Stop token concepts [stoptoken.concepts]

1. The exposition-only *`stoppable-callback-for`* concept checks for a callback compatible with a given `Token` type.

```
template<class CallbackFn, class Token, class Initializer = CallbackFn>
  concept stoppable-callback-for = // exposition only
    invocable<CallbackFn> &&
    constructible_from<CallbackFn, Initializer> &&
    requires { typename stop_callback_for_t<Token, CallbackFn>; } &&
    constructible_from<stop_callback_for_t<Token, CallbackFn>, const Token&,
```

2. Let `t` and `u` be distinct, valid objects of type `Token` that reference the same logical stop state; let `init` be an expression such that `same_as<decltype(init), Initializer>` is `true`; and let `SCB` denote the type `stop_callback_for_t<Token, CallbackFn>`.
3. The concept *`stoppable-callback-for`*`<CallbackFn, Token, Initializer>` is modeled only if:
    1. The following concepts are modeled:
        — `constructible_from<SCB, Token, Initializer>`
        — `constructible_from<SCB, Token&, Initializer>`
        — `constructible_from<SCB, const Token, Initializer>`
    2. An object of type `SCB` has an associated *callback function* of type `CallbackFn`. Let `scb` be an object of type `SCB` and let `callback_fn` denote `scb`'s associated callback function. Direct-non-list-initializing `scb` from arguments `t` and `init` shall execute a *stoppable callback registration* as follows:
        1. If `t.stop_possible()` is `true`:
            1. `callback_fn` shall be direct-initialized with `init`.
            2. Construction of `scb` shall only throw exceptions thrown by the initialization of `callback_fn` from `init`.
            3. The callback invocation `std::forward<CallbackFn>(callback_fn)()` shall be registered with `t`'s associated stop state as follows:

1. If `t.stop_requested()` evaluates to `false` at the time of registration, the callback invocation is added to the stop state's list of callbacks such that `std::forward<CallbackFn>(callback_fn)()` is evaluated if a stop request is made on the stop state.
2. Otherwise, `std::forward<CallbackFn>(callback_fn)()` shall be immediately evaluated on the thread executing `scb`'s constructor, and the callback invocation shall not be added to the list of callback invocations.

4. If the callback invocation was added to stop state's list of callbacks, `scb` shall be associated with the stop state.

2. *[Note:* If `t.stop_possible()` is `false`, there is no requirement that the initialization of `scb` causes the initialization of `callback_fn`. — *end note]*

3. Destruction of `scb` shall execute a *stoppable callback deregistration* as follows (in order):

1. If the constructor of `scb` did not register a callback invocation with `t`'s stop state, then the stoppable callback deregistration shall have no effect other than destroying `callback_fn` if it was constructed.
2. Otherwise, the invocation of `callback_fn` shall be removed from the associated stop state.
3. If `callback_fn` is concurrently executing on another thread then the stoppable callback deregistration shall block ([defns.block]) until the invocation of `callback_fn` returns such that the return from the invocation of `callback_fn` strongly happens before ([intro.races]) the destruction of `callback_fn`.
4. If `callback_fn` is executing on the current thread, then the destructor shall not block waiting for the return from the invocation of `callback_fn`.
5. A stoppable callback deregistration shall not block on the completion of the invocation of some other callback registered with the same logical stop state.
6. The stoppable callback deregistration shall destroy `callback_fn`.

4. The `stoppable_token` concept checks for the basic interface of a stop token that is copyable and allows polling to see if stop has been requested and also whether a stop request is possible. The `unstoppable_token` concept checks for a `stoppable_token` type that does not allow stopping.

```
template<template<class> class>
  struct check-type-alias-exists; // exposition-only

template<class Token>
  concept stoppable_token =
    requires (const Token tok) {
      typename check-type-alias-exists<Token::template callback_type>;
      { tok.stop_requested() } noexcept -> same_as<bool>;
      { tok.stop_possible() } noexcept -> same_as<bool>;
```

```
      { Token(tok) } noexcept; // see implicit expression variations
                               // ([concepts.equality])
    } &&
    copyable<Token> &&
    equality_comparable<Token> &&
    swappable<Token>;

template<class Token>
  concept unstoppable_token =
    stoppable_token<Token> &&
    requires (const Token tok) {
      requires bool_constant<(!tok.stop_possible())>::value;
    };
```

5. An object whose type models `stoppable_token` has at most one associated logical stop state. A `stoppable_token` object with no associated stop state is said to be *disengaged*.
6. Let `SP` be an evaluation of `t.stop_possible()` that is `false`, and let `SR` be an evaluation of `t.stop_requested()` that is `true`.
7. The type `Token` models `stoppable_token` only if:
    1. Any evaluation of `u.stop_possible()` or `u.stop_requested()` that happens after ([intro.races]) `SP` is `false`.
    2. Any evaluation of `u.stop_possible()` or `u.stop_requested()` that happens after `SR` is `true`.
    3. For any types `CallbackFn` and `Initializer` such that *`stoppable-callback-for`*`<CallbackFn, Token, Initializer>` is satisfied, *`stoppable-callback-for`*`<CallbackFn, Token, Initializer>` is modeled.
    4. If `t` is disengaged, evaluations of `t.stop_possible()` and `t.stop_requested()` are `false`.
    5. If `t` and `u` reference the same stop state, or if both `t` and `u` are disengaged, `t == u` is `true`; otherwise, it is `false`.
8. An object whose type models the exposition-only *`stoppable-source`* concept can be queried whether stop has been requested (`stop_requested`) and whether stop is possible (`stop_possible`). It is a factory for associated stop tokens (`get_token`), and a stop request can be made on it (`request_stop`). It maintains a list of registered stop callback invocations that it executes when a stop request is first made.

```
template<class Source>
  concept stoppable-source = // exposition only
    requires (Source& src, const Source csrc) { // see implicit expression va
                                                // ([concepts.equality])
      { csrc.get_token() } -> stoppable_token;
      { csrc.stop_possible() } noexcept -> same_as<bool>;
```

```
    { csrc.stop_requested() } noexcept -> same_as<bool>;
    { src.request_stop() } -> same_as<bool>;
  };
```

1. An object whose type models *`stoppable-source`* has at most one associated logical stop state. If it has no associated stop state, it is said to be disengaged. Let `s` be an object whose type models *`stoppable-source`* and that is disengaged. `s.stop_possible()` and `s.stop_requested()` shall return `false`.
2. Let `t` be an object whose type models *`stoppable-source`*. If `t` is disengaged, `t.get_token()` shall return a disengaged stop token; otherwise, it shall return a stop token that is associated with the stop state of `t`.

[Editorial note: The following paragraph is moved from the introduction, with minor modifications (underlined in green). — *end note]*

3. Calls to the member functions `request_stop`, `stop_requested`, and `stop_possible` and similarly named member functions on associated `stoppable_token` objects do not introduce data races. A call to `request_stop` that returns `true` synchronizes with a call to `stop_requested` on an associated `stoppable_token` or ~~`stop_source`~~ *`stoppable-source`* object that returns `true`. Registration of a callback synchronizes with the invocation of that callback.

[Editorial note: The following paragraph is taken from § 33.3.5.3 Member functions [stopsource.mem] and modified. — *end note]*

4. If the *`stoppable-source`* is disengaged, `request_stop` shall have no effect and return `false`. Otherwise, it shall execute a *stop request operation* on the associated stop state. A stop request operation determines whether the stop state has received a stop request, and if not, makes a stop request. The determination and making of the stop request shall happen atomically, as-if by a read-modify-write operation ([intro.races]). If the request was made, the stop state's registered callback invocations shall be synchronously executed. If an invocation of a callback exits via an exception then `terminate` shall be invoked ([except.terminate]). *[Note:* No constraint is placed on the order in which the callback invocations are executed. — *end note]* `request_stop` shall return `true` if a stop request was made, and `false` otherwise. After a call to `request_stop` either a call to `stop_possible` shall return `false` or a call to `stop_requested` shall return `true`.

   [Note: A stop request includes notifying all condition variables of type `condition_variable_any` temporarily registered during an interruptible wait ([thread.condvarany.intwait]). — *end note]*

[Editorial note: Modify subclause **[stoptoken]** as follows: — *end note]*

§ 33.3.4. Class `stop_token` [stoptoken]

§ 33.3.4.1. General **[stoptoken.general]**

1. ~~The class `stop_token` provides an interface for querying whether a stop request has been made (`stop_requested`) or can ever be made (`stop_possible`) using an associated `stop_source` object ([stopsource]). A `stop_token` can also be passed to a `stop_callback` ([stopcallback]) constructor to register a callback to be called when a stop request has been made from an associated `stop_source`.~~ The class `stop_token` models the concept `stoppable_token`. It shares ownership of its stop state, if any, with its associated `stop_source` object ([stopsource]) and any `stop_token` objects to which it compares equal.

```
namespace std {
  class stop_token {
  public:
    template<class CallbackFn>
      using callback_type = stop_callback<CallbackFn>;

    // [stoptoken.cons], constructors, copy, and assignment
    stop_token() noexcept = default;

    stop_token(const stop_token&) noexcept;
    stop_token(stop_token&&) noexcept;
    stop_token& operator=(const stop_token&) noexcept;
    stop_token& operator=(stop_token&&) noexcept;
    ~stop_token();


    // [stoptoken.mem], Member functions
    void swap(stop_token&) noexcept;

    // [stoptoken.mem], stop handling
    [[nodiscard]] bool stop_requested() const noexcept;
    [[nodiscard]] bool stop_possible() const noexcept;

    bool operator==(const stop_token& rhs) const noexcept = default;
    [[nodiscard]] friend bool operator==(const stop_token& lhs, const stop_token& rh
    friend void swap(stop_token& lhs, stop_token& rhs) noexcept;
  private:
    shared_ptr<unspecified> stop-state{}; // exposition only
  };
}
```

1. *`stop-state`* refers to the `stop_token`'s associated stop state. A `stop_token` object is disengaged when *`stop-state`* is empty.

§ ~~33.3.4.2. Constructors, copy, and assignment **[stoptoken.cons]**~~

~~`stop_token() noexcept;`~~

1. ~~*Postconditions:* `stop_possible()` is `false` and `stop_requested()` is `false`. *[Note:* Because the created `stop_token` object can never receive a stop request, no resources are allocated for a stop state. — *end note]*~~

~~`stop_token(const stop_token& rhs) noexcept;`~~

2. ~~*Postconditions:* `*this == rhs` is `true`. *[Note:* `*this` and `rhs` share the ownership of the same stop state, if any. — *end note]*~~

~~`stop_token(stop_token&& rhs) noexcept;`~~

3. ~~*Postconditions:* `*this` contains the value of `rhs` prior to the start of construction and `rhs.stop_possible()` is `false`.~~

~~`~stop_token();`~~

4. ~~*Effects:* Releases ownership of the stop state, if any.~~

~~`stop_token& operator=(const stop_token& rhs) noexcept;`~~

5. ~~*Effects:* Equivalent to: `stop_token(rhs).swap(*this)`.~~
6. ~~*Returns:* `*this`.~~

~~`stop_token& operator=(stop_token&& rhs) noexcept;`~~

7. ~~*Effects:* Equivalent to: `stop_token(std::move(rhs)).swap(*this)`.~~
8. ~~*Returns:* `*this`.~~

[Editorial note: Move `swap` into [stoptoken.mem]: — *end note]*

§ 33.3.4.3. Member functions **[stoptoken.mem]**

```
void swap(stop_token& rhs) noexcept;
```

1. *Effects:* ~~Exchanges the values of `*this` and `rhs`.~~ Equivalent to: *`stop-state`*`.swap(rhs.`*`stop-state`*`)`.

~~[[nodiscard]]~~ `bool stop_requested() const noexcept;`

2. *Returns:* `true` if ~~`*this` has ownership of~~ *`stop-state`* refers to a stop state that has received a stop request; otherwise, `false`.

~~[[nodiscard]]~~ `bool stop_possible() const noexcept;`

3. *Returns:* `false` if:

(2.1) — `*this` ~~does not have ownership of a stop state~~ is disengaged , or

(2.2) — a stop request was not made and there are no associated `stop_source` objects; otherwise, `true`.

[Editorial note: The following are covered by the `equality_comparable` and `swappable` concepts. — *end note]*

§ ~~33.3.4.4. Non-member functions **[stoptoken.nonmembers]**~~

~~`[[nodiscard]] bool operator==(const stop_token& lhs, const stop_token& rhs) noexcept;`~~

1. ~~*Returns:* `true` if `lhs` and `rhs` have ownership of the same stop state or if both `lhs` and `rhs` do not have ownership of a stop state; otherwise `false`.~~

~~`friend void swap(stop_token& x, stop_token& y) noexcept;`~~

2. ~~*Effects:* Equivalent to: `x.swap(y)`.~~

§ **33.3.5. Class `stop_source` [stopsource]**

§ 33.3.5.1. General **[stopsource.general]**

1. ~~The class `stop_source` implements the semantics of making a stop request. A stop request made on a `stop_source` object is visible to all associated `stop_source` and `stop_token` ([thread.stoptoken]) objects. Once a stop request has been made it cannot be withdrawn (a subsequent stop request has no effect).~~

```
namespace std {
```

[Editorial note: The following definitions are already specified in the `<stop_token>` synopsis

~~`// no-shared-stop-state indicator`~~

```
  struct nostopstate_t {
    explicit nostopstate_t() = default;
  };
  inline constexpr nostopstate_t nostopstate{};


  class stop_source {
  public:
    // 33.3.4.2, constructors, copy, and assignment
    stop_source();
    explicit stop_source(nostopstate_t) noexcept; {}

    stop_source(const stop_source&) noexcept;
    stop_source(stop_source&&) noexcept;
    stop_source& operator=(const stop_source&) noexcept;
    stop_source& operator=(stop_source&&) noexcept;
    ~stop_source();


    // [stopsource.mem], Member functions
    void swap(stop_source&) noexcept;

    // 33.3.4.3, stop handling
    [[nodiscard]] stop_token get_token() const noexcept;
    [[nodiscard]] bool stop_possible() const noexcept;
    [[nodiscard]] bool stop_requested() const noexcept;
    bool request_stop() noexcept;

    bool operator==(const stop_source& rhs) const noexcept = default;
        [[nodiscard]] friend bool
      operator==(const stop_source& lhs, const stop_source& rhs) noexcept;
    friend void swap(stop_source& lhs, stop_source& rhs) noexcept;

  private:
    shared_ptr<unspecified> stop-state{}; // exposition only
  };
}
```

1. *stop-state* refers to the stop_source's associated stop state. A stop_source object is disengaged when *stop-state* is empty.
2. stop_source models *stoppable-source*, copyable, equality_comparable, and swappable.

§ 33.3.5.2. Constructors, copy, and assignment **[stopsource.cons]**

`stop_source();`

1. *Effects:* Initialises ~~`*this` to have ownership of~~ *`stop-state`* with a pointer to a new stop state.
2. *Postconditions:* `stop_possible()` is `true` and `stop_requested()` is `false`.
3. *Throws:* `bad_alloc` if memory cannot be allocated for the stop state.

~~`explicit stop_source(nostopstate_t) noexcept;`~~

4. ~~*Postconditions:* `stop_possible()` is `false` and `stop_requested()` is `false`. *[Note:* No resources are allocated for the state. — *end note]*~~

~~`stop_source(const stop_source& rhs) noexcept;`~~

5. ~~*Postconditions:* `*this` == rhs is `true`. *[Note:* `*this` and `rhs` share the ownership of the same stop state, if any. — *end note]*~~

~~`stop_source(stop_source&& rhs) noexcept;`~~

6. ~~*Postconditions:* `*this` contains the value of `rhs` prior to the start of construction and `rhs.stop_possible()` is `false`.~~

~~`~stop_source();`~~

7. ~~*Effects:* Releases ownership of the stop state, if any.~~

~~`stop_source& operator=(const stop_source& rhs) noexcept;`~~

8. ~~*Effects:* Equivalent to: `stop_source(rhs).swap(*this)`.~~
9. ~~*Returns:* `*this`.~~

~~`stop_source& operator=(stop_source&& rhs) noexcept;`~~

10. ~~*Effects:* Equivalent to: `stop_source(std::move(rhs)).swap(*this)`.~~
11. ~~*Returns:* `*this`.~~

[Editorial note: Move `swap` into [stopsource.mem]: — *end note]*

§ 33.3.5.3. Member functions **[stopsource.mem]**

```
void swap(stop_source& rhs) noexcept;
```

1. *Effects:* ~~Exchanges the values of `*this` and `rhs`~~ Equivalent to: `stop-state.swap(rhs.stop-state)` .

```
[[nodiscard]] stop_token get_token() const noexcept;
```

2. *Returns:* `stop_token()` if `stop_possible()` is `false`; otherwise a new associated `stop_token` object ; *i.e.*, its `stop-state` member is equal to the `stop-state` member of `*this` .

```
[[nodiscard]] bool stop_possible() const noexcept;
```

3. *Returns:* ~~`true` if `*this` has ownership of a stop state; otherwise, `false`~~ `stop-state != nullptr` .

```
[[nodiscard]] bool stop_requested() const noexcept;
```

4. *Returns:* `true` if ~~`*this` has ownership of~~ `stop-state` refers to a stop state that has received a stop request; otherwise, `false`.

```
bool request_stop() noexcept;
```

5. *Effects:* Executes a stop request operation ([stoptoken.concepts]) on the associated stop state, if any.

4. ~~*Effects:* If `*this` does not have ownership of a stop state, returns `false`. Otherwise, atomically determines whether the owned stop state has received a stop request, and if not, makes a stop request. The determination and making of the stop request are an atomic read-modify-write operation ([intro.races]). If the request was made, the callbacks registered by associated `stop_callback` objects are synchronously called. If an invocation of a callback exits via an exception then `terminate` is invoked ([except.terminate]).~~

   ~~*[Note:* A stop request includes notifying all condition variables of type `condition_variable_any` temporarily registered during an interruptible wait ([thread.condvarany.intwait]). *— end note]*~~

5. ~~*Postconditions:* `stop_possible()` is `false` or `stop_requested()` is `true`.~~

6. ~~*Returns:* `true` if this call made a stop request; otherwise `false`.~~

§ ~~33.3.5.4. Non-member functions~~ **[stopsource.nonmembers]**

```
[[nodiscard]] friend bool
  operator==(const stop_source& lhs, const stop_source& rhs) noexcept;
```

1. ~~*Returns:* `true` if `lhs` and `rhs` have ownership of the same stop state or if both `lhs` and `rhs` do not have ownership of a stop state; otherwise `false`.~~

~~`friend void swap(stop_source& x, stop_source& y) noexcept;`~~

2. ~~*Effects:* Equivalent to: `x.swap(y)`.~~

§ 33.3.6. Class template `stop_callback` [stopcallback]

§ 33.3.6.1. General [stopcallback.general]

1.

```
namespace std {
  template<class CallbackFn>
  class stop_callback {
  public:
    using callback_type = CallbackFn;

    // 33.3.5.2, constructors and destructor
    template<class CInitializer>
      explicit stop_callback(const stop_token& st, CInitializer&& cbinit)
        noexcept(is_nothrow_constructible_v<CallbackFn, CInitializer>);
    template<class CInitializer>
      explicit stop_callback(stop_token&& st, CInitializer&& cbinit)
        noexcept(is_nothrow_constructible_v<CallbackFn, CInitializer>);
    ~stop_callback();

    stop_callback(const stop_callback&) = delete;
    stop_callback(stop_callback&&) = delete;
    stop_callback& operator=(const stop_callback&) = delete;
    stop_callback& operator=(stop_callback&&) = delete;

  private:
    CallbackFn callbackcallback-fn; // exposition only
  };

  template<class CallbackFn>
    stop_callback(stop_token, CallbackFn) -> stop_callback<CallbackFn>;
}
```

2. *Mandates:* `stop_callback` is instantiated with an argument for the template parameter `CallbackFn` that satisfies both `invocable` and `destructible`.

3. ~~*Preconditions:* `stop_callback` is instantiated with an argument for the template parameter `Callback` that models both `invocable` and `destructible`.~~

3. *Remarks:* For a type `Initializer`, if *`stoppable-callback-for`*`<CallbackFn, stop_token, Initializer>` is satisfied, then *`stoppable-callback-for`*`<CallbackFn, stop_token, Initializer>` is modeled. The exposition-only *`callback-fn`* member is the associated callback function ([stoptoken.concepts]) of `stop_callback<CallbackFn>` objects.

§ 33.3.6.2. Constructors and destructor **[stopcallback.cons]**

```
template<class ~~C~~Initializer>
explicit stop_callback(const stop_token& st, ~~C~~Initializer&& ~~cb~~init)
  noexcept(is_nothrow_constructible_v<CallbackFn, ~~C~~Initializer>);
template<class ~~C~~Initializer>
explicit stop_callback(stop_token&& st, ~~C~~Initializer&& ~~cb~~init)
  noexcept(is_nothrow_constructible_v<CallbackFn, ~~C~~Initializer>);
```

1. *Constraints:* `CallbackFn` and ~~`C`~~`Initializer` satisfy `constructible_from<CallbackFn, ~~C~~Initializer>`.

2. ~~*Preconditions:* `Callback` and `C` model `constructible_from<Callback, C>`.~~

3. *Effects:* Initializes ~~`callback`~~*`callback-fn`* with `std::forward<~~C~~Initializer>(~~cb~~init)` and executes a stoppable callback registration ([stoptoken.concepts]) . ~~If `st.stop_requested()` is `true`, then `std::forward<Callback>(callback)()` is evaluated in the current thread before the constructor returns. Otherwise, if `st` has ownership of a stop state, acquires shared ownership of that stop state and registers the callback with that stop state such that `std::forward<Callback>(callback)()` is evaluated by the first call to `request_stop()` on an associated `stop_source`.~~ If a callback is registered with `st`'s shared stop state, then `*this` acquires shared ownership of that stop state.

4. ~~*Throws:* Any exception thrown by the initialization of `callback`.~~

5. ~~*Remarks:* If evaluating `std::forward<Callback>(callback)()` exits via an exception, then `terminate` is invoked ([except.terminate]).~~

```
~stop_callback();
```

6. *Effects:* ~~Unregisters the callback from the owned stop state, if any. The destructor does not block waiting for the execution of another callback registered by an associated `stop_callback`. If `callback` is concurrently executing on another thread, then the return from the invocation of `callback` strongly happens before ([intro.races]) `callback` is destroyed. If `callback` is executing on the current thread, then the destructor does not block ([defns.block]) waiting for the return from~~

~~the invocation of `callback`. Releases~~ Executes a stoppable callback deregistration ([stoptoken.concepts]) and releases ownership of the stop state, if any.

[Editorial note: Insert a new subclause, Class `never_stop_token` **[stoptoken.never]**, after subclause Class template `stop_callback` **[stopcallback]**, as a new subclause of Stop tokens **[thread.stoptoken]**. — *end note]*

§ 33.3.7. Class `never_stop_token` [stoptoken.never]

§ 33.3.7.1. General [stoptoken.never.general]

1. The class `never_stop_token` models the `unstoppable_token` concept. It provides a stop token interface, but also provides static information that a stop is never possible nor requested.

```
namespace std {
  class never_stop_token {
    struct callback-type { // exposition only
      explicit callback-type(never_stop_token, auto&&) noexcept {}
    };
  public:
    template<class>
      using callback_type = callback-type;

    static constexpr bool stop_requested() noexcept { return false; }
    static constexpr bool stop_possible() noexcept { return false; }

    bool operator==(const never_stop_token&) const = default;
  };
}
```

[Editorial note: Insert a new subclause, Class `inplace_stop_token` **[stoptoken.inplace]**, after the subclause added above, as a new subclause of Stop tokens **[thread.stoptoken]**. — *end note]*

§ 33.3.8. Class `inplace_stop_token` [stoptoken.inplace]

§ 33.3.8.1. General [stoptoken.inplace.general]

1. The class `inplace_stop_token` models the concept `stoppable_token`. It references the stop state of its associated `inplace_stop_source` object ([stopsource.inplace]), if any.

```
namespace std {
  class inplace_stop_token {
  public:
```

```
    template<class CallbackFn>
      using callback_type = inplace_stop_callback<CallbackFn>;

    inplace_stop_token() = default;
    bool operator==(const inplace_stop_token&) const = default;

    // [stoptoken.inplace.mem], member functions
    bool stop_requested() const noexcept;
    bool stop_possible() const noexcept;
    void swap(inplace_stop_token&) noexcept;

  private:
    const inplace_stop_source* stop-source = nullptr; // exposition only
  };
}
```

§ 33.3.8.2. Member functions **[stoptoken.inplace.members]**

`void swap(inplace_stop_token& rhs) noexcept;`

1. *Effects*: Exchanges the values of `stop-source` and `rhs.stop-source`.

`bool stop_requested() const noexcept;`

1. *Effects*: Equivalent to: `return stop-source != nullptr && stop-source->stop_requested();`
2. *[Note:* As specified in [basic.life], the behavior of `stop_requested()` is undefined unless the call strongly happens before the start of the destructor of the associated `inplace_stop_source`, if any. — *end note]*

`bool stop_possible() const noexcept;`

3. *Returns*: `stop-source != nullptr`.
4. *[Note:* As specified in [basic.stc.general], the behavior of `stop_possible()` is implementation-defined unless the call strongly happens before the end of the storage duration of the associated `inplace_stop_source` object, if any. — *end note]*

[Editorial note: Insert a new subclause, Class `inplace_stop_source` **[stopsource.inplace]**, after the subclause added above, as a new subclause of Stop tokens **[thread.stoptoken]**. — *end note]*

§ 33.3.9. Class `inplace_stop_source` [stopsource.inplace]

§ 33.3.9.1. General [stopsource.inplace.general]

1. The class `inplace_stop_source` models *`stoppable-source`*.

```
namespace std {
  class inplace_stop_source {
  public:
    // [stopsource.inplace.cons], constructors, copy, and assignment
    constexpr inplace_stop_source() noexcept;

    inplace_stop_source(inplace_stop_source&&) = delete;
    inplace_stop_source(const inplace_stop_source&) = delete;
    inplace_stop_source& operator=(inplace_stop_source&&) = delete;
    inplace_stop_source& operator=(const inplace_stop_source&) = delete;
    ~inplace_stop_source();

    //[stopsource.inplace.mem], stop handling
    constexpr inplace_stop_token get_token() const noexcept;
    static constexpr bool stop_possible() noexcept { return true; }
    bool stop_requested() const noexcept;
    bool request_stop() noexcept;
  };
}
```

§ 33.3.9.2. Constructors, copy, and assignment [stopsource.inplace.cons]

```
constexpr inplace_stop_source() noexcept;
```

1. *Effects*: Initializes a new stop state inside `*this`.
2. *Postconditions*: `stop_requested()` is `false`.

§ 33.3.9.3. Members [stopsource.inplace.mem]

```
constexpr inplace_stop_token get_token() const noexcept;
```

1. *Returns*: A new associated `inplace_stop_token` object. The `inplace_stop_token` object's *`stop-source`* member is equal to `this`.

```
bool stop_requested() const noexcept;
```

3. *Returns*: `true` if the stop state inside `*this` has received a stop request; otherwise, `false`.

```
bool request_stop() noexcept;
```

4. *Effects*: Executes a stop request operation ([stoptoken.concepts]).

5. *Postconditions*: `stop_requested()` is `true`.

[Editorial note: Insert a new subclause, Class template `inplace_stop_callback` **[stopcallback.inplace]**, after the subclause added above, as a new subclause of Stop tokens **[thread.stoptoken]**. *— end note]*

§ 33.3.10. Class template `inplace_stop_callback` [stopcallback.inplace]

§ 33.3.10.1. General [stopcallback.inplace.general]

```
namespace std {
  template<class CallbackFn>
  class inplace_stop_callback {
  public:
    using callback_type = CallbackFn;

    // [stopcallback.inplace.cons], constructors and destructor
    template<class Initializer>
      explicit inplace_stop_callback(inplace_stop_token st, Initializer&& init)
        noexcept(is_nothrow_constructible_v<CallbackFn, Initializer>);
    ~inplace_stop_callback();

    inplace_stop_callback(inplace_stop_callback&&) = delete;
    inplace_stop_callback(const inplace_stop_callback&) = delete;
    inplace_stop_callback& operator=(inplace_stop_callback&&) = delete;
    inplace_stop_callback& operator=(const inplace_stop_callback&) = delete;

  private:
    CallbackFn callback-fn;      // exposition only
  };

  template<class CallbackFn>
    inplace_stop_callback(inplace_stop_token, CallbackFn)
      -> inplace_stop_callback<CallbackFn>;
}
```

1. *Mandates*: `CallbackFn` satisfies both `invocable` and `destructible`.
2. *Remarks:* For a type `Initializer`, if *`stoppable-callback-for`*`<CallbackFn, inplace_stop_token, Initializer>` is satisfied, then *`stoppable-callback-for`*`<CallbackFn, inplace_stop_token, Initializer>` is modeled. For an `inplace_stop_callback<CallbackFn>` object, the exposition-only *`callback-fn`* member is its associated callback function ([stoptoken.concepts]).

§ 33.3.10.2. Constructors and destructor **[stopcallback.inplace.cons]**

```
template<class Initializer>
  explicit inplace_stop_callback(inplace_stop_token st, Initializer&& init)
    noexcept(is_nothrow_constructible_v<CallbackFn, Initializer>);
```

1. *Constraints*: `constructible_from<CallbackFn, Initializer>` is satisfied.
2. *Effects*: Initializes *`callback-fn`* with `std::forward<Initializer>(init)` and executes a stoppable callback registration ([stoptoken.concepts]).

```
~inplace_stop_callback();
```

3. *Effects*: Executes a stoppable callback deregistration ([stoptoken.concepts]).

[Editorial note: Insert a new top-level clause — *end note]*

§ 34. Execution control library **[exec]**

§ 34.1. General **[exec.general]**

1. This Clause describes components supporting execution of function objects [function.objects].
2. The following subclauses describe the requirements, concepts, and components for execution control primitives as summarized in Table 1.

Table N: Execution control library summary
[tab:execution.summary]

	Subclause	Header
[exec.sched]	Schedulers	`<execution>`
[exec.recv]	Receivers	
[exec.opstate]	Operation states	
[exec.snd]	Senders	

3. Table 2 shows the types of customization point objects [customization.point.object] used in the

execution control library:

Table N+1: Types of customization point objects in the execution control library ***[tab:execution.cpos]***

Customization point object type	Purpose	Examples
core	provide core execution functionality, and connection between core components	e.g., `connect`, `start`
completion functions	called by senders to announce the completion of the work (success, error, or cancellation)	`set_value`, `set_error`, `set_stopped`
senders	allow the specialization of the provided sender algorithms	— sender factories (e.g., `schedule`, `just`, `read_env`) — sender adaptors (e.g., `continues_on`, `then`, `let_value`) — sender consumers (e.g., `sync_wait`)
queries	allow querying different properties of objects	— general queries (e.g., `get_allocator`, `get_stop_token`) — environment queries (e.g., `get_scheduler`, `get_delegation_scheduler`) — scheduler queries (e.g., `get_forward_progress_guarantee`) — sender attribute queries (e.g., `get_completion_scheduler`)

4. This clause makes use of the following exposition-only entities:

   1. For a subexpression *`expr`*, let *`MANDATE-NOTHROW`*(*`expr`*) be expression-equivalent to *`expr`*.

      — *Mandates:* `noexcept`(*`expr`*) is `true`.

   2.
```
namespace std {
  template<class T>
    concept movable-value = // exposition only
      move_constructible<decay_t<T>> &&
      constructible_from<decay_t<T>, T> &&
```

```
        (!is_array_v<remove_reference_t<T>>);
    }
```

3. For function types `F1` and `F2` denoting `R1(Args1...)` and `R2(Args2...)` respectively, *`MATCHING-SIG`*`(F1, F2)` is `true` if and only if `same_as<R1(Args1&&...), R2(Args2&&...)>` is `true`.
4. For a subexpression `err`, let `Err` be `decltype((err))` and let *`AS-EXCEPT-PTR`*`(err)` be:
    1. `err` if `decay_t<Err>` denotes the type `exception_ptr`.

        — *Mandates:* `err != exception_ptr()` is `true`.
    2. Otherwise, `make_exception_ptr(system_error(err))` if `decay_t<Err>` denotes the type `error_code`.
    3. Otherwise, `make_exception_ptr(err)`.

§ 34.2. Queries and queryables [exec.queryable]

§ 34.2.1. General [exec.queryable.general]

1. A *queryable object* is a read-only collection of key/value pairs where each key is a customization point object known as a *query object*. A *query* is an invocation of a query object with a queryable object as its first argument and a (possibly empty) set of additional arguments. *[Note:* A query imposes syntactic and semantic requirements on its invocations. — *end note]*
2. Let `q` be a query object, let `args` be a (possibly empty) pack of subexpressions, let `env` be a subexpression that refers to a queryable object `o` of type `O`, and let `cenv` be a subexpression referring to `o` such that `decltype((cenv))` is `const O&`. The expression `q(env, args...)` is equal to ([concepts.equality]) the expression `q(cenv, args...)`.
3. The type of a query expression can not be `void`.
4. The expression `q(env, args...)` is equality-preserving ([concepts.equality]) and does not modify the query object or the arguments.
5. If the expression `env.query(q, args...)` is well-formed, then it is expression-equivalent to `q(env, args...)`.
6. Unless otherwise specified, the result of a query is valid as long as the queryable object is valid.

§ 34.2.2. *`queryable`* concept [exec.queryable.concept]

```
namespace std {
  template<class T>
    concept queryable = destructible<T>; // exposition only
}
```

1. The exposition-only `queryable` concept specifies the constraints on the types of queryable objects.
2. Let `env` be an object of type `Env`. The type `Env` models `queryable` if for each callable object `q` and a pack of subexpressions `args`, if `requires { q(env, args...) }` is `true` then `q(env, args...)` meets any semantic requirements imposed by `q`.

§ 34.3. Asynchronous operations **[async.ops]**

1. An *execution resource* is a program entity that manages a (possibly dynamic) set of execution agents ([thread.req.lockable.general]), which it uses to execute parallel work on behalf of callers. [*Example 1*: The currently active thread, a system-provided thread pool, and uses of an API associated with an external hardware accelerator are all examples of execution resources. -- *end example*] Execution resources execute asynchronous operations. An execution resource is either valid or invalid.
2. An *asynchronous operation* is a distinct unit of program execution that:
   1. ... is explicitly created.
   2. ... can be explicitly *started* once at most.
   3. ... once started, eventually *completes* exactly once with a (possibly empty) set of result datums and in exactly one of three *dispositions*: success, failure, or cancellation.
      — A successful completion, also known as a *value completion*, can have an arbitrary number of result datums.
      — A failure completion, also known as an *error completion*, has a single result datum.
      — A cancellation completion, also known as a *stopped completion*, has no result datum.

      An asynchronous operation's *async result* is its disposition and its (possibly empty) set of result datums.
   4. ... can complete on a different execution resource than the execution resource on which it started.
   5. ... can create and start other asynchronous operations called *child operations*. A child operation is an asynchronous operation that is created by the parent operation and, if started, completes before the parent operation completes. A *parent operation* is the asynchronous operation that created a particular child operation.

[Note: An asynchronous operation can in fact execute synchronously; that is, it can complete during the execution of its start operation on the thread of execution that started it. — *end note]*

3. An asynchronous operation has associated state known as its *operation state*.

4. An asynchronous operation has an associated environment. An *environment* is a queryable object ([exec.queryable]) representing the execution-time properties of the operation's caller. The *caller of an asynchronous operation* is its parent operation or the function that created it. An asynchronous operation's operation state owns the operation's environment.

5. An asynchronous operation has an associated receiver. A *receiver* is an aggregation of three handlers for the three asynchronous completion dispositions: a value completion handler for a value completion, an error completion handler for an error completion, and a stopped completion handler for a stopped completion. A receiver has an associated environment. An asynchronous operation's operation state owns the operation's receiver. The environment of an asynchronous operation is equal to its receiver's environment.

6. For each completion disposition, there is a *completion function*. A completion function is a customization point object ([customization.point.object]) that accepts an asynchronous operation's receiver as the first argument and the result datums of the asynchronous operation as additional arguments. The value completion function invokes the receiver's value completion handler with the value result datums; likewise for the error completion function and the stopped completion function. A completion function has an associated type known as its *completion tag* that is the unqualified type of the completion function. A valid invocation of a completion function is called a *completion operation*.

7. The *lifetime of an asynchronous operation*, also known as the operation's *async lifetime*, begins when its start operation begins executing and ends when its completion operation begins executing. If the lifetime of an asynchronous operation's associated operation state ends before the lifetime of the asynchronous operation, the behavior is undefined. After an asynchronous operation executes a completion operation, its associated operation state is invalid. Accessing any part of an invalid operation state is undefined behavior.

8. An asynchronous operation shall not execute a completion operation before its start operation has begun executing. After its start operation has begun executing, exactly one completion operation shall execute. The lifetime of an asynchronous operation's operation state can end during the execution of the completion operation.

9. A *sender* is a factory for one or more asynchronous operations. *Connecting* a sender and a receiver creates an asynchronous operation. The asynchronous operation's associated receiver is equal to the receiver used to create it, and its associated environment is equal to the environment associated with the receiver used to create it. The lifetime of an asynchronous operation's associated operation state does not depend on the lifetimes of either the sender or the receiver from which it was created. A sender is started when it is connected to a receiver and the resulting asynchronous operation is started. A sender's async result is the async result of the asynchronous operation created by connecting it to a receiver. A sender *sends* its results by way of the asynchronous operation(s) it produces, and a receiver *receives* those results. A sender is either valid or invalid; it becomes invalid when its parent sender (see below) becomes invalid.

10. A *scheduler* is an abstraction of an execution resource with a uniform, generic interface for scheduling work onto that resource. It is a factory for senders whose asynchronous operations execute value completion operations on an execution agent belonging to the scheduler's associated execution resource. A *schedule-expression* obtains such a sender from a scheduler. A *schedule sender* is the result of a schedule expression. On success, an asynchronous operation produced by a schedule sender executes a value completion operation with an empty set of result datums. Multiple schedulers can refer to the same execution resource. A scheduler can be valid or invalid. A scheduler becomes invalid when the execution resource to which it refers becomes invalid, as do any schedule senders obtained from the scheduler, and any operation states obtained from those senders.

11. An asynchronous operation has one or more associated completion schedulers for each of its possible dispositions. A *completion scheduler* is a scheduler whose associated execution resource is used to execute a completion operation for an asynchronous operation. A value completion scheduler is a scheduler on which an asynchronous operation's value completion operation can execute. Likewise for error completion schedulers and stopped completion schedulers.

12. A sender has an associated queryable object ([exec.queryable]) known as its *attributes* that describes various characteristics of the sender and of the asynchronous operation(s) it produces. For each disposition, there is a query object for reading the associated completion scheduler from a sender's attributes; i.e., a value completion scheduler query object for reading a sender's value completion scheduler, etc. If a completion scheduler query is well-formed, the returned completion scheduler is unique for that disposition for any asynchronous operation the sender creates. A schedule sender is required to have a value completion scheduler attribute whose value is equal to the scheduler that produced the schedule sender.

13. A *completion signature* is a function type that describes a completion operation. An asynchronous operation has a finite set of possible completion signatures corresponding to the completion operations that the asynchronous operation potentially evaluates ([basic.def.odr]). For a completion function `set`, receiver `rcvr`, and pack of arguments `args`, let `c` be the completion operation `set(rcvr, args...)`, and let `F` be the function type `decltype(auto(set))(decltype((args))...)`. A completion signature `Sig` is associated with `c` if and only if `MATCHING-SIG(Sig, F)` is `true` ([exec.general]). Together, a sender type and an environment type `Env` determine the set of completion signatures of an asynchronous operation that results from connecting the sender with a receiver that has an environment of type `Env`. *[Note:* The type of the receiver does not affect an asynchronous operation's completion signatures, only the type of the receiver's environment. — *end note]*

14. A *sender algorithm* is a function that takes and/or returns a sender. There are three categories of sender algorithms:

    — A *sender factory* is a function that takes non-senders as arguments and that returns a sender.

    — A *sender adaptor* is a function that constructs and returns a *parent sender* from a set of one or more *child senders* and a (possibly empty) set of additional arguments. An asynchronous operation created by a parent sender is a parent operation to the child operations created by the child senders.

— A *sender consumer* is a function that takes one or more senders and a (possibly empty) set of additional arguments, and whose return type is not the type of a sender.

§ 34.4. Header `<execution>` synopsis [exec.syn]

```
namespace std {
  // [exec.general], helper concepts
  template<class T>
    concept movable-value = see below; // exposition only

  template<class From, class To>
    concept decays-to = same_as<decay_t<From>, To>; // exposition only

  template<class T>
    concept class-type = decays-to<T, T> && is_class_v<T>;  // exposition only

  // [exec.queryable], queryable objects
  template<class T>
    concept queryable = see above; // exposition only

  // [exec.queries], queries
  struct forwarding_query_t { see below };
  struct get_allocator_t { see below };
  struct get_stop_token_t { see below };

  inline constexpr forwarding_query_t forwarding_query{};
  inline constexpr get_allocator_t get_allocator{};
  inline constexpr get_stop_token_t get_stop_token{};

  template<class T>
    using stop_token_of_t =
      remove_cvref_t<decltype(get_stop_token(declval<T>()))>;

  template<class T>
    concept forwarding-query = // exposition only
      forwarding_query(T{});
}

namespace std::execution {
  // [exec.queries], queries
  enum class forward_progress_guarantee {
    concurrent,
    parallel,
    weakly_parallel
  };
  struct get_domain_t { see below };
```

```
struct get_scheduler_t { see below };
struct get_delegation_scheduler_t { see below };
struct get_forward_progress_guarantee_t { see below };
template<class CPO>
  struct get_completion_scheduler_t { see below };

inline constexpr get_domain_t get_domain{};
inline constexpr get_scheduler_t get_scheduler{};
inline constexpr get_delegation_scheduler_t get_delegation_scheduler{};
inline constexpr get_forward_progress_guarantee_t get_forward_progress_guarante
template<class CPO>
  inline constexpr get_completion_scheduler_t<CPO> get_completion_scheduler{};

struct empty_env {};
struct get_env_t { see below };
inline constexpr get_env_t get_env{};

template<class T>
  using env_of_t = decltype(get_env(declval<T>()));

// [exec.domain.default], execution domains
struct default_domain;

// [exec.sched], schedulers
struct scheduler_t {};

template<class Sch>
  concept scheduler = see below;

// [exec.recv], receivers
struct receiver_t {};

template<class Rcvr>
  concept receiver = see below;

template<class Rcvr, class Completions>
  concept receiver_of = see below;

struct set_value_t { see below };
struct set_error_t { see below };
struct set_stopped_t { see below };

inline constexpr set_value_t set_value{};
inline constexpr set_error_t set_error{};
inline constexpr set_stopped_t set_stopped{};

// [exec.opstate], operation states
struct operation_state_t {};
```

```
template<class O>
  concept operation_state = see below;

struct start_t { see below };
inline constexpr start_t start{};

// [exec.snd], senders
struct sender_t {};

template<class Sndr>
  concept sender = see below;

template<class Sndr, class Env = empty_env>
  concept sender_in = see below;

template<class Sndr, class Rcvr>
  concept sender_to = see below;

template<class... Ts>
  struct type-list; // exposition only

// [exec.getcomplsigs], completion signatures
struct get_completion_signatures_t { see below };
inline constexpr get_completion_signatures_t get_completion_signatures {};

template<class Sndr, class Env = empty_env>
    requires sender_in<Sndr, Env>
  using completion_signatures_of_t = call-result-t<get_completion_signatures_t,

template<class... Ts>
  using decayed-tuple = tuple<decay_t<Ts>...>; // exposition only

template<class... Ts>
  using variant-or-empty = see below; // exposition only

template<class Sndr,
         class Env = empty_env,
         template<class...> class Tuple = decayed-tuple,
         template<class...> class Variant = variant-or-empty>
    requires sender_in<Sndr, Env>
  using value_types_of_t = see below;

template<class Sndr,
         class Env = empty_env,
         template<class...> class Variant = variant-or-empty>
    requires sender_in<Sndr, Env>
  using error_types_of_t = see below;
```

```
template<class Sndr, class Env = empty_env>
    requires sender_in<Sndr, Env>
  inline constexpr bool sends_stopped = see below;

template<class Sndr, class Env>
  using single-sender-value-type = see below; // exposition only

template<class Sndr, class Env>
  concept single-sender = see below; // exposition only

template<sender Sndr>
  using tag_of_t = see below;

// [exec.snd.transform], sender transformations
template<class Domain, sender Sndr, queryable... Env>
    requires (sizeof...(Env) <= 1)
  constexpr sender decltype(auto) transform_sender(
    Domain dom, Sndr&& sndr, const Env&... env) noexcept(see below);

// [exec.snd.transform.env], environment transformations
template<class Domain, sender Sndr, queryable Env>
  constexpr queryable decltype(auto) transform_env(
    Domain dom, Sndr&& sndr, Env&& env) noexcept;

// [exec.snd.apply], sender algorithm application
template<class Domain, class Tag, sender Sndr, class... Args>
  constexpr decltype(auto) apply_sender(
    Domain dom, Tag, Sndr&& sndr, Args&&... args) noexcept(see below);

// [exec.connect], the connect sender algorithm
struct connect_t { see below };
inline constexpr connect_t connect{};

template<class Sndr, class Rcvr>
  using connect_result_t =
    decltype(connect(declval<Sndr>(), declval<Rcvr>()));

// [exec.factories], sender factories
struct just_t { see below };
struct just_error_t { see below };
struct just_stopped_t { see below };
struct schedule_t { see below };

inline constexpr just_t just{};
inline constexpr just_error_t just_error{};
inline constexpr just_stopped_t just_stopped{};
inline constexpr schedule_t schedule{};
```

```
inline constexpr unspecified read{};

template<scheduler Sndr>
  using schedule_result_t = decltype(schedule(declval<Sndr>()));

// [exec.adapt], sender adaptors
template<class-type D>
  struct sender_adaptor_closure { };

struct starts_on_t { see below };
struct continues_on_t { see below };
struct on_t { see below };
struct schedule_from_t { see below };
struct then_t { see below };
struct upon_error_t { see below };
struct upon_stopped_t { see below };
struct let_value_t { see below };
struct let_error_t { see below };
struct let_stopped_t { see below };
struct bulk_t { see below };
struct split_t { see below };
struct when_all_t { see below };
struct when_all_with_variant_t { see below };
struct into_variant_t { see below };
struct stopped_as_optional_t { see below };
struct stopped_as_error_t { see below };

inline constexpr starts_on_t starts_on{};
inline constexpr continues_on_t continues_on{};
inline constexpr on_t on{};
inline constexpr schedule_from_t schedule_from{};
inline constexpr then_t then{};
inline constexpr upon_error_t upon_error{};
inline constexpr upon_stopped_t upon_stopped{};
inline constexpr let_value_t let_value{};
inline constexpr let_error_t let_error{};
inline constexpr let_stopped_t let_stopped{};
inline constexpr bulk_t bulk{};
inline constexpr split_t split{};
inline constexpr when_all_t when_all{};
inline constexpr when_all_with_variant_t when_all_with_variant{};
inline constexpr into_variant_t into_variant{};
inline constexpr stopped_as_optional_t stopped_as_optional{};
inline constexpr stopped_as_error_t stopped_as_error{};

// [exec.utils], sender and receiver utilities
// [exec.utils.cmplsigs]
template<class Fn>
```

```
    concept completion-signature = // exposition only
      see below;

  template<completion-signature... Fns>
    struct completion_signatures {};

  template<class Sigs> // exposition only
    concept valid-completion-signatures = see below;

  // [exec.utils.tfxcmplsigs]
  template<
    valid-completion-signatures InputSignatures,
    valid-completion-signatures AdditionalSignatures = completion_signatures<>,
    template<class...> class SetValue = see below,
    template<class> class SetError = see below,
    valid-completion-signatures SetStopped = completion_signatures<set_stopped_t(
  using transform_completion_signatures = completion_signatures<see below>;

  template<
    sender Sndr,
    class Env = empty_env,
    valid-completion-signatures AdditionalSignatures = completion_signatures<>,
    template<class...> class SetValue = see below,
    template<class> class SetError = see below,
    valid-completion-signatures SetStopped = completion_signatures<set_stopped_t(
      requires sender_in<Sndr, Env>
  using transform_completion_signatures_of =
    transform_completion_signatures<
      completion_signatures_of_t<Sndr, Env>,
      AdditionalSignatures, SetValue, SetError, SetStopped>;

  // [exec.ctx], execution resources
  // [exec.run.loop], run_loop
  class run_loop;
}

namespace std::this_thread {
  // [exec.consumers], consumers
  struct sync_wait_t { see below };
  struct sync_wait_with_variant_t { see below };

  inline constexpr sync_wait_t sync_wait{};
  inline constexpr sync_wait_with_variant_t sync_wait_with_variant{};
}

namespace std::execution {
  // [exec.as.awaitable]
  struct as_awaitable_t { see below };
```

```
  inline constexpr as_awaitable_t as_awaitable{};

  // [exec.with.awaitable.senders]
  template<class-type Promise>
    struct with_awaitable_senders;
}
```

1. The exposition-only type `variant-or-empty<Ts...>` is defined as follows:

    1. If `sizeof...(Ts)` is greater than zero, `variant-or-empty<Ts...>` denotes `variant<Us...>` where `Us...` is the pack `decay_t<Ts>...` with duplicate types removed.

    2. Otherwise, `variant-or-empty<Ts...>` denotes the exposition-only class type:

```
namespace std::execution {
  struct empty-variant { // exposition only
    empty-variant() = delete;
  };
}
```

2. For types `Sndr` and `Env`, `single-sender-value-type<Sndr, Env>` is an alias for:

    1. `value_types_of_t<Sndr, Env, decay_t, type_identity_t>` if that type is well-formed,

    2. Otherwise, `void` if `value_types_of_t<Sndr, Env, tuple, variant>` is `variant<tuple<>>` or `variant<>`,

    3. Otherwise, `value_types_of_t<Sndr, Env, decayed-tuple, type_identity_t>` if that type is well-formed,

    4. Otherwise, `single-sender-value-type<Sndr, Env>` is ill-formed.

3. The exposition-only concept `single-sender` is defined as follows:

```
namespace std::execution {
  template<class Sndr, class Env>
    concept single-sender =
      sender_in<Sndr, Env> &&
      requires {
        typename single-sender-value-type<Sndr, Env>;
      };
}
```

§ 34.5. Queries **[exec.queries]**

§ 34.5.1. `forwarding_query` [exec.fwd.env]

1. `forwarding_query` asks a query object whether it should be forwarded through queryable adaptors.
2. The name `forwarding_query` denotes a query object. For some query object `q` of type `Q`, `forwarding_query(q)` is expression-equivalent to:
   1. *`MANDATE-NOTHROW`*`(q.query(forwarding_query))` if that expression is well-formed.
      — *Mandates:* The expression above has type `bool` and is a core constant expression if `q` is a core constant expression.
   2. Otherwise, `true` if `derived_from<Q, forwarding_query_t>` is `true`.
   3. Otherwise, `false`.

§ 34.5.2. `get_allocator` [exec.get.allocator]

1. `get_allocator` asks a queryable object for its associated allocator.
2. The name `get_allocator` denotes a query object. For a subexpression `env`, `get_allocator(env)` is expression-equivalent to *`MANDATE-NOTHROW`*`(as_const(env).query(get_allocator))`.
   — *Mandates:* If the expression above is well-formed, its type satisfies *`simple-allocator`* ([allocator.requirements.general]).
3. `forwarding_query(get_allocator)` is a core constant expression and has value `true`.

§ 34.5.3. `get_stop_token` [exec.get.stop.token]

1. `get_stop_token` asks a queryable object for an associated stop token.
2. The name `get_stop_token` denotes a query object. For a subexpression `env`, `get_stop_token(env)` is expression-equivalent to:
   1. *`MANDATE-NOTHROW`*`(as_const(env).query(get_stop_token))` if that expression is well-formed.
      — *Mandates:* The type of the expression above satisfies `stoppable_token`.
   2. Otherwise, `never_stop_token{}`.
3. `forwarding_query(get_stop_token)` is a core constant expression and has value `true`.

§ 34.5.4. `execution::get_env` [exec.get.env]

1. `execution::get_env` is a customization point object. For a subexpression `o`, `execution::get_env(o)` is expression-equivalent to:

    1. *`MANDATE-NOTHROW`*`(as_const(o).get_env())` if that expression is well-formed.

        — *Mandates:* The type of the expression above satisfies *`queryable`* ([exec.queryable]).

    2. Otherwise, `empty_env{}`.

2. The value of `get_env(o)` shall be valid while `o` is valid.

3. *[Note:* When passed a sender object, `get_env` returns the sender's associated attributes. When passed a receiver, `get_env` returns the receiver's associated execution environment. — *end note]*

§ 34.5.5. `execution::get_domain` [exec.get.domain]

1. `get_domain` asks a queryable object for its associated execution domain tag.

2. The name `get_domain` denotes a query object. For a subexpression `env`, `get_domain(env)` is expression-equivalent to *`MANDATE-NOTHROW`*`(as_const(env).query(get_domain))`.

3. `forwarding_query(execution::get_domain)` is a core constant expression and has value `true`.

§ 34.5.6. `execution::get_scheduler` [exec.get.scheduler]

1. `get_scheduler` asks a queryable object for its associated scheduler.

2. The name `get_scheduler` denotes a query object. For a subexpression `env`, `get_scheduler(env)` is expression-equivalent to *`MANDATE-NOTHROW`*`(as_const(env).query(get_scheduler))`.

    — *Mandates:* If the expression above is well-formed, its type satisfies `scheduler`.

3. `forwarding_query(execution::get_scheduler)` is a core constant expression and has value `true`.

§ 34.5.7. `execution::get_delegation_scheduler` [exec.get.delegation.scheduler]

1. `get_delegation_scheduler` asks a queryable object for a scheduler that can be used to delegate work to for the purpose of forward progress delegation ([intro.progress]).

2. The name `get_delegation_scheduler` denotes a query object. For a subexpression `env`, `get_delegation_scheduler(env)` is expression-equivalent to *`MANDATE-NOTHROW`*`(as_const(env).query(get_delegation_scheduler))`.

    — *Mandates:* If the expression above is well-formed, its type satisfies `scheduler`.

3. `forwarding_query(execution::get_delegation_scheduler)` is a core constant expression and has value `true`.

§ 34.5.8. `execution::get_forward_progress_guarantee` [exec.get.forward.progress.guarantee]

```
namespace std::execution {
  enum class forward_progress_guarantee {
    concurrent,
    parallel,
    weakly_parallel
  };
}
```

1. `get_forward_progress_guarantee` asks a scheduler about the forward progress guarantee of execution agents created by that scheduler's associated execution resource ([intro.progress]).
2. The name `get_forward_progress_guarantee` denotes a query object. For a subexpression `sch`, let `Sch` be `decltype((sch))`. If `Sch` does not satisfy `scheduler`, `get_forward_progress_guarantee` is ill-formed. Otherwise, `get_forward_progress_guarantee(sch)` is expression-equivalent to:
   1. *`MANDATE-NOTHROW`*`(as_const(sch).query(get_forward_progress_guarantee))`, if that expression is well-formed.

      — *Mandates:* The type of the expression above is `forward_progress_guarantee`.
   2. Otherwise, `forward_progress_guarantee::weakly_parallel`.
3. If `get_forward_progress_guarantee(sch)` for some scheduler `sch` returns `forward_progress_guarantee::concurrent`, all execution agents created by that scheduler's associated execution resource shall provide the concurrent forward progress guarantee. If it returns `forward_progress_guarantee::parallel`, all such execution agents shall provide at least the parallel forward progress guarantee.

§ 34.5.9. `execution::get_completion_scheduler` [exec.completion.scheduler]

1. `get_completion_scheduler<`*`completion-tag`*`>` obtains the completion scheduler associated with a completion tag from a sender's attributes.
2. The name `get_completion_scheduler` denotes a query object template. For a subexpression `q`, the expression `get_completion_scheduler<`*`completion-tag`*`>(q)` is ill-formed if *`completion-tag`* is not one of `set_value_t`, `set_error_t`, or `set_stopped_t`. Otherwise, `get_completion_scheduler<`*`completion-tag`*`>(q)` is expression-equivalent to *`MANDATE-NOTHROW`*`(as_const(q).query(get_completion_scheduler<`*`completion-tag`*`>))`.

— *Mandates:* If the expression above is well-formed, its type satisfies `scheduler`.

3. Let *`completion-fn`* be a completion function ([async.ops]); let *`completion-tag`* be the associated completion tag of *`completion-fn`*; let `args` be a pack of subexpressions; and let `sndr` be a subexpression such that `sender<decltype((sndr))>` is `true` and `get_completion_scheduler<completion-tag>(get_env(sndr))` is well-formed and denotes a scheduler `sch`. If an asynchronous operation created by connecting `sndr` with a receiver `rcvr` causes the evaluation of `completion-fn(rcvr, args...)`, the behavior is undefined unless the evaluation happens on an execution agent that belongs to `sch`'s associated execution resource.

4. The expression `forwarding_query(get_completion_scheduler<completion-tag>)` is a core constant expression and has value `true`.

§ 34.6. Schedulers [exec.sched]

1. The `scheduler` concept defines the requirements of a scheduler type ([async.ops]). `schedule` is a customization point object that accepts a scheduler. A valid invocation of `schedule` is a schedule-expression.

```
namespace std::execution {
  template<class Sch>
    concept scheduler =
      derived_from<typename remove_cvref_t<Sch>::scheduler_concept, scheduler
      queryable<Sch> &&
      requires(Sch&& sch) {
        { schedule(std::forward<Sch>(sch)) } -> sender;
        { auto(get_completion_scheduler<set_value_t>(
            get_env(schedule(std::forward<Sch>(sch))))) }
              -> same_as<remove_cvref_t<Sch>>;
      } &&
      equality_comparable<remove_cvref_t<Sch>> &&
      copy_constructible<remove_cvref_t<Sch>>;
}
```

2. Let `Sch` be the type of a scheduler and let `Env` be the type of an execution environment for which `sender_in<schedule_result_t<Sch>, Env>` is satisfied. Then *`sender-in-of`*`<schedule_result_t<Sch>, Env>` shall be modeled.

3. None of a scheduler's copy constructor, destructor, equality comparison, or `swap` member functions shall exit via an exception. None of these member functions, nor a scheduler type's `schedule` function, shall introduce data races as a result of potentially concurrent ([intro.races]) invocations of those functions from different threads.

4. For any two values `sch1` and `sch2` of some scheduler type `Sch`, `sch1 == sch2` shall return `true` only if both `sch1` and `sch2` share the same associated execution resource.

5. For a given scheduler expression `sch`, the expression

`get_completion_scheduler<set_value_t>(get_env(schedule(sch)))` shall compare equal to `sch`.

6. For a given scheduler expression `sch`, if the expression `get_domain(sch)` is well-formed, then the expression `get_domain(get_env(schedule(sch)))` is also well-formed and has the same type.

7. A scheduler type's destructor shall not block pending completion of any receivers connected to the sender objects returned from `schedule`. *[Note:* The ability to wait for completion of submitted function objects can be provided by the associated execution resource of the scheduler. — *end note]*

§ 34.7. Receivers **[exec.recv]**

§ 34.7.1. Receiver concepts [exec.recv.concepts]

1. A receiver represents the continuation of an asynchronous operation. The `receiver` concept defines the requirements for a receiver type ([async.ops]). The `receiver_of` concept defines the requirements for a receiver type that is usable as the first argument of a set of completion operations corresponding to a set of completion signatures. The `get_env` customization point object is used to access a receiver's associated environment.

```
namespace std::execution {
  template<class Rcvr>
    concept receiver =
      derived_from<typename remove_cvref_t<Rcvr>::receiver_concept, receiver_
      requires(const remove_cvref_t<Rcvr>& rcvr) {
        { get_env(rcvr) } -> queryable;
      } &&
      move_constructible<remove_cvref_t<Rcvr>> &&  // rvalues are movable, an
      constructible_from<remove_cvref_t<Rcvr>, Rcvr>; // lvalues are copyable

  template<class Signature, class Rcvr>
    concept valid-completion-for = // exposition only
      requires (Signature* sig) {
        []<class Tag, class... Args>(Tag(*)(Args...))
            requires callable<Tag, remove_cvref_t<Rcvr>, Args...>
        {}(sig);
      };

  template<class Rcvr, class Completions>
    concept has-completions = // exposition only
      requires (Completions* completions) {
        []<valid-completion-for<Rcvr>...Sigs>(completion_signatures<Sigs...>*
        {}(completions);
      };
```

```
    template<class Rcvr, class Completions>
      concept receiver_of =
        receiver<Rcvr> && has-completions<Rcvr, Completions>;
  }
```

2. Class types that are marked `final` do not model the `receiver` concept.

3. Let `rcvr` be a receiver and let `op_state` be an operation state associated with an asynchronous operation created by connecting `rcvr` with a sender. Let `token` be a stop token equal to `get_stop_token(get_env(rcvr))`. `token` shall remain valid for the duration of the asynchronous operation's lifetime ([async.ops]). *[Note:* This means that, unless it knows about further guarantees provided by the type of `rcvr`, the implementation of `op_state` can not use `token` after it executes a completion operation. This also implies that any stop callbacks registered on `token` must be destroyed before the invocation of the completion operation. — *end note]*

§ 34.7.2. `execution::set_value` [exec.set.value]

1. `set_value` is a value completion function ([async.ops]). Its associated completion tag is `set_value_t`. The expression `set_value(rcvr, vs...)` for a subexpression `rcvr` and pack of subexpressions `vs` is ill-formed if `rcvr` is an lvalue or an rvalue of const type. Otherwise, it is expression-equivalent to *`MANDATE-NOTHROW`*`(rcvr.set_value(vs...))`.

§ 34.7.3. `execution::set_error` [exec.set.error]

1. `set_error` is an error completion function ([async.ops]). Its associated completion tag is `set_error_t`. The expression `set_error(rcvr, err)` for some subexpressions `rcvr` and `err` is ill-formed if `rcvr` is an lvalue or an rvalue of const type. Otherwise, it is expression-equivalent to *`MANDATE-NOTHROW`*`(rcvr.set_error(err))`.

§ 34.7.4. `execution::set_stopped` [exec.set.stopped]

1. `set_stopped` is a stopped completion function ([async.ops]). Its associated completion tag is `set_stopped_t`. The expression `set_stopped(rcvr)` for a subexpression `rcvr` is ill-formed if `rcvr` is an lvalue or an rvalue of `const` type. Otherwise, it is expression-equivalent to *`MANDATE-NOTHROW`*`(rcvr.set_stopped())`.

§ 34.8. Operation states [exec.opstate]

1. The `operation_state` concept defines the requirements of an operation state type ([async.ops]).

```
namespace std::execution {
  template<class O>
    concept operation_state =
      derived_from<typename O::operation_state_concept, operation_state_t> &&
      is_object_v<O> &&
      requires (O& o) {
        { start(o) } noexcept;
      };
}
```

2. If an `operation_state` object is destroyed during the lifetime of its asynchronous operation ([async.ops]), the behavior is undefined. *[Note:* The `operation_state` concept does not impose requirements on any operations other than destruction and `start`, including copy and move operations. Invoking any such operation on an object whose type models `operation_state` can lead to undefined behavior. — *end note]*
3. The program is ill-formed if it performs a copy or move construction or assigment operation on an operation state object created by connecting a library-provided sender.

§ 34.8.1. `execution::start` [exec.opstate.start]

1. The name `start` denotes a customization point object that starts ([async.ops]) the asynchronous operation associated with the operation state object. For a subexpression `op`, the expression `start(op)` is ill-formed if `op` is an rvalue. Otherwise, it is expression-equivalent to *`MANDATE-NOTHROW`*`(op.start())`.
2. If `op.start()` does not start ([async.ops]) the asynchronous operation associated with the operation state `op`, the behavior of calling `start(op)` is undefined.

§ 34.9. Senders [exec.snd]

§ 34.9.1. General [exec.snd.general]

1. For the purposes of this subclause, a sender is an object whose type satisfies the `sender` concept ([async.ops]).
2. Subclauses [exec.factories] and [exec.adapt] define customizable algorithms that return senders. Each algorithm has a default implementation. Let `sndr` be the result of an invocation of such an algorithm or an object equal to the result ([concepts.equality]), and let `Sndr` be `decltype((sndr))`. Let `rcvr` be a receiver of type `Rcvr` with associated environment `env` of type `Env` such that `sender_to<Sndr, Rcvr>` is `true`. For the default implementation of the algorithm that produced `sndr`, connecting `sndr` to `rcvr` and starting the resulting operation state ([async.ops]) necessarily results in the potential evaluation ([basic.def.odr]) of a set of completion operations whose first argument is a subexpression equal to `rcvr`. Let `Sigs` be a pack of

completion signatures corresponding to this set of completion operations. Then the type of the expression `get_completion_signatures(sndr, env)` is a specialization of the class template `completion_signatures` ([exec.utils.cmplsigs]), the set of whose template arguments is `Sigs`. If a user-provided implementation of the algorithm that produced `sndr` is selected instead of the default, any completion signature that is in the set of types denoted by `completion_signatures_of_t<Sndr, Env>` and that is not part of `Sigs` shall correspond to error or stopped completion operations, unless otherwise specified.

3. This subclause makes use of the following exposition-only entities.

    1. For a queryable object `env`, *`FWD-ENV`*`(env)` is an expression whose type satisfies *`queryable`* such that for a query object `q` and a pack of subexpressions `as`, the expression *`FWD-ENV`*`(env).query(q, as...)` is ill-formed if `forwarding_query(q)` is `false`; otherwise, it is expression-equivalent to `env.query(q, as...)`.

    2. For a query object `q` and a subexpression `v`, *`MAKE-ENV`*`(q, v)` is an expression `env` whose type satisfies *`queryable`* such that the result of `env.query(q)` has a value equal to `v` ([concepts.equality]). Unless otherwise stated, the object to which `env.query(q)` refers remains valid while `env` remains valid.

    3. For two queryable objects `env1` and `env2`, a query object `q` and a pack of subexpressions `as`, *`JOIN-ENV`*`(env1, env2)` is an expression `env3` whose type satisfies *`queryable`* such that `env3.query(q, as...)` is expression-equivalent to:

        — `env1.query(q, as...)` if that expression is well-formed,

        — otherwise, `env2.query(q, as...)` if that expression is well-formed,

        — otherwise, `env3.query(q, as...)` is ill-formed.

    4. The results of *`FWD-ENV`*, *`MAKE-ENV`*, and *`JOIN-ENV`* can be context-dependent; i.e., they can evaluate to expressions with different types and value categories in different contexts for the same arguments.

    5. For a scheduler `sch`, *`SCHED-ATTRS`*`(sch)` is an expression `o1` whose type satisfies *`queryable`* such that `o1.query(get_completion_scheduler<`*`Tag`*`>)` is a expression with the same type and value as `sch` where *`Tag`* is one of `set_value_t` or `set_stopped_t`, and such that `o1.query(get_domain)` is expression-equivalent to `sch.query(get_domain)`. *`SCHED-ENV`*`(sch)` is an expression `o2` whose type satisfies *`queryable`* such that `o1.query(get_scheduler)` is a prvalue with the same type and value as `sch`, and such that `o2.query(get_domain)` is expression-equivalent to `sch.query(get_domain)`.

    6. For two subexpressions `rcvr` and `expr`, *`SET-VALUE`*`(rcvr, expr)` is expression-equivalent to `(expr, set_value(std::move(rcvr)))` if the type of `expr` is `void`; otherwise, `set_value(std::move(rcvr), expr)`. *`TRY-EVAL`*`(rcvr, expr)` is equivalent to:

```
try {
  expr;
} catch(...) {
```

```
    set_error(std::move(rcvr), current_exception());
  }
```

if `expr` is potentially-throwing; otherwise, `expr.` *`TRY-SET-VALUE`*`(rcvr, expr)` is *`TRY-EVAL`*`(rcvr,` *`SET-VALUE`*`(rcvr, expr))` except that `rcvr` is evaluated only once.

7. ```
   template<class Default = default_domain, class Sndr>
     constexpr auto completion-domain(const Sndr& sndr) noexcept;
   ```

   1. *`COMPL-DOMAIN`*`(T)` is the type of the expression `get_domain(get_completion_scheduler<T>(get_env(sndr)))`.
   2. *Effects:* If all of the types *`COMPL-DOMAIN`*`(set_value_t)`, *`COMPL-DOMAIN`*`(set_error_t)`, and *`COMPL-DOMAIN`*`(set_stopped_t)` are ill-formed, *`completion-domain`*`<Default>(sndr)` is a default-constructed prvalue of type `Default`. Otherwise, if they all share a common type ([meta.trans.other]) (ignoring those types that are ill-formed), then *`completion-domain`*`<Default>(sndr)` is a default-constructed prvalue of that type. Otherwise, *`completion-domain`*`<Default>(sndr)` is ill-formed.

8. ```
   template<class Tag, class Env, class Default>
     constexpr decltype(auto) query-with-default(
       Tag, const Env& env, Default&& value) noexcept(see below);
   ```

   1. Let `e` be the expression `Tag()(env)` if that expression is well-formed; otherwise, it is `static_cast<Default>(std::forward<Default>(value))`.
   2. *Returns:* `e`.
   3. *Remarks:* The expression in the `noexcept` clause is `noexcept(e)`.

9. ```
   template<class Sndr>
     constexpr auto get-domain-early(const Sndr& sndr) noexcept;
   ```

   1. *Effects:* Equivalent to: `return Domain();` where `Domain` is the decayed type of the first of the following expressions that is well-formed:

      — `get_domain(get_env(sndr))`

      — *`completion-domain`*`(sndr)`

      — `default_domain()`

10. ```
    template<class Sndr, class Env>
      constexpr auto get-domain-late(const Sndr& sndr, const Env& env) noexce
    ```

    1. *Effects:* Equivalent to:

— If `sender-for<Sndr, continues_on_t>` is `true`, then `return Domain();` where `Domain` is the type of the following expression:

```
[] {
  auto [_, sch, _] = sndr;
  return query-or-default(get_domain, sch, default_domain());
}();
```

[Note: The `continues_on` algorithm works in tandem with `schedule_from` ([exec.schedule.from])) to give scheduler authors a way to customize both how to transition onto (`continues_on`) and off of (`schedule_from`) a given execution context. Thus, `continues_on` ignores the domain of the predecessor and uses the domain of the destination scheduler to select a customization, a property that is unique to `continues_on`. That is why it is given special treatment here. — *end note]*

— Otherwise, `return Domain();` where `Domain` is the first of the following expressions that is well-formed and whose type is not `void`:

— `get_domain(get_env(sndr))`

— `completion-domain<void>(sndr)`

— `get_domain(env)`

— `get_domain(get_scheduler(env))`

— `default_domain()`.

11.
```
template<callable Fun>
  requires is_nothrow_move_constructible_v<Fun>
struct emplace-from { // exposition only
  Fun fun; // exposition only
  using type = call-result-t<Fun>;

  constexpr operator type() && noexcept(nothrow-callable<Fun>) {
    return std::move(fun)();
  }

  constexpr type operator()() && noexcept(nothrow-callable<Fun>) {
    return std::move(fun)();
  }
};
```

1. *[Note:* `emplace-from` is used to emplace non-movable types into `tuple`, `optional`, `variant`, and similar types. — *end note]*

12.
```
struct on-stop-request { // exposition only
```

```
  inplace_stop_source& stop-src; // exposition only
  void operator()() noexcept { stop-src.request_stop(); }
};
```

13.
```
template<class T0, class T1, ... class Tn>
struct product-type {  // exposition only
  T0 t0;      // exposition only
  T1 t1;      // exposition only
    ...
  Tn tn;      // exposition only

  template<size_t I, class Self>
  constexpr decltype(auto) get(this Self&& self) noexcept; // exposition

  template<class Self, class Fn>
  constexpr decltype(auto) apply(this Self&& self, Fn&& fn) // exposition
    noexcept(see below);
};
```

1. *[Note:* `product-type` is presented here in pseudo-code form for the sake of exposition. It can be approximated in standard C++ with a `tuple`-like implementation that takes care to keep the type an aggregate that can be used as the initializer of a structured binding declaration. — *end note]*

2. *[Note:* An expression of type `product-type` is usable as the initializer of a structured binding declaration [dcl.struct.bind]. — *end note]*

3.
```
template<size_t I, class Self>
constexpr decltype(auto) get(this Self&& self) noexcept;
```

   1. *Effects:* Equivalent to:

```
auto& [...ts] = self;
return std::forward_like<Self>(ts...[I]);
```

4.
```
template<class Self, class Fn>
constexpr decltype(auto) apply(this Self&& self, Fn&& fn) noexcept(se
```

   1. *Effects:* Equivalent to:

```
auto& [...ts] = self;
return std::forward<Fn>(fn)(std::forward_like<Self>(ts)...);
```

   2. *Requires:* The expression in the `return` statement above is well-formed.

3. *Remarks:* The expression in the `noexcept` clause is `true` if the `return` statement above is not potentially throwing; otherwise, `false`.

14.
```
template<class Tag, class Data = see below, class... Child>
  constexpr auto make-sender(Tag tag, Data&& data, Child&&... child);
```

1. *Mandates:* The following expressions are `true`:

   — `semiregular<Tag>`

   — `movable-value<Data>`

   — `(sender<Child> &&...)`

2. *Returns:* A prvalue of type `basic-sender<Tag, decay_t<Data>, decay_t<Child>...>` that has been direct-list-initialized with the forwarded arguments, where `basic-sender` is the following exposition-only class template except as noted below:

```
namespace std::execution {
  template<class Tag>
  concept completion-tag = // exposition only
    same_as<Tag, set_value_t> || same_as<Tag, set_error_t> || same_as

  template<template<class...> class T, class... Args>
  concept valid-specialization = requires { typename T<Args...>; }; /

  struct default-impls {  // exposition only
    static constexpr auto get-attrs = see below;
    static constexpr auto get-env = see below;
    static constexpr auto get-state = see below;
    static constexpr auto start = see below;
    static constexpr auto complete = see below;
  };

  template<class Tag>
  struct impls-for : default-impls {}; // exposition only

  template<class Sndr, class Rcvr> // exposition only
  using state-type = decay_t<call-result-t<
    decltype(impls-for<tag_of_t<Sndr>>::get-state), Sndr, Rcvr&>>;

  template<class Index, class Sndr, class Rcvr> // exposition only
  using env-type = call-result-t<
    decltype(impls-for<tag_of_t<Sndr>>::get-env), Index,
    state-type<Sndr, Rcvr>&, const Rcvr&>;

  template<class Sndr, size_t I = 0>
```

```
using child-type = decltype(declval<Sndr>().template get<I+2>()); /

template<class Sndr>
using indices-for = remove_reference_t<Sndr>::indices-for; // expos

template<class Sndr, class Rcvr>
struct basic-state { // exposition only
  basic-state(Sndr&& sndr, Rcvr&& rcvr) noexcept(see below)
    : rcvr(std::move(rcvr))
    , state(impls-for<tag_of_t<Sndr>>::get-state(std::forward<Sndr>

  Rcvr rcvr; // exposition only
  state-type<Sndr, Rcvr> state; // exposition only
};

template<class Sndr, class Rcvr, class Index>
  requires valid-specialization<env-type, Index, Sndr, Rcvr>
struct basic-receiver {  // exposition only
  using receiver_concept = receiver_t;

  using tag-t = tag_of_t<Sndr>; // exposition only
  using state-t = state-type<Sndr, Rcvr>; // exposition only
  static constexpr const auto& complete = impls-for<tag-t>::complet

  template<class... Args>
    requires callable<decltype(complete), Index, state-t&, Rcvr&, s
  void set_value(Args&&... args) && noexcept {
    complete(Index(), op->state, op->rcvr, set_value_t(), std::forw
  }

  template<class Error>
    requires callable<decltype(complete), Index, state-t&, Rcvr&, s
  void set_error(Error&& err) && noexcept {
    complete(Index(), op->state, op->rcvr, set_error_t(), std::forw
  }

  void set_stopped() && noexcept
    requires callable<decltype(complete), Index, state-t&, Rcvr&, s
    complete(Index(), op->state, op->rcvr, set_stopped_t());
  }

  auto get_env() const noexcept -> env-type<Index, Sndr, Rcvr> {
    return impls-for<tag-t>::get-env(Index(), op->state, op->rcvr);
  }

  basic-state<Sndr, Rcvr>* op; // exposition only
};
```

```
constexpr auto connect-all = see below; // exposition only

template<class Sndr, class Rcvr>
using connect-all-result = call-result-t<  // exposition only
  decltype(connect-all), basic-state<Sndr, Rcvr>*, Sndr, indices-fo

template<class Sndr, class Rcvr>
  requires valid-specialization<state-type, Sndr, Rcvr> &&
           valid-specialization<connect-all-result, Sndr, Rcvr>
struct basic-operation : basic-state<Sndr, Rcvr> {  // exposition o
  using operation_state_concept = operation_state_t;
  using tag-t = tag_of_t<Sndr>; // exposition only

  connect-all-result<Sndr, Rcvr> inner-ops; // exposition only

  basic-operation(Sndr&& sndr, Rcvr&& rcvr) noexcept(see below)  //
    : basic-state<Sndr, Rcvr>(std::forward<Sndr>(sndr), std::move(r
    , inner-ops(connect-all(this, std::forward<Sndr>(sndr), indices
  {}

  void start() & noexcept {
    auto& [...ops] = inner-ops;
    impls-for<tag-t>::start(this->state, this->rcvr, ops...);
  }
};

template<class Sndr, class Env>
using completion-signatures-for = see below; // exposition only

template<class Tag, class Data, class... Child>
struct basic-sender : product-type<Tag, Data, Child...> {  // expos
  using sender_concept = sender_t;
  using indices-for = index_sequence_for<Child...>; // exposition o

  decltype(auto) get_env() const noexcept {
    auto& [_, data, ...child] = *this;
    return impls-for<Tag>::get-attrs(data, child...);
  }

  template<decays-to<basic-sender> Self, receiver Rcvr>
  auto connect(this Self&& self, Rcvr rcvr) noexcept(see below)
    -> basic-operation<Self, Rcvr> {
    return {std::forward<Self>(self), std::move(rcvr)};
  }

  template<decays-to<basic-sender> Self, class Env>
  auto get_completion_signatures(this Self&& self, Env&& env) noexc
    -> completion-signatures-for<Self, Env> {
```

```
      return {};
    }
  };
}
```

3. *Remarks:* The default template argument for the `Data` template parameter denotes an unspecified empty trivially copyable class type that models `semiregular`.
4. It is unspecified whether a specialization of *`basic-sender`* is an aggregate.
5. An expression of type *`basic-sender`* is usable as the initializer of a structured binding declaration [dcl.struct.bind].
6. The expression in the `noexcept` clause of the constructor of *`basic-state`* is:

```
is_nothrow_move_constructible_v<Rcvr> &&
nothrow-callable<decltype(impls-for<tag_of_t<Sndr>>::get-state), Sndr
```

7. The object *`connect-all`* is initialized with a callable object equivalent to the following lambda:

```
[]<class Sndr, class Rcvr, size_t... Is>(
  basic-state<Sndr, Rcvr>* op, Sndr&& sndr, index_sequence<Is...>) nc
    -> decltype(auto) {
    auto& [_, data, ...child] = sndr;
    return product-type{connect(
      std::forward_like<Sndr>(child),
      basic-receiver<Sndr, Rcvr, integral_constant<size_t, Is>>{op}).
  }
```

   1. *Requires:* The expression in the `return` statement is well-formed.
   2. *Remarks:* The expression in the `noexcept` clause is `true` if the `return` statement is not potentially throwing; otherwise, `false`.
8. The expression in the `noexcept` clause of the constructor of *`basic-operation`* is:

```
is_nothrow_constructible_v<basic-state<Self, Rcvr>, Self, Rcvr> &&
noexcept(connect-all(this, std::forward<Sndr>(sndr), indices-for<Sndr
```

9. The expression in the `noexcept` clause of the `connect` member function of *`basic-sender`* is:

```
is_nothrow_constructible_v<basic-operation<Self, Rcvr>, Self, Rcvr>
```

10. The member *`default-impls::get-attrs`* is initialized with a callable object equivalent to the following lambda:

```
[](const auto&, const auto&... child) noexcept -> decltype(auto) {
  if constexpr (sizeof...(child) == 1)
    return (FWD-ENV(get_env(child)), ...);
  else
    return empty_env();
}
```

11. The member *default-impls::get-env* is initialized with a callable object equivalent to the following lambda:

```
[](auto, auto&, const auto& rcvr) noexcept -> decltype(auto) {
  return FWD-ENV(get_env(rcvr));
}
```

12. The member *default-impls::get-state* is initialized with a callable object equivalent to the following lambda:

```
[]<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr) noexcept -> declt
  auto& [_, data, ...child] = sndr;
  return std::forward_like<Sndr>(data);
}
```

13. The member *default-impls::start* is initialized with a callable object equivalent to the following lambda:

```
[](auto&, auto&, auto&... ops) noexcept -> void {
  (execution::start(ops), ...);
}
```

14. The member *default-impls::complete* is initialized with a callable object equivalent to the following lambda:

```
[]<class Index, class Rcvr, class Tag, class... Args>(
  Index, auto& state, Rcvr& rcvr, Tag, Args&&... args) noexcept
    -> void requires callable<Tag, Rcvr, Args...> {
  // Mandates: Index::value == 0
  Tag()(std::move(rcvr), std::forward<Args>(args)...);
}
```

15. For a subexpression `sndr` let `Sndr` be `decltype((sndr))`. Let `rcvr` be a receiver with an associated environment of type `Env` such that `sender_in<Sndr, Env>` is `true`. *completion-signatures-for*`<Sndr, Env>` denotes a specialization of `completion_signatures`, the set of whose template arguments correspond to the set of completion operations that are potentially evaluated as a result of starting

([async.ops]) the operation state that results from connecting `sndr` and `rcvr`. When `sender_in<Sndr, Env>` is `false`, the type denoted by *`completion-signatures-for`*`<Sndr, Env>`, if any, is not a specialization of `completion_signatures`.

Recommended practice: When `sender_in<Sndr, Env>` is `false`, implementations are encouraged to use the type denoted by *`completion-signatures-for`*`<Sndr, Env>` to communicate to users why.

15.
```
template<sender Sndr, queryable Env>
  constexpr auto write-env(Sndr&& sndr, Env&& env); // exposition only
```

    1. *`write-env`* is an exposition-only sender adaptor that, when connected with a receiver `rcvr`, connects the adapted sender with a receiver whose execution environment is the result of joining the *`queryable`* argument `env` to the result of `get_env(rcvr)`.
    2. Let *`write-env-t`* be an exposition-only empty class type.
    3. *Returns:* *`make-sender`*`(`*`write-env-t`*`(), std::forward<Env>(env), std::forward<Sndr>(sndr))`.
    4. *Remarks:* The exposition-only class template *`impls-for`* ([exec.snd.general]) is specialized for *`write-env-t`* as follows:

```
template<>
struct impls-for<write-env-t> : default-impls {
  static constexpr auto get-env =
    [](auto, const auto& state, const auto& rcvr) noexcept {
      return JOIN-ENV(state, get_env(rcvr));
    };
};
```

§ 34.9.2. Sender concepts [exec.snd.concepts]

1. The `sender` concept defines the requirements for a sender type ([async.ops]). The `sender_in` concept defines the requirements for a sender type that can create asynchronous operations given an associated environment type. The `sender_to` concept defines the requirements for a sender type that can connect with a specific receiver type. The `get_env` customization point object is used to access a sender's associated attributes. The `connect` customization point object is used to connect ([async.ops]) a sender and a receiver to produce an operation state.

```
namespace std::execution {
  template<class Sigs>
    concept valid-completion-signatures = see below; // exposition only

  template<class Sndr>
    concept is-sender = // exposition only
```

```
    derived_from<typename Sndr::sender_concept, sender_t>;

  template<class Sndr>
    concept enable-sender = // exposition only
      is-sender<Sndr> ||
      is-awaitable<Sndr, env-promise<empty_env>>;  // [exec.awaitables]

  template<class Sndr>
    concept sender =
      bool(enable-sender<remove_cvref_t<Sndr>>) && // atomic constraint ([tem
      requires (const remove_cvref_t<Sndr>& sndr) {
        { get_env(sndr) } -> queryable;
      } &&
      move_constructible<remove_cvref_t<Sndr>> &&  // senders are movable and
      constructible_from<remove_cvref_t<Sndr>, Sndr>; // decay copyable

  template<class Sndr, class Env = empty_env>
    concept sender_in =
      sender<Sndr> &&
      queryable<Env> &&
      requires (Sndr&& sndr, Env&& env) {
        { get_completion_signatures(std::forward<Sndr>(sndr), std::forward<En
          -> valid-completion-signatures;
      };

  template<class Sndr, class Rcvr>
    concept sender_to =
      sender_in<Sndr, env_of_t<Rcvr>> &&
      receiver_of<Rcvr, completion_signatures_of_t<Sndr, env_of_t<Rcvr>>> &&
      requires (Sndr&& sndr, Rcvr&& rcvr) {
        connect(std::forward<Sndr>(sndr), std::forward<Rcvr>(rcvr));
      };
}
```

2. Given a subexpression `sndr`, let `Sndr` be `decltype((sndr))` and let `rcvr` be a receiver with an associated environment whose type is `Env`. A completion operation is a *permissible completion* for `Sndr` and `Env` if its completion signature appears in the argument list of the specialization of `completion_signatures` denoted by `completion_signatures_of_t<Sndr, Env>`. `Sndr` and `Env` model `sender_in<Sndr, Env>` if all the completion operations that are potentially evaluated by connecting `sndr` to `rcvr` and starting the resulting operation state are permissible completions for `Sndr` and `Env`.
3. A type models the exposition-only concept *`valid-completion-signatures`* if it denotes a specialization of the `completion_signatures` class template.
4. The exposition-only concepts *`sender-of`* and *`sender-in-of`* define the requirements for a sender type that completes with a given unique set of value result types.

```
namespace std::execution {
  template<class... As>
    using value-signature = set_value_t(As...); // exposition only

  template<class Sndr, class Env, class... Values>
    concept sender-in-of =
      sender_in<Sndr, Env> &&
      MATCHING-SIG( // see [exec.general]
        set_value_t(Values...),
        value_types_of_t<Sndr, Env, value-signature, type_identity_t>);

  template<class Sndr, class... Values>
    concept sender-of = sender-in-of<Sndr, empty_env, Values...>;
}
```

5. Let `sndr` be an expression such that `decltype((sndr))` is `Sndr`. The type `tag_of_t<Sndr>` is as follows:

   — If the declaration `auto&& [tag, data, ...children] = sndr;` would be well-formed, `tag_of_t<Sndr>` is an alias for `decltype(auto(tag))`.

   — Otherwise, `tag_of_t<Sndr>` is ill-formed.

6. Let *`sender-for`* be an exposition-only concept defined as follows:

```
namespace std::execution {
  template<class Sndr, class Tag>
  concept sender-for =
    sender<Sndr> &&
    same_as<tag_of_t<Sndr>, Tag>;
}
```

7. For a type `T`, *`SET-VALUE-SIG`*`(T)` denotes the type `set_value_t()` if `T` is *cv* `void`; otherwise, it denotes the type `set_value_t(T)`.

8. Library-provided sender types:

   — Always expose an overload of a member `connect` that accepts an rvalue sender.

   — Only expose an overload of a member `connect` that accepts an lvalue sender if they model `copy_constructible`.

§ 34.9.3. Awaitable helpers [exec.awaitables]

1. The sender concepts recognize awaitables as senders. For [exec], an *awaitable* is an expression that would be well-formed as the operand of a `co_await` expression within a given context.

2. For a subexpression `c`, let *`GET-AWAITER`*`(c, p)` be expression-equivalent to the series of trans-

formations and conversions applied to `c` as the operand of an *await-expression* in a coroutine, resulting in lvalue `e` as described by [expr.await], where `p` is an lvalue referring to the coroutine's promise, which has type `Promise`. *[Note:* This includes the invocation of the promise type's `await_transform` member if any, the invocation of the `operator co_await` picked by overload resolution if any, and any necessary implicit conversions and materializations. — *end note]*

[Editorial note: I have opened cwg#250 to give these transformations a term-of-art so we can more easily refer to it here. — *end note]*

3. Let `is-awaitable` be the following exposition-only concept:

```
namespace std {
  template<class T>
  concept await-suspend-result = see below; // exposition only

  template<class A, class Promise>
  concept is-awaiter = // exposition only
    requires (A& a, coroutine_handle<Promise> h) {
      a.await_ready() ? 1 : 0;
      { a.await_suspend(h) } -> await-suspend-result;
      a.await_resume();
    };

  template<class C, class Promise>
  concept is-awaitable =
    requires (C (*fc)() noexcept, Promise& p) {
      { GET-AWAITER(fc(), p) } -> is-awaiter<Promise>;
    };
}
```

`await-suspend-result<T>` is `true` if and only if one of the following is `true`:

— `T` is `void`, or

— `T` is `bool`, or

— `T` is a specialization of `coroutine_handle`.

4. For a subexpression `c` such that `decltype((c))` is type `C`, and an lvalue `p` of type `Promise`, `await-result-type<C, Promise>` denotes the type `decltype(GET-AWAITER(c, p).await_resume())`.

5. Let `with-await-transform` be the exposition-only class template:

```
namespace std::execution {
  template<class T, class Promise>
    concept has-as-awaitable = // exposition only
```

```
      requires (T&& t, Promise& p) {
        { std::forward<T>(t).as_awaitable(p) } -> is-awaitable<Promise&>;
      };

    template<class Derived>
      struct with-await-transform {
        template<class T>
          T&& await_transform(T&& value) noexcept {
            return std::forward<T>(value);
          }

        template<has-as-awaitable<Derived> T>
          decltype(auto) await_transform(T&& value)
            noexcept(noexcept(std::forward<T>(value).as_awaitable(declval<Deriv
            return std::forward<T>(value).as_awaitable(static_cast<Derived&>(*t
          }
      };
  }
```

6. Let *env-promise* be the exposition-only class template:

```
namespace std::execution {
  template<class Env>
  struct env-promise : with-await-transform<env-promise<Env>> {
    unspecified get_return_object() noexcept;
    unspecified initial_suspend() noexcept;
    unspecified final_suspend() noexcept;
    void unhandled_exception() noexcept;
    void return_void() noexcept;
    coroutine_handle<> unhandled_stopped() noexcept;

    const Env& get_env() const noexcept;
  };
}
```

[Note: Specializations of *env-promise* are only used for the purpose of type computation; its members need not be defined. — *end note]*

§ 34.9.4. `execution::default_domain` [exec.domain.default]

```
namespace std::execution {
  struct default_domain {
    template<sender Sndr, queryable... Env>
      requires (sizeof...(Env) <= 1)
    static constexpr sender decltype(auto) transform_sender(Sndr&& sndr, const
```

```
        noexcept(see below);

    template<sender Sndr, queryable Env>
      static constexpr queryable decltype(auto) transform_env(Sndr&& sndr, Env&&

    template<class Tag, sender Sndr, class... Args>
      static constexpr decltype(auto) apply_sender(Tag, Sndr&& sndr, Args&&... ar
        noexcept(see below);
  };
}
```

§ 34.9.4.1. Static members **[exec.domain.default.statics]**

```
template<sender Sndr, queryable... Env>
    requires (sizeof...(Env) <= 1)
  constexpr sender decltype(auto) transform_sender(Sndr&& sndr, const Env&... env
    noexcept(see below);
```

1. Let `e` be the expression `tag_of_t<Sndr>().transform_sender(std::forward<Sndr>(sndr), env...)` if that expression is well-formed; otherwise, `std::forward<Sndr>(sndr)`.
2. *Returns:* `e`.
3. *Remarks:* The exception specification is equivalent to `noexcept(e)`.

```
template<sender Sndr, queryable Env>
  constexpr queryable decltype(auto) transform_env(Sndr&& sndr, Env&& env) noexce
```

4. Let `e` be the expression `tag_of_t<Sndr>().transform_env(std::forward<Sndr>(sndr), std::forward<Env>(env))` if that expression is well-formed; otherwise, `static_cast<Env>(std::forward<Env>(env))`.
5. *Mandates:* `noexcept(e)` is `true`.
6. *Returns:* `e`.

```
template<class Tag, sender Sndr, class... Args>
  constexpr decltype(auto) apply_sender(Tag, Sndr&& sndr, Args&&... args)
    noexcept(see below);
```

7. Let `e` be the expression `Tag().apply_sender(std::forward<Sndr>(sndr), std::forward<Args>(args)...)`.
8. *Constraints:* `e` is a well-formed expression.

9. *Returns:* `e`.

10. *Remarks:* The exception specification is equivalent to `noexcept(e)`.

§ 34.9.5. `execution::transform_sender` [exec.snd.transform]

```
namespace std::execution {
  template<class Domain, sender Sndr, queryable... Env>
      requires (sizeof...(Env) <= 1)
    constexpr sender decltype(auto) transform_sender(Domain dom, Sndr&& sndr, con
      noexcept(see below);
}
```

1. Let *`transformed-sndr`* be the expression `dom.transform_sender(std::forward<Sndr>(sndr), env...)` if that expression is well-formed; otherwise, `default_domain().transform_sender(std::forward<Sndr>(sndr), env...)`. Let *`final-sndr`* be the expression *`transformed-sndr`* if *`transformed-sndr`* and `sndr` have the same type ignoring *cv* qualifiers; otherwise, it is the expression `transform_sender(dom, transformed-sndr, env...)`.

2. *Returns:* *`final-sndr`*.

3. *Remarks:* The exception specification is equivalent to `noexcept(final-sndr)`.

§ 34.9.6. `execution::transform_env` [exec.snd.transform.env]

```
namespace std::execution {
  template<class Domain, sender Sndr, queryable Env>
    constexpr queryable decltype(auto) transform_env(Domain dom, Sndr&& sndr, Env
}
```

1. Let `e` be the expression `dom.transform_env(std::forward<Sndr>(sndr), std::forward<Env>(env))` if that expression is well-formed; otherwise, `default_domain().transform_env(std::forward<Sndr>(sndr), std::forward<Env>(env))`.

2. *Mandates:* `noexcept(e)` is `true`.

3. *Returns:* `e`.

§ 34.9.7. `execution::apply_sender` [exec.snd.apply]

```
namespace std::execution {
```

```
  template<class Domain, class Tag, sender Sndr, class... Args>
    constexpr decltype(auto) apply_sender(Domain dom, Tag, Sndr&& sndr, Args&&...
      noexcept(see below);
}
```

1. Let *e* be the expression `dom.apply_sender(Tag(), std::forward<Sndr>(sndr), std::forward<Args>(args)...)` if that expression is well-formed; otherwise, `default_domain().apply_sender(Tag(), std::forward<Sndr>(sndr), std::forward<Args>(args)...)`.
2. *Constraints:* The expression *e* is well-formed.
3. *Returns:* *e*.
4. *Remarks:* The exception specification is equivalent to `noexcept(e)`.

§ 34.9.8. `execution::get_completion_signatures` [exec.getcomplsigs]

1. `get_completion_signatures` is a customization point object. Let `sndr` be an expression such that `decltype((sndr))` is `Sndr`, and let `env` be an expression such that `decltype((env))` is `Env`. Let `new_sndr` be the expression `transform_sender(decltype(get-domain-late(sndr, env)){}, sndr, env)`, and let `NewSndr` be `decltype((new_sndr))`. Then `get_completion_signatures(sndr, env)` is expression-equivalent to `(void(sndr), void(env), CS())` except that `void(sndr)` and `void(env)` are indeterminately sequenced, where *CS* is:

   1. `decltype(new_sndr.get_completion_signatures(env))` if that type is well-formed,
   2. Otherwise, `remove_cvref_t<NewSndr>::completion_signatures` if that type is well-formed,
   3. Otherwise, if `is-awaitable<NewSndr, env-promise<Env>>` is `true`, then:

      ```
      completion_signatures<
        SET-VALUE-SIG(await-result-type<NewSndr,
                      env-promise<Env>>), // see [exec.snd.concepts]
        set_error_t(exception_ptr),
        set_stopped_t()>
      ```

   4. Otherwise, *CS* is ill-formed.

2. Let `rcvr` be an rvalue whose type `Rcvr` models `receiver`, and let `Sndr` be the type of a sender such that `sender_in<Sndr, env_of_t<Rcvr>>` is `true`. Let `Sigs...` be the template arguments of the `completion_signatures` specialization named by `completion_signatures_of_t<Sndr, env_of_t<Rcvr>>`. Let *CSO* be a completion function. If sender `Sndr` or its operation state cause the expression `CSO(rcvr, args...)` to be potentially evaluated ([basic.def.odr]) then there shall be a signature `Sig` in `Sigs...` such that

MATCHING-SIG(*decayed-typeof*<*CSO*>(decltype(args)...), Sig) is true ([exec.general]).

§ 34.9.9. execution::connect [exec.connect]

1. connect connects ([async.ops]) a sender with a receiver.
2. The name connect denotes a customization point object. For subexpressions sndr and rcvr, let Sndr be decltype((sndr)) and Rcvr be decltype((rcvr)), let new_sndr be the expression transform_sender(decltype(*get-domain-late*(sndr, get_env(rcvr))){}, sndr, get_env(rcvr)), and let DS and DR be decay_t<decltype((new_sndr))> and decay_t<Rcvr>, respectively.
3. Let *connect-awaitable-promise* be the following exposition-only class:

```
namespace std::execution {
  struct connect-awaitable-promise
    : with-await-transform<connect-awaitable-promise> {

    connect-awaitable-promise(DS&, DR& rcvr) noexcept : rcvr(rcvr) {}

    suspend_always initial_suspend() noexcept { return {}; }
    [[noreturn]] suspend_always final_suspend() noexcept { terminate(); }
    [[noreturn]] void unhandled_exception() noexcept { terminate(); }
    [[noreturn]] void return_void() noexcept { terminate(); }

    coroutine_handle<> unhandled_stopped() noexcept {
      set_stopped(std::move(rcvr));
      return noop_coroutine();
    }

    operation-state-task get_return_object() noexcept {
      return operation-state-task{
        coroutine_handle<connect-awaitable-promise>::from_promise(*this)};
    }

    env_of_t<DR> get_env() const noexcept {
      return execution::get_env(rcvr);
    }

  private:
    DR& rcvr; // exposition only
  };
}
```

4. Let *operation-state-task* be the following exposition-only class:

```
namespace std::execution {
  struct operation-state-task {
    using operation_state_concept = operation_state_t;
    using promise_type = connect-awaitable-promise;

    explicit operation-state-task(coroutine_handle<> h) noexcept : coro(h) {}
    operation-state-task(operation-state-task&& o) noexcept
      : coro(exchange(o.coro, {})) {}
    ~operation-state-task() { if (coro) coro.destroy(); }

    void start() & noexcept {
      coro.resume();
    }

  private:
    coroutine_handle<> coro; // exposition only
  };
}
```

5. Let `V` name the type `await-result-type<DS, connect-awaitable-promise>`, let `Sigs` name the type:

```
completion_signatures<
  SET-VALUE-SIG(V), // see [exec.snd.concepts]
  set_error_t(exception_ptr),
  set_stopped_t()>
```

and let `connect-awaitable` be an exposition-only coroutine defined as follows:

```
namespace std::execution {
  template<class Fun, class... Ts>
  auto suspend-complete(Fun fun, Ts&&... as) noexcept { // exposition only
    auto fn = [&, fun]() noexcept { fun(std::forward<Ts>(as)...); };

    struct awaiter {
      decltype(fn) fn;

      static constexpr bool await_ready() noexcept { return false; }
      void await_suspend(coroutine_handle<>) noexcept { fn(); }
      [[noreturn]] void await_resume() noexcept { unreachable(); }
    };
    return awaiter{fn};
  }

  operation-state-task connect-awaitable(DS sndr, DR rcvr) requires receiver_
    exception_ptr ep;
```

```
      try {
        if constexpr (same_as<V, void>) {
          co_await std::move(sndr);
          co_await suspend-complete(set_value, std::move(rcvr));
        } else {
          co_await suspend-complete(set_value, std::move(rcvr), co_await std::m
        }
      } catch(...) {
        ep = current_exception();
      }
      co_await suspend-complete(set_error, std::move(rcvr), std::move(ep));
    }
  }
```

6. The expression `connect(sndr, rcvr)` is expression-equivalent to:

   1. `new_sndr.connect(rcvr)` if that expression is well-formed.

      — *Mandates:* The type of the expression above satisfies `operation_state`.

   2. Otherwise, *`connect-awaitable`*`(new_sndr, rcvr)`.

   3. *Mandates:* `sender<Sndr> && receiver<Rcvr>` is `true`.

§ 34.9.10. Sender factories [exec.factories]

§ 34.9.10.1. `execution::schedule` [exec.schedule]

1. `schedule` obtains a schedule sender ([async.ops]) from a scheduler.

2. The name `schedule` denotes a customization point object. For a subexpression `sch`, the expression `schedule(sch)` is expression-equivalent to `sch.schedule()`.

   1. If the expression `get_completion_scheduler<set_value_t>(get_env(sch.schedule())) == sch` is ill-formed or evaluates to `false`, the behavior of calling `schedule(sch)` is undefined.

   2. *Mandates:* The type of `sch.schedule()` satisfies `sender`.

§ 34.9.10.2. `execution::just`, `execution::just_error`, `execution::just_stopped` [exec.just]

1. `just`, `just_error`, and `just_stopped` are sender factories whose asynchronous operations complete synchronously in their start operation with a value completion operation, an error completion operation, or a stopped completion operation respectively.

2. The names `just`, `just_error`, and `just_stopped` denote customization point objects. Let *`just-cpo`* be one of `just`, `just_error`, or `just_stopped`. For a pack of subexpressions `ts`, let

`Ts` be the pack of types `decltype((ts))`. The expression *`just-cpo`*`(ts...)` is ill-formed if:

— (*`movable-value`*`<Ts> &&...)` is `false`, or

— *`just-cpo`* is `just_error` and `sizeof...(ts) == 1` is `false`, or

— *`just-cpo`* is `just_stopped` and `sizeof...(ts) == 0` is `false`;

Otherwise, it is expression-equivalent to *`make-sender`*(*`just-cpo`*, *`product-type`*`{ts...}`).

3. For `just`, `just_error`, and `just_stopped`, let *`set-cpo`* be `set_value`, `set_error`, and `set_stopped` respectively. The exposition-only class template *`impls-for`* ([exec.snd.general]) is specialized for *`just-cpo`* as follows:

```
namespace std::execution {
  template<>
  struct impls-for<decayed-typeof<just-cpo>> : default-impls {
    static constexpr auto start =
      [](auto& state, auto& rcvr) noexcept -> void {
        auto& [...ts] = state;
        set-cpo(std::move(rcvr), std::move(ts)...);
      };
  };
}
```

§ 34.9.10.3. `execution::read_env` [exec.read.env]

1. `read_env` is a sender factory for a sender whose asynchronous operation completes synchronously in its start operation with a value completion result equal to a value read from the receiver's associated environment.

2. `read_env` is a customization point object. For some query object `q`, the expression `read_env(q)` is expression-equivalent to *`make-sender`*(`read_env`, `q`).

3. The exposition-only class template *`impls-for`* ([exec.snd.general]) is specialized for `read_env` as follows:

```
namespace std::execution {
  template<>
  struct impls-for<decayed-typeof<read_env>> : default-impls {
    static constexpr auto start =
      [](auto query, auto& rcvr) noexcept -> void {
        TRY-SET-VALUE(rcvr, query(get_env(rcvr)));
      };
  };
}
```

§ 34.9.11. Sender adaptors [exec.adapt]

§ 34.9.11.1. General [exec.adapt.general]

1. [exec.adapt] specifies a set of sender adaptors.
2. The bitwise inclusive OR operator is overloaded for the purpose of creating sender chains. The adaptors also support function call syntax with equivalent semantics.
3. Unless otherwise specified:
   1. A sender adaptor is prohibited from causing observable effects, apart from moving and copying its arguments, before the returned sender is connected with a receiver using `connect`, and `start` is called on the resulting operation state.
   2. A parent sender ([async.ops]) with a single child sender `sndr` has an associated attribute object equal to `FWD-ENV(get_env(sndr))` ([exec.fwd.env]).
   3. A parent sender with more than one child sender has an associated attributes object equal to `empty_env{}`.
   4. When a parent sender is connected to a receiver `rcvr`, any receiver used to connect a child sender has an associated environment equal to `FWD-ENV(get_env(rcvr))`.

   These requirements apply to any function that is selected by the implementation of the sender adaptor.
4. If a sender returned from a sender adaptor specified in [exec.adapt] is specified to include `set_error_t(Err)` among its set of completion signatures where `decay_t<Err>` denotes the type `exception_ptr`, but the implementation does not potentially evaluate an error completion operation with an `exception_ptr` argument, the implementation is allowed to omit the `exception_ptr` error completion signature from the set.

§ 34.9.11.2. Sender adaptor closure objects [exec.adapt.objects]

1. A *pipeable sender adaptor closure object* is a function object that accepts one or more `sender` arguments and returns a `sender`. For a pipeable sender adaptor closure object `c` and an expression `sndr` such that `decltype((sndr))` models `sender`, the following expressions are equivalent and yield a `sender`:

   ```
   c(sndr)
   sndr | c
   ```

   Given an additional pipeable sender adaptor closure object `d`, the expression `c | d` produces another pipeable sender adaptor closure object `e`:

   `e` is a perfect forwarding call wrapper ([func.require]) with the following properties:

— Its target object is an object `d2` of type `decltype(auto(d))` direct-non-list-initialized with `d`.

— It has one bound argument entity, an object `c2` of type `decltype(auto(c))` direct-non-list-initialized with `c`.

— Its call pattern is `d2(c2(arg))`, where `arg` is the argument used in a function call expression of `e`.

The expression `c | d` is well-formed if and only if the initializations of the state entities ([func.def]) of `e` are all well-formed.

2. An object `t` of type `T` is a pipeable sender adaptor closure object if `T` models `derived_from<sender_adaptor_closure<T>>`, `T` has no other base classes of type `sender_adaptor_closure<U>` for any other type `U`, and `T` does not satisfy `sender`.

3. The template parameter `D` for `sender_adaptor_closure` can be an incomplete type. Before any expression of type `cv D` appears as an operand to the `|` operator, `D` shall be complete and model `derived_from<sender_adaptor_closure<D>>`. The behavior of an expression involving an object of type `cv D` as an operand to the `|` operator is undefined if overload resolution selects a program-defined `operator|` function.

4. A *pipeable sender adaptor object* is a customization point object that accepts a `sender` as its first argument and returns a `sender`.

5. If a pipeable sender adaptor object accepts only one argument, then it is a pipeable sender adaptor closure object.

6. If a pipeable sender adaptor object `adaptor` accepts more than one argument, then let `sndr` be an expression such that `decltype((sndr))` models `sender`, let `args...` be arguments such that `adaptor(sndr, args...)` is a well-formed expression as specified below, and let `BoundArgs` be a pack that denotes `decltype(auto(args))...`. The expression `adaptor(args...)` produces a pipeable sender adaptor closure object `f` that is a perfect forwarding call wrapper with the following properties:

   — Its target object is a copy of `adaptor`.

   — Its bound argument entities `bound_args` consist of objects of types `BoundArgs...` direct-non-list-initialized with `std::forward<decltype((args))>(args)...`, respectively.

   — Its call pattern is `adaptor(rcvr, bound_args...)`, where `rcvr` is the argument used in a function call expression of `f`.

   The expression `adaptor(args...)` is well-formed if and only if the initializations of the bound argument entities of the result, as specified above, are all well-formed.

§ 34.9.11.3. `execution::starts_on` [exec.starts.on]

1. `starts_on` adapts an input sender into a sender that will start on an execution agent belonging

to a particular scheduler's associated execution resource.

2. The name `starts_on` denotes a customization point object. For subexpressions `sch` and `sndr`, if `decltype((sch))` does not satisfy `scheduler`, or `decltype((sndr))` does not satisfy `sender`, `starts_on(sch, sndr)` is ill-formed.

3. Otherwise, the expression `starts_on(sch, sndr)` is expression-equivalent to:

```
transform_sender(
  query-or-default(get_domain, sch, default_domain()),
  make-sender(starts_on, sch, sndr))
```

   except that `sch` is evaluated only once.

4. Let `out_sndr` and `env` be subexpressions such that `OutSndr` is `decltype((out_sndr))`. If `sender-for<OutSndr, starts_on_t>` is `false`, then the expressions `starts_on.transform_env(out_sndr, env)` and `starts_on.transform_sender(out_sndr, env)` are ill-formed; otherwise:

   — `starts_on.transform_env(out_sndr, env)` is equivalent to:

```
auto&& [_, sch, _] = out_sndr;
return JOIN-ENV(SCHED-ENV(sch), FWD-ENV(env));
```

   — `starts_on.transform_sender(out_sndr, env)` is equivalent to:

```
auto&& [_, sch, sndr] = out_sndr;
return let_value(
  schedule(sch),
  [sndr = std::forward_like<OutSndr>(sndr)]() mutable
    noexcept(is_nothrow_move_constructible_v) {
    return std::move(sndr);
  });
```

5. Let `out_sndr` be a subexpression denoting a sender returned from `starts_on(sch, sndr)` or one equal to such, and let `OutSndr` be the type `decltype((out_sndr))`. Let `out_rcvr` be a subexpression denoting a receiver that has an environment of type `Env` such that `sender_in<OutSndr, Env>` is `true`. Let `op` be an lvalue referring to the operation state that results from connecting `out_sndr` with `out_rcvr`. Calling `start(op)` shall start `sndr` on an execution agent of the associated execution resource of `sch`. If scheduling onto `sch` fails, an error completion on `out_rcvr` shall be executed on an unspecified execution agent.

§ 34.9.11.4. `execution::continues_on` **[exec.continues.on]**

1. `continues_on` adapts a sender into one that completes on the specified scheduler.

2. The name `continues_on` denotes a pipeable sender adaptor object. For subexpressions `sch` and `sndr`, if `decltype((sch))` does not satisfy `scheduler`, or `decltype((sndr))` does not satisfy `sender`, `continues_on(sndr, sch)` is ill-formed.
3. Otherwise, the expression `continues_on(sndr, sch)` is expression-equivalent to:

```
transform_sender(
  get-domain-early(sndr),
  make-sender(continues_on, sch, sndr))
```

   except that `sndr` is evaluated only once.
4. The exposition-only class template *`impls-for`* is specialized for `continues_on_t` as follows:

```
namespace std::execution {
  template<>
  struct impls-for<continues_on_t> : default-impls {
    static constexpr auto get_attrs =
      [](const auto& data, const auto& child) noexcept -> decltype(auto) {
        return JOIN-ENV(SCHED-ATTRS(data), FWD-ENV(get_env(child)));
      };
  };
}
```

5. Let `sndr` and `env` be subexpressions such that `Sndr` is `decltype((sndr))`. If *`sender-for`*`<Sndr, continues_on_t>` is `false`, then the expression `continues_on.transform_sender(sndr, env)` is ill-formed; otherwise, it is equal to:

```
auto [_, data, child] = sndr;
return schedule_from(std::move(data), std::move(child));
```

   [Note: This causes the `continues_on(sndr, sch)` sender to become `schedule_from(sch, sndr)` when it is connected with a receiver whose execution domain does not customize `continues_on`. — *end note]*
6. Let `out_sndr` be a subexpression denoting a sender returned from `continues_on(sndr, sch)` or one equal to such, and let `OutSndr` be the type `decltype((out_sndr))`. Let `out_rcvr` be a subexpression denoting a receiver that has an environment of type `Env` such that `sender_in<OutSndr, Env>` is `true`. Let `op` be an lvalue referring to the operation state that results from connecting `out_sndr` with `out_rcvr`. Calling `start(op)` shall start `sndr` on the current execution agent and execute completion operations on `out_rcvr` on an execution agent of the execution resource associated with `sch`. If scheduling onto `sch` fails, an error completion on `out_rcvr` shall be executed on an unspecified execution agent.

§ 34.9.11.5. `execution::schedule_from` [exec.schedule.from]

1. `schedule_from` schedules work dependent on the completion of a sender onto a scheduler's associated execution resource. *[Note:* `schedule_from` is not meant to be used in user code; it is used in the implementation of `continues_on`. — *end note]*
2. The name `schedule_from` denotes a customization point object. For some subexpressions `sch` and `sndr`, let `Sch` be `decltype((sch))` and `Sndr` be `decltype((sndr))`. If `Sch` does not satisfy `scheduler`, or `Sndr` does not satisfy `sender`, `schedule_from(sch, sndr)` is ill-formed.
3. Otherwise, the expression `schedule_from(sch, sndr)` is expression-equivalent to:

```
transform_sender(
  query-or-default(get_domain, sch, default_domain()),
  make-sender(schedule_from, sch, sndr))
```

   except that `sch` is evaluated only once.
4. The exposition-only class template *`impls-for`* ([exec.snd.general]) is specialized for `schedule_from_t` as follows:

```
namespace std::execution {
  template<>
  struct impls-for<schedule_from_t> : default-impls {
    static constexpr auto get-attrs = see below;
    static constexpr auto get-state = see below;
    static constexpr auto complete = see below;
  };
}
```

   1. The member *`impls-for`*`<schedule_from_t>::`*`get-attrs`* is initialized with a callable object equivalent to the following lambda:

```
[](const auto& data, const auto& child) noexcept -> decltype(auto) {
  return JOIN-ENV(SCHED-ATTRS(data), FWD-ENV(get_env(child)));
}
```

   2. The member *`impls-for`*`<schedule_from_t>::`*`get-state`* is initialized with a callable object equivalent to the following lambda:

```
[]<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr) noexcept(see below)
    requires sender_in<child-type<Sndr>, env_of_t<Rcvr>> {

  auto& [_, sch, child] = sndr;

  using sched_t = decltype(auto(sch));
```

```
  using variant_t = see below;
  using receiver_t = see below;
  using operation_t = connect_result_t<schedule_result_t<sched_t>, receiv
  constexpr bool nothrow = noexcept(connect(schedule(sch), receiver_t{nul

  struct state-type {
    Rcvr& rcvr;          // exposition only
    variant_t async-result; // exposition only
    operation_t op-state;   // exposition only

    explicit state-type(sched_t sch, Rcvr& rcvr) noexcept(nothrow)
      : rcvr(rcvr), op-state(connect(schedule(sch), receiver_t{this})) {}
  };

  return state-type{sch, rcvr};
}
```

1. Objects of the local class `state-type` can be used to initialize a structured binding.
2. Let `Sigs` be a pack of the arguments to the `completion_signatures` specialization named by `completion_signatures_of_t<child-type<Sndr>, env_of_t<Rcvr>>`. Let `as-tuple` be an alias template that transforms a completion signature `Tag(Args...)` into the `tuple` specialization `decayed-tuple<Tag, Args...>`. Then `variant_t` denotes the type `variant<monostate, as-tuple<Sigs>...>`, except with duplicate types removed.
3. `receiver_t` is an alias for the following exposition-only class:

```
namespace std::execution {
  struct receiver-type {
    using receiver_concept = receiver_t;
    state-type* state; // exposition only

    void set_value() && noexcept {
      visit(
        [this]<class Tuple>(Tuple& result) noexcept -> void {
          if constexpr (!same_as<monostate, Tuple>) {
            auto& [tag, ...args] = result;
            tag(std::move(state->rcvr), std::move(args)...);
          }
        },
        state->async-result);
    }

    template<class Error>
    void set_error(Error&& err) && noexcept {
      execution::set_error(std::move(state->rcvr), std::forward<Error
    }
```

```
      void set_stopped() && noexcept {
        execution::set_stopped(std::move(state->rcvr));
      }

      decltype(auto) get_env() const noexcept {
        return FWD-ENV(execution::get_env(state->rcvr));
      }
    };
  }
```

4. The expression in the `noexcept` clause of the lambda is `true` if the construction of the returned *`state-type`* object is not potentially throwing; otherwise, `false`.

3. The member *`impls-for`*`<schedule_from_t>::`*`complete`* is initialized with a callable object equivalent to the following lambda:

```
[]<class Tag, class... Args>(auto, auto& state, auto& rcvr, Tag, Args&&..
  using result_t = decayed-tuple<Tag, Args...>;
  constexpr bool nothrow = is_nothrow_constructible_v<result_t, Tag, Args

  TRY-EVAL(rcvr, [&]() noexcept(nothrow) {
    state.async-result.template emplace<result_t>(Tag(), std::forward<Arg
  }());

  if (state.async-result.valueless_by_exception())
    return;
  if (state.async-result.index() == 0)
    return;

  start(state.op-state);
};
```

5. Let `out_sndr` be a subexpression denoting a sender returned from `schedule_from(sch, sndr)` or one equal to such, and let `OutSndr` be the type `decltype((out_sndr))`. Let `out_rcvr` be a subexpression denoting a receiver that has an environment of type `Env` such that `sender_in<OutSndr, Env>` is `true`. Let `op` be an lvalue referring to the operation state that results from connecting `out_sndr` with `out_rcvr`. Calling `start(op)` shall start `sndr` on the current execution agent and execute completion operations on `out_rcvr` on an execution agent of the execution resource associated with `sch`. If scheduling onto `sch` fails, an error completion on `out_rcvr` shall be executed on an unspecified execution agent.

§ 34.9.11.6. `execution::on` **[exec.on]**

1. The `on` sender adaptor has two forms:

— `on(sch, sndr)`, which starts a sender `sndr` on an execution agent belonging to a scheduler `sch`'s associated execution resource and that, upon `sndr`'s completion, transfers execution back to the execution resource on which the `on` sender was started.

— `on(sndr, sch, closure)`, which upon completion of a sender `sndr`, transfers execution to an execution agent belonging to a scheduler `sch`'s associated execution resource, then executes a sender adaptor closure `closure` with the async results of the sender, and that then transfers execution back to the execution resource on which `sndr` completed.

2. The name `on` denotes a pipeable sender adaptor object. For subexpressions `sch` and `sndr`, `on(sch, sndr)` is ill-formed if any of the following is true:

   — `decltype((sch))` does not satisfy `scheduler`, or

   — `decltype((sndr))` does not satisfy `sender` and `sndr` is not a pipeable sender adaptor closure object ([exec.adapt.objects]), or

   — `decltype((sndr))` satisfies `sender` and `sndr` is also a pipeable sender adaptor closure object.

3. Otherwise, if `decltype((sndr))` satisfies `sender`, the expression `on(sch, sndr)` is expression-equivalent to:

```
transform_sender(
  query-or-default(get_domain, sch, default_domain()),
  make-sender(on, sch, sndr))
```

   except that `sch` is evaluated only once.

4. For subexpressions `sndr`, `sch`, and `closure`, if `decltype((sch))` does not satisfy `scheduler`, or `decltype((sndr))` does not satisfy `sender`, or `closure` is not a pipeable sender adaptor closure object ([exec.adapt.objects]), the expression `on(sndr, sch, closure)` is ill-formed; otherwise, it is expression-equivalent to:

```
transform_sender(
  get-domain-early(sndr),
  make-sender(on, product-type{sch, closure}, sndr))
```

   except that `sndr` is evaluated only once.

5. Let `out_sndr` and `env` be subexpressions, let `OutSndr` be `decltype((out_sndr))`, and let `Env` be `decltype((env))`. If *`sender-for`*`<OutSndr, on_t>` is `false`, then the expressions `on.transform_env(out_sndr, env)` and `on.transform_sender(out_sndr, env)` are ill-formed; otherwise:

   1. Let *`not-a-scheduler`* be an unspecified empty class type, and let *`not-a-sender`* be the exposition-only type:

```
struct not-a-sender {
  using sender_concept = sender_t;

  auto get_completion_signatures(auto&&) const {
    return see below;
  }
};
```

where the member function `get_completion_signatures` returns an object of a type that is not a specialization of the `completion_signatures` class template.

2. The expression `on.transform_env(out_sndr, env)` has effects equivalent to:

```
auto&& [_, data, _] = out_sndr;
if constexpr (scheduler<decltype(data)>) {
  return JOIN-ENV(SCHED-ENV(std::forward_like<OutSndr>(data)), FWD-ENV(st
} else {
  return std::forward<Env>(env);
}
```

3. The expression `on.transform_sender(out_sndr, env)` has effects equivalent to:

```
auto&& [_, data, child] = out_sndr;
if constexpr (scheduler<decltype(data)>) {
  auto orig_sch =
    query-with-default(get_scheduler, env, not-a-scheduler());

  if constexpr (same_as<decltype(orig_sch), not-a-scheduler>) {
    return not-a-sender{};
  } else {
    return continues_on(
      starts_on(std::forward_like<OutSndr>(data), std::forward_like<OutSn
      std::move(orig_sch));
  }
} else {
  auto& [sch, closure] = data;
  auto orig_sch = query-with-default(
    get_completion_scheduler<set_value_t>,
    get_env(child),
    query-with-default(get_scheduler, env, not-a-scheduler()));

  if constexpr (same_as<decltype(orig_sch), not-a-scheduler>) {
    return not-a-sender{};
  } else {
    return write-env(
      continues_on(
```

```
            std::forward_like<OutSndr>(closure)(
              continues_on(
                write-env(std::forward_like<OutSndr>(child), SCHED-ENV(orig_s
                sch)),
            orig_sch),
          SCHED-ENV(sch));
      }
    }
```

4. *Recommended practice:* Implementations should use the return type of `not-a-sender::get_completion_signatures` to inform users that their usage of `on` is incorrect because there is no available scheduler onto which to restore execution.

6. Let `out_sndr` be a subexpression denoting a sender returned from `on(sch, sndr)` or one equal to such, and let `OutSndr` be the type `decltype((out_sndr))`. Let `out_rcvr` be a subexpression denoting a receiver that has an environment of type `Env` such that `sender_in<OutSndr, Env>` is `true`. Let `op` be an lvalue referring to the operation state that results from connecting `out_sndr` with `out_rcvr`. Calling `start(op)` shall:

   1. Remember the current scheduler, `get_scheduler(get_env(rcvr))`.
   2. Start `sndr` on an execution agent belonging to `sch`'s associated execution resource.
   3. Upon `sndr`'s completion, transfer execution back to the execution resource associated with the scheduler remembered in step 1.
   4. Forward `sndr`'s async result to `out_rcvr`.

   If any scheduling operation fails, an error completion on `out_rcvr` shall be executed on an unspecified execution agent.

7. Let `out_sndr` be a subexpression denoting a sender returned from `on(sndr, sch, closure)` or one equal to such, and let `OutSndr` be the type `decltype((out_sndr))`. Let `out_rcvr` be a subexpression denoting a receiver that has an environment of type `Env` such that `sender_in<OutSndr, Env>` is `true`. Let `op` be an lvalue referring to the operation state that results from connecting `out_sndr` with `out_rcvr`. Calling `start(op)` shall:

   1. Remember the current scheduler, which is the first of the following expressions that is well-formed:

      — `get_completion_scheduler<set_value_t>(get_env(sndr))`

      — `get_scheduler(get_env(rcvr))`

   2. Start `sndr` on the current execution agent.
   3. Upon `sndr`'s completion, transfer execution to an agent owned by `sch`'s associated execution resource.
   4. Forward `sndr`'s async result as if by connecting and starting a sender `closure(S)`, where `S` is a sender that completes synchronously with `sndr`'s async result.

5. Upon completion of the operation started in step 4, transfer execution back to the execution resource associated with the scheduler remembered in step 1 and forward the operation's async result to `out_rcvr`.

If any scheduling operation fails, an error completion on `out_rcvr` shall be executed on an unspecified execution agent.

§ 34.9.11.7. `execution::then`, `execution::upon_error`, `execution::upon_stopped` [exec.then]

1. `then` attaches an invocable as a continuation for an input sender's value completion operation. `upon_error` and `upon_stopped` do the same for the error and stopped completion operations respectively, sending the result of the invocable as a value completion.
2. The names `then`, `upon_error`, and `upon_stopped` denote pipeable sender adaptor objects. Let the expression *`then-cpo`* be one of `then`, `upon_error`, or `upon_stopped`. For subexpressions `sndr` and `f`, if `decltype((sndr))` does not satisfy `sender`, or `decltype((f))` does not satisfy *`movable-value`*, *`then-cpo`*`(sndr, f)` is ill-formed.
3. Otherwise, the expression *`then-cpo`*`(sndr, f)` is expression-equivalent to:

```
transform_sender(
  get-domain-early(sndr),
  make-sender(then-cpo, f, sndr))
```

   except that `sndr` is evaluated only once.
4. For `then`, `upon_error`, and `upon_stopped`, let *`set-cpo`* be `set_value`, `set_error`, and `set_stopped` respectively. The exposition-only class template *`impls-for`* ([exec.snd.general]) is specialized for *`then-cpo`* as follows:

```
namespace std::execution {
  template<>
  struct impls-for<decayed-typeof<then-cpo>> : default-impls {
    static constexpr auto complete =
      []<class Tag, class... Args>
        (auto, auto& fn, auto& rcvr, Tag, Args&&... args) noexcept -> void {
          if constexpr (same_as<Tag, decayed-typeof<set-cpo>>) {
            TRY-SET-VALUE(rcvr,
                          invoke(std::move(fn), std::forward<Args>(args)...))
          } else {
            Tag()(std::move(rcvr), std::forward<Args>(args)...);
          }
        };
  };
}
```

5. The expression `then-cpo(sndr, f)` has undefined behavior unless it returns a sender `out_sndr` that:

    1. Invokes `f` or a copy of such with the value, error, or stopped result datums of `sndr` for `then`, `upon_error`, and `upon_stopped` respectively, using the result value of `f` as `out_sndr`'s value completion, and
    2. Forwards all other completion operations unchanged.

§ 34.9.11.8. `execution::let_value`, `execution::let_error`, `execution::let_stopped`, **[exec.let]**

1. `let_value`, `let_error`, and `let_stopped` transform a sender's value, error, and stopped completions respectively into a new child asynchronous operation by passing the sender's result datums to a user-specified callable, which returns a new sender that is connected and started.
2. For `let_value`, `let_error`, and `let_stopped`, let `set-cpo` be `set_value`, `set_error`, and `set_stopped` respectively. Let the expression `let-cpo` be one of `let_value`, `let_error`, or `let_stopped`. For a subexpression `sndr`, let `let-env(sndr)` be expression-equivalent to the first well-formed expression below:

    — `SCHED-ENV(get_completion_scheduler<decayed-typeof<set-cpo>>(get_env(sndr)))`

    — `MAKE-ENV(get_domain, get_domain(get_env(sndr)))`

    — `(void(sndr), empty_env{})`
3. The names `let_value`, `let_error`, and `let_stopped` denote pipeable sender adaptor objects. For subexpressions `sndr` and `f`, let `F` be the decayed type of `f`. If `decltype((sndr))` does not satisfy `sender` or if `decltype((f))` does not satisfy `movable-value`, the expression `let-cpo(sndr, f)` is ill-formed. If `F` does not satisfy `invocable`, the expression `let_stopped(sndr, f)` is ill-formed.
4. Otherwise, the expression `let-cpo(sndr, f)` is expression-equivalent to:

```
transform_sender(
  get-domain-early(sndr),
  make-sender(let-cpo, f, sndr))
```

    except that `sndr` is evaluated only once.
5. The exposition-only class template `impls-for` ([exec.snd.general]) is specialized for `let-cpo` as follows:

```
namespace std::execution {
  template<class State, class Rcvr, class... Args>
  void let-bind(State& state, Rcvr& rcvr, Args&&... args); // exposition only
```

```
  template<>
  struct impls-for<decayed-typeof<let-cpo>> : default-impls {
    static constexpr auto get-state = see below;
    static constexpr auto complete = see below;
  };
}
```

1. Let *receiver2* denote the following exposition-only class template:

```
namespace std::execution {
  template<class Rcvr, class Env>
  struct receiver2 {
    using receiver_concept = receiver_t;

    template<class... Args>
    void set_value(Args&&... args) && noexcept {
      execution::set_value(std::move(rcvr), std::forward<Args>(args)...);
    }

    template<class Error>
    void set_error(Error&& err) && noexcept {
      execution::set_error(std::move(rcvr), std::forward<Error>(err));
    }

    void set_stopped() && noexcept {
      execution::set_stopped(std::move(rcvr));
    }

    decltype(auto) get_env() const noexcept {
      return JOIN-ENV(env, FWD-ENV(execution::get_env(rcvr)));
    }

    Rcvr& rcvr; // exposition only
    Env env; // exposition only
  };
}
```

2. *impls-for<decayed-typeof<let-cpo>>::get-state* is initialized with a callable object equivalent to the following:

```
[]<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr) requires see below {
  auto& [_, fn, child] = sndr;
  using fn_t = decay_t<decltype(fn)>;
  using env_t = decltype(let-env(child));
  using args_variant_t = see below;
  using ops2_variant_t = see below;
```

```
  struct state-type {
    fn_t fn;    // exposition only
    env_t env;    // exposition only
    args_variant_t args;    // exposition only
    ops2_variant_t ops2;    // exposition only
  };
  return state-type{std::forward_like<Sndr>(fn), let-env(child), {}, {}};
}
```

1. Let `Sigs` be a pack of the arguments to the `completion_signatures` specialization named by `completion_signatures_of_t<child-type<Sndr>, env_of_t<Rcvr>>`. Let `LetSigs` be a pack of those types in `Sigs` with a return type of `decayed-typeof<set-cpo>`. Let `as-tuple` be an alias template such that `as-tuple<Tag(Args...)>` denotes the type `decayed-tuple<Args...>`. Then `args_variant_t` denotes the type `variant<monostate, as-tuple<LetSigs>...>` except with duplicate types removed.
2. Given a type `Tag` and a pack `Args`, let `as-sndr2` be an alias template such that `as-sndr2<Tag(Args...)>` denotes the type `call-result-t<Fn, decay_t<Args>&...>`. Then `ops2_variant_t` denotes the type `variant<monostate, connect_result_t<as-sndr2<LetSigs>, receiver2<Rcvr, Env>>...>` except with duplicate types removed.
3. The *requires-clause* constraining the above lambda is satisfied if and only if the types `args_variant_t` and `ops2_variant_t` are well-formed.

3. The exposition-only function template `let-bind` has effects equivalent to:

```
using args_t = decayed-tuple<Args...>;
auto mkop2 = [&] {
  return connect(
    apply(std::move(state.fn),
          state.args.template emplace<args_t>(std::forward<Args>(args)...
    receiver2{rcvr, std::move(state.env)});
};
start(state.ops2.template emplace<decltype(mkop2())>(emplace-from{mkop2})
```

4. `impls-for<decayed-typeof<let-cpo>>::complete` is initialized with a callable object equivalent to the following:

```
[]<class Tag, class... Args>
  (auto, auto& state, auto& rcvr, Tag, Args&&... args) noexcept -> void {
    if constexpr (same_as<Tag, decayed-typeof<set-cpo>>) {
      TRY-EVAL(rcvr, let-bind(state, rcvr, std::forward<Args>(args)...));
    } else {
      Tag()(std::move(rcvr), std::forward<Args>(args)...);
```

```
    }
  }
```

6. Let `sndr` and `env` be subexpressions, and let `Sndr` be `decltype((sndr))`. If *`sender-for`*`<Sndr, decayed-typeof<let-cpo>>` is `false`, then the expression *`let-cpo`*`.transform_env(sndr, env)` is ill-formed. Otherwise, it is equal to *`JOIN-ENV`*`(`*`let-env`*`(sndr), `*`FWD-ENV`*`(env))`.
7. Let the subexpression `out_sndr` denote the result of the invocation *`let-cpo`*`(sndr, f)` or an object equal to such, and let the subexpression `rcvr` denote a receiver such that the expression `connect(out_sndr, rcvr)` is well-formed. The expression `connect(out_sndr, rcvr)` has undefined behavior unless it creates an asynchronous operation ([async.ops]) that, when started:

   — invokes `f` when *`set-cpo`* is called with `sndr`'s result datums,

   — makes its completion dependent on the completion of a sender returned by `f`, and

   — propagates the other completion operations sent by `sndr`.

§ 34.9.11.9. `execution::bulk` [exec.bulk]

1. `bulk` runs a task repeatedly for every index in an index space.
2. The name `bulk` denotes a pipeable sender adaptor object. For subexpressions `sndr`, `shape`, and `f`, let `Shape` be `decltype(auto(shape))`. If `decltype((sndr))` does not satisfy `sender`, or if `Shape` does not satisfy `integral`, or if `decltype((f))` does not satisfy *`movable-value`*, `bulk(sndr, shape, f)` is ill-formed.
3. Otherwise, the expression `bulk(sndr, shape, f)` is expression-equivalent to:

```
transform_sender(
  get-domain-early(sndr),
  make-sender(bulk, product-type{shape, f}, sndr))
```

   except that `sndr` is evaluated only once.

4. The exposition-only class template *`impls-for`* ([exec.snd.general]) is specialized for `bulk_t` as follows:

```
namespace std::execution {
  template<>
  struct impls-for<bulk_t> : default-impls {
    static constexpr auto complete = see below;
  };
}
```

   1. The member *`impls-for`*`<bulk_t>::complete` is initialized with a callable object equiva-

lent to the following lambda:

```
[]<class Index, class State, class Rcvr, class Tag, class... Args>
  (Index, State& state, Rcvr& rcvr, Tag, Args&&... args) noexcept -> void
    if constexpr (same_as<Tag, set_value_t>) {
      auto& [shape, f] = state;
      constexpr bool nothrow = noexcept(f(auto(shape), args...));
      TRY-EVAL(rcvr, [&]() noexcept(nothrow) {
        for (decltype(auto(shape)) i = 0; i < shape; ++i) {
          f(auto(i), args...);
        }
        Tag()(std::move(rcvr), std::forward<Args>(args)...);
      }());
    } else {
      Tag()(std::move(rcvr), std::forward<Args>(args)...);
    }
  }
```

1. The expression in the *requires-clause* of the lambda above is `true` if and only if `Tag` denotes a type other than `set_value_t` or if the expression `f(auto(shape), args...)` is well-formed.

5. Let the subexpression `out_sndr` denote the result of the invocation `bulk(sndr, shape, f)` or an object equal to such, and let the subexpression `rcvr` denote a receiver such that the expression `connect(out_sndr, rcvr)` is well-formed. The expression `connect(out_sndr, rcvr)` has undefined behavior unless it creates an asynchronous operation ([async.ops]) that, when started:

   — on a value completion operation, invokes `f(i, args...)` for every `i` of type `Shape` from `0` to `shape`, where `args` is a pack of lvalue subexpressions referring to the value completion result datums of the input sender, and

   — propagates all completion operations sent by `sndr`.

§ 34.9.11.10. `execution::split` [exec.split]

1. `split` adapts an arbitrary sender into a sender that can be connected multiple times.

2. Let *`split-env`* be the type of an environment such that, given an instance `env`, the expression `get_stop_token(env)` is well-formed and has type `inplace_stop_token`.

3. The name `split` denotes a pipeable sender adaptor object. For a subexpression `sndr`, let `Sndr` be `decltype((sndr))`. If `sender_in<Sndr, split-env>` is `false`, `split(sndr)` is ill-formed.

4. Otherwise, the expression `split(sndr)` is expression-equivalent to:

```
transform_sender(
  get-domain-early(sndr),
  make-sender(split, {}, sndr))
```

except that `sndr` is evaluated only once.

— *[Note:* The default implementation of `transform_sender` will have the effect of connecting the sender to a receiver. It will return a sender with a different tag type. — *end note]*

5. Let *`local-state`* denote the following exposition-only class template:

```
namespace std::execution {
  struct local-state-base {          // exposition only
    virtual ~local-state-base() = default;
    virtual void notify() noexcept = 0; // exposition only
  };

  template<class Sndr, class Rcvr>
  struct local-state : local-state-base { // exposition only
    using on-stop-callback =     // exposition only
      stop_callback_of_t<stop_token_of_t<env_of_t<Rcvr>>, on-stop-request>;

    local-state(Sndr&& sndr, Rcvr& rcvr) noexcept;
    ~local-state();

    void notify() noexcept override;

  private:
    optional<on-stop-callback> on_stop; // exposition only
    shared-state<Sndr>* sh_state; // exposition only
    Rcvr* rcvr; // exposition only
  };
}
```

1. `local-state(Sndr&& sndr, Rcvr& rcvr) noexcept;`

   1. *Effects:* Equivalent to:

```
auto& [_, data, _] = sndr;
this->sh_state = data.sh_state.get();
this->sh_state->inc-ref();
this->rcvr = addressof(rcvr);
```

2. `~local-state();`

1. *Effects:* Equivalent to:

```
sh_state->dec-ref();
```

3. `void notify() noexcept override;`

1. *Effects:* Equivalent to:

```
on_stop.reset();
visit(
  [this](const auto& tupl) noexcept -> void {
    apply(
      [this](auto tag, const auto&... args) noexcept -> void {
        tag(std::move(*rcvr), args...);
      },
      tupl);
  },
  sh_state->result);
```

6. Let `split-receiver` denote the following exposition-only class template:

```
namespace std::execution {
  template<class Sndr>
  struct split-receiver {
    using receiver_concept = receiver_t;

    template<class Tag, class... Args>
    void complete(Tag, Args&&... args) noexcept { // exposition only
      using tuple_t = decayed-tuple<Tag, Args...>;
      try {
        sh_state->result.template emplace<tuple_t>(Tag(), std::forward<Args>(
      } catch (...) {
        using tuple_t = tuple<set_error_t, exception_ptr>;
        sh_state->result.template emplace<tuple_t>(set_error, current_excepti
      }
      sh_state->notify();
    }

    template<class... Args>
    void set_value(Args&&... args) && noexcept {
      complete(execution::set_value, std::forward<Args>(args)...);
    }

    template<class Error>
    void set_error(Error&& err) && noexcept {
```

```
      complete(execution::set_error, std::forward<Error>(err));
    }

    void set_stopped() && noexcept {
      complete(execution::set_stopped);
    }

    struct env { // exposition only
      shared-state<Sndr>* sh-state; // exposition only

      inplace_stop_token query(get_stop_token_t) const noexcept {
        return sh-state->stop_src.get_token();
      }
    };

    env get_env() const noexcept {
      return env{sh_state};
    }

    shared-state<Sndr>* sh_state; // exposition only
  };
}
```

7. Let *shared-state* denote the following exposition-only class template:

```
namespace std::execution {
  template<class Sndr>
  struct shared-state {
    using variant-type = see below;   // exposition only
    using state-list-type = see below;    // exposition only

    explicit shared-state(Sndr&& sndr);

    void start-op() noexcept;  // exposition only
    void notify() noexcept;  // exposition only
    void inc-ref() noexcept; // exposition only
    void dec-ref() noexcept; // exposition only

    inplace_stop_source stop_src{};   // exposition only
    variant-type result{};   // exposition only
    state-list-type waiting_states;    // exposition only
    atomic<bool> completed{false};   // exposition only
    atomic<size_t> ref_count{1};   // exposition only
    connect_result_t<Sndr, split-receiver<Sndr>> op_state;    // exposition o
  };
}
```

1. Let `Sigs` be a pack of the arguments to the `completion_signatures` specialization named by `completion_signatures_of_t<Sndr>`. For type `Tag` and pack `Args`, let *`as-tuple`* be an alias template such that *`as-tuple`*`<Tag(Args...)>` denotes the type *`decayed-tuple`*`<Tag, Args...>`. Then *`variant-type`* denotes the type `variant<tuple<set_stopped_t>, tuple<set_error_t, exception_ptr>,` *`as-tuple`*`<Sigs>...>`, but with duplicate types removed.
2. Let *`state-list-type`* be a type that stores a list of pointers to *`local-state-base`* objects and that permits atomic insertion.
3. `explicit` *`shared-state`*`(Sndr&& sndr);`
   1. *Effects:* Initializes `op_state` with the result of `connect(std::forward<Sndr>(sndr),` *`split-receiver`*`{this})`.
   2. *Postcondition:* `waiting_states` is empty, and `completed` is `false`.
4. `void` *`start-op`*`() noexcept;`
   1. *Effects:* Calls *`inc-ref`*`()`. If `stop_src.stop_requested()` is `true`, calls *`notify`*`();` otherwise, calls `start(op_state)`.
5. `void` *`notify`*`() noexcept;`
   1. *Effects:* Atomically does the following:
      — Sets `completed` to `true`, and
      — Exchanges `waiting_states` with an empty list, storing the old value in a local `prior_states`.

      Then, for each pointer `p` in `prior_states`, calls `p->`*`notify`*`()`. Finally, calls *`dec-ref`*`()`.
6. `void` *`inc-ref`*`() noexcept;`
   1. *Effects:* Increments `ref_count`.
7. `void` *`dec-ref`*`() noexcept;`
   1. *Effects:* Decrements `ref_count`. If the new value of `ref_count` is `0`, calls `delete this`.
   2. *Synchronization:* If *`dec-ref`*`()` does not decrement the `ref_count` to `0` then synchronizes with the call to *`dec-ref`*`()` that decrements `ref_count` to `0`.

8. Let *`split-impl-tag`* be an empty exposition-only class type. Given an expression `sndr`, the expression `split.transform_sender(sndr)` is equivalent to:

```
auto&& [tag, _, child] = sndr;
auto* sh_state = new shared-state{std::forward_like<decltype((sndr))>(child)}
return make-sender(split-impl-tag(), shared-wrapper{sh_state, tag});
```

where *shared-wrapper* is an exposition-only class that manages the reference count of the *shared-state* object pointed to by `sh_state`. *shared-wrapper* models `copyable` with move operations nulling out the moved-from object, copy operations incrementing the reference count by calling `sh_state->inc-ref()`, and assignment operations performing a copy-and-swap operation. The destructor has no effect if `sh_state` is null; otherwise, it decrements the reference count by calling `sh_state->dec-ref()`.

9. The exposition-only class template *impls-for* ([exec.snd.general]) is specialized for *split-impl-tag* as follows:

```
namespace std::execution {
  template<>
  struct impls-for<split-impl-tag> : default-impls {
    static constexpr auto get-state = see below;
    static constexpr auto start = see below;
  };
}
```

   1. The member *impls-for*`<`*split-impl-tag*`>::`*get-state* is initialized with a callable object equivalent to the following lambda expression:

```
[]<class Sndr>(Sndr&& sndr, auto& rcvr) noexcept {
  return local-state{std::forward<Sndr>(sndr), rcvr};
}
```

   2. The member *impls-for*`<`*split-impl-tag*`>::`*start* is initialized with a callable object that has a function call operator equivalent to the following:

```
template<class Sndr, class Rcvr>
void operator()(local-state<Sndr, Rcvr>& state, Rcvr& rcvr) const noexcep
```

      1. *Effects:* If `state.sh_state->completed` is `true`, calls `state.notify()` and returns. Otherwise, does the following in order:

         1. Calls:

```
state.on_stop.emplace(
  get_stop_token(get_env(rcvr)),
  on-stop-request{state.sh_state->stop_src});
```

2. Then atomically does the following:

   — Reads the value `c` of `state.sh_state->completed`, and

   — Inserts `addressof(state)` into `state.sh_state->waiting_states` if `c` is `false`.

3. If `c` is `true`, calls `state.notify()` and returns.

4. Otherwise, if `addressof(state)` is the first item added to `state.sh_state->waiting_states`, calls `state.sh_state->start-op()`.

§ 34.9.11.11. `execution::when_all` [exec.when.all]

1. `when_all` and `when_all_with_variant` both adapt multiple input senders into a sender that completes when all input senders have completed. `when_all` only accepts senders with a single value completion signature and on success concatenates all the input senders' value result datums into its own value completion operation. `when_all_with_variant(sndrs...)` is semantically equivalent to `when_all(into_variant(sndrs)...)`, where `sndrs` is a pack of subexpressions whose types model `sender`.

2. The names `when_all` and `when_all_with_variant` denote customization point objects. Let `sndrs` be a pack of subexpressions, let `Sndrs` be a pack of the types `decltype((sndrs))...`, and let *`CD`* be the type `common_type_t<decltype(get-domain-early(sndrs))...>`. The expressions `when_all(sndrs...)` and `when_all_with_variant(sndrs...)` are ill-formed if any of the following is true:

   — `sizeof...(sndrs)` is 0, or

   — `(sender<Sndrs> && ...)` is `false`, or

   — *`CD`* is ill-formed.

3. The expression `when_all(sndrs...)` is expression-equivalent to:

```
transform_sender(
  CD(),
  make-sender(when_all, {}, sndrs...))
```

4. The exposition-only class template *`impls-for`* ([exec.snd.general]) is specialized for `when_all_t` as follows:

```
namespace std::execution {
  template<>
  struct impls-for<when_all_t> : default-impls {
    static constexpr auto get-attrs = see below;
    static constexpr auto get-env = see below;
    static constexpr auto get-state = see below;
```

```
    static constexpr auto start = see below;
    static constexpr auto complete = see below;
  };
}
```

1. The member *impls-for*<when_all_t>::*get-attrs* is initialized with a callable object equivalent to the following lambda expression:

```
[](auto&&, auto&&... child) noexcept {
  if constexpr (same_as<CD, default_domain>) {
    return empty_env();
  } else {
    return MAKE-ENV(get_domain, CD());
  }
}
```

2. The member *impls-for*<when_all_t>::*get-env* is initialized with a callable object equivalent to the following lambda expression:

```
[]<class State, class Rcvr>(auto&&, State& state, const Receiver& rcvr) n
  return JOIN-ENV(
    MAKE-ENV(get_stop_token, state.stop_src.get_token()), get_env(rcvr));
}
```

3. The member *impls-for*<when_all_t>::*get-state* is initialized with a callable object equivalent to the following lambda expression:

```
[]<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr) noexcept(e) -> declty
  return e;
}
```

where *e* is the expression:

```
std::forward<Sndr>(sndr).apply(make-state<Rcvr>())
```

and where *make-state* is the following exposition-only class template:

```
template<class Sndr, class Env>
concept max-1-sender-in = sender_in<Sndr, Env> && // exposition only
  (tuple_size_v<value_types_of_t<Sndr, Env, tuple, tuple>> <= 1);

enum class disposition { started, error, stopped }; // exposition only

template<class Rcvr>
```

```
struct make-state {
  template<max-1-sender-in<env_of_t<Rcvr>>... Sndrs>
  auto operator()(auto, auto, Sndrs&&... sndrs) const {
    using values_tuple = see below;
    using errors_variant = see below;
    using stop_callback = stop_callback_of_t<stop_token_of_t<env_of_t<Rcv

    struct state-type {
      void arrive(Rcvr& rcvr) noexcept {
        if (0 == --count) {
          complete(rcvr);
        }
      }

      void complete(Rcvr& rcvr) noexcept; // see below

      atomic<size_t> count{sizeof...(sndrs)};   // exposition only
      inplace_stop_source stop_src{};   // exposition only
      atomic<disposition> disp{disposition::started};   // exposition onl
      errors_variant errors{};   // exposition only
      values_tuple values{};   // exposition only
      optional<stop_callback> on_stop{nullopt};   // exposition only
    };

    return state-type{};
  }
};
```

1. Let *copy-fail* be `exception_ptr` if decay-copying any of the child senders' result datums can potentially throw; otherwise, `none-such`, where `none-such` is an unspecified empty class type.
2. The alias `values_tuple` denotes the type `tuple<value_types_of_t<Sndrs, env_of_t<Rcvr>, decayed-tuple, optional>...>` if that type is well-formed; otherwise, `tuple<>`.
3. The alias `errors_variant` denotes the type `variant<none-such, copy-fail, Es...>` with duplicate types removed, where `Es` is the pack of the decayed types of all the child senders' possible error result datums.
4. The member `void state::complete(Rcvr& rcvr) noexcept` behaves as follows:
    1. If `disp` is equal to `disposition::started`, evaluates:

```
auto tie = []<class... T>(tuple<T...>& t) noexcept { return tuple
auto set = [&](auto&... t) noexcept { set_value(std::move(rcvr),

on_stop.reset();
```

```
apply(
  [&](auto&... opts) noexcept {
    apply(set, tuple_cat(tie(*opts)...));
  },
  values);
```

2. Otherwise, if `disp` is equal to *`disposition::error`*, evaluates:

```
on_stop.reset();
visit(
  [&]<class Error>(Error& error) noexcept {
    if constexpr (!same_as<Error, none-such>) {
      set_error(std::move(rcvr), std::move(error));
    }
  },
  errors);
```

3. Otherwise, evaluates:

```
on_stop.reset();
set_stopped(std::move(rcvr));
```

4. The member *`impls-for`*`<when_all_t>::`*`start`* is initialized with a callable object equivalent to the following lambda expression:

```
[]<class State, class Rcvr, class... Ops>(
    State& state, Rcvr& rcvr, Ops&... ops) noexcept -> void {
  state.on_stop.emplace(
    get_stop_token(get_env(rcvr)),
    on-stop-request{state.stop_src});
  if (state.stop_src.stop_requested()) {
    state.on_stop.reset();
    set_stopped(std::move(rcvr));
  } else {
    (start(ops), ...);
  }
}
```

5. The member *`impls-for`*`<when_all_t>::`*`complete`* is initialized with a callable object equivalent to the following lambda expression:

```
[]<class Index, class State, class Rcvr, class Set, class... Args>(
    this auto& complete, Index, State& state, Rcvr& rcvr, Set, Args&&...
  if constexpr (same_as<Set, set_error_t>) {
    if (disposition::error != state.disp.exchange(disposition::error)) {
```

```
        state.stop_src.request_stop();
        TRY-EMPLACE-ERROR(state.errors, std::forward<Args>(args)...);
      }
    } else if constexpr (same_as<Set, set_stopped_t>) {
      auto expected = disposition::started;
      if (state.disp.compare_exchange_strong(expected, disposition::stoppea
        state.stop_src.request_stop();
      }
    } else if constexpr (!same_as<decltype(State::values), tuple<>>) {
      if (state.disp == disposition::started) {
        auto& opt = get<Index::value>(state.values);
        TRY-EMPLACE-VALUE(complete, opt, std::forward<Args>(args)...);
      }
    }

    state.arrive(rcvr);
  }
```

where `TRY-EMPLACE-ERROR(v, e)`, for subexpressions `v` and `e`, is equivalent to:

```
try {
  v.template emplace<decltype(auto(e))>(e);
} catch (...) {
  v.template emplace<exception_ptr>(current_exception());
}
```

if the expression `decltype(auto(e))(e)` is potentially throwing; otherwise, `v.template emplace<decltype(auto(e))>(e)`; and where `TRY-EMPLACE-VALUE(c, o, as...)`, for subexpressions `c`, `o`, and pack of subexpressions `as`, is equivalent to:

```
try {
  o.emplace(as...);
} catch (...) {
  c(Index(), state, rcvr, set_error, current_exception());
  return;
}
```

if the expression `decayed-tuple<decltype(as)...>{as...}` is potentially throwing; otherwise, `o.emplace(as...)`.

5. The expression `when_all_with_variant(sndrs...)` is expression-equivalent to:

```
transform_sender(
  CD(),
  make-sender(when_all_with_variant, {}, sndrs...));
```

6. Given subexpressions `sndr` and `env`, if *`sender-for`*`<decltype((sndr)), when_all_with_variant_t>` is `false`, then the expression `when_all_with_variant.transform_sender(sndr, env)` is ill-formed; otherwise, it is equivalent to:

```
auto&& [_, _, ...child] = sndr;
return when_all(into_variant(std::forward_like<decltype((sndr))>(child))...);
```

[Note: This causes the `when_all_with_variant(sndrs...)` sender to become `when_all(into_variant(sndrs)...)` when it is connected with a receiver whose execution domain does not customize `when_all_with_variant`. — *end note]*

§ 34.9.11.12. `execution::into_variant` [exec.into.variant]

1. `into_variant` adapts a sender with multiple value completion signatures into a sender with just one value completion signature consisting of a `variant` of `tuple`s.
2. The name `into_variant` denotes a pipeable sender adaptor object. For a subexpression `sndr`, let `Sndr` be `decltype((sndr))`. If `Sndr` does not satisfy `sender`, `into_variant(sndr)` is ill-formed.
3. Otherwise, the expression `into_variant(sndr)` is expression-equivalent to:

```
transform_sender(
  get-domain-early(sndr),
  make-sender(into_variant, {}, sndr))
```

except that `sndr` is only evaluated once.

4. The exposition-only class template *`impls-for`* ([exec.snd.general]) is specialized for `into_variant` as follows:

```
namespace std::execution {
  template<>
  struct impls-for<into_variant_t> : default-impls {
    static constexpr auto get-state = see below;
    static constexpr auto complete = see below;
  };
}
```

1. The member *`impls-for`*`<into_variant_t>::`*`get-state`* is initialized with a callable object equivalent to the following lambda:

```
[]<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr) noexcept
```

```
    -> type_identity<value_types_of_t<child-type<Sndr>, env_of_t<Rcvr>>> {
    return {};
  }
```

2. The member *impls-for*<into_variant_t>::*complete* is initialized with a callable object equivalent to the following lambda:

```
[]<class State, class Rcvr, class Tag, class... Args>(
    auto, State, Rcvr& rcvr, Tag, Args&&... args) noexcept -> void {
  if constexpr (same_as<Tag, set_value_t>) {
    using variant_type = typename State::type;
    TRY-SET-VALUE(rcvr, variant_type(decayed-tuple<Args...>{std::forward<
  } else {
    Tag()(std::move(rcvr), std::forward<Args>(args)...);
  }
}
```

§ 34.9.11.13. `execution::stopped_as_optional` [exec.stopped.as.optional]

1. `stopped_as_optional` maps a sender's stopped completion operation into a value completion operation as an disengaged `optional`. The sender's value completion operation is also converted into an `optional`. The result is a sender that never completes with stopped, reporting cancellation by completing with an disengaged `optional`.

2. The name `stopped_as_optional` denotes a pipeable sender adaptor object. For a subexpression `sndr`, let `Sndr` be `decltype((sndr))`. The expression `stopped_as_optional(sndr)` is expression-equivalent to:

```
transform_sender(
  get-domain-early(sndr),
  make-sender(stopped_as_optional, {}, sndr))
```

   except that `sndr` is only evaluated once.

3. Let `sndr` and `env` be subexpressions such that `Sndr` is `decltype((sndr))` and `Env` is `decltype((env))`. If *sender-for*<Sndr, stopped_as_optional_t> is `false`, or if the type *single-sender-value-type*<Sndr, Env> is ill-formed or `void`, then the expression `stopped_as_optional.transform_sender(sndr, env)` is ill-formed; otherwise, it is equivalent to:

```
auto&& [_, _, child] = sndr;
using V = single-sender-value-type<Sndr, Env>;
return let_stopped(
    then(std::forward_like<Sndr>(child),
```

```
            []<class... Ts>(Ts&&... ts) noexcept(is_nothrow_constructible_v<V, T
              return optional<V>(in_place, std::forward<Ts>(ts)...);
            }),
        []() noexcept { return just(optional<V>()); });
```

§ 34.9.11.14. `execution::stopped_as_error` [exec.stopped.as.error]

1. `stopped_as_error` maps an input sender's stopped completion operation into an error completion operation as a custom error type. The result is a sender that never completes with stopped, reporting cancellation by completing with an error.
2. The name `stopped_as_error` denotes a pipeable sender adaptor object. For some subexpressions `sndr` and `err`, let `Sndr` be `decltype((sndr))` and let `Err` be `decltype((err))`. If the type `Sndr` does not satisfy `sender` or if the type `Err` doesn't satisfy *`movable-value`*, `stopped_as_error(sndr, err)` is ill-formed. Otherwise, the expression `stopped_as_error(sndr, err)` is expression-equivalent to:

```
transform_sender(
  get-domain-early(sndr),
  make-sender(stopped_as_error, err, sndr))
```

   except that `sndr` is only evaluated once.
3. Let `sndr` and `env` be subexpressions such that `Sndr` is `decltype((sndr))` and `Env` is `decltype((env))`. If *`sender-for`*`<Sndr, stopped_as_error_t>` is `false`, then the expression `stopped_as_error.transform_sender(sndr, env)` is ill-formed; otherwise, it is equivalent to:

```
auto&& [_, err, child] = sndr;
using E = decltype(auto(err));
return let_stopped(
    std::forward_like<Sndr>(child),
    [err = std::forward_like<Sndr>(err)]() mutable noexcept(is_nothrow_move_c
      return just_error(std::move(err));
    });
```

§ 34.9.12. Sender consumers [exec.consumers]

§ 34.9.12.1. `this_thread::sync_wait` [exec.sync.wait]

1. `this_thread::sync_wait` and `this_thread::sync_wait_with_variant` are used to block the current thread of execution until the specified sender completes and to return its async result.

`sync_wait` mandates that the input sender has exactly one value completion signature.

2. Let *`sync-wait-env`* be the following exposition-only class type:

```
namespace std::this_thread {
  struct sync-wait-env {
    execution::run_loop* loop; // exposition only

    auto query(execution::get_scheduler_t) const noexcept {
      return loop->get_scheduler();
    }

    auto query(execution::get_delegation_scheduler_t) const noexcept {
      return loop->get_scheduler();
    }
  };
}
```

3. Let *`sync-wait-result-type`* and *`sync-wait-with-variant-result-type`* be exposition-only alias templates defined as follows:

```
namespace std::this_thread {
  template<execution::sender_in<sync-wait-env> Sndr>
    using sync-wait-result-type =
      optional<execution::value_types_of_t<Sndr, sync-wait-env, decayed-tuple

  template<execution::sender_in<sync-wait-env> Sndr>
    using sync-wait-with-variant-result-type =
      optional<execution::value_types_of_t<Sndr, sync-wait-env>>;
}
```

4. The name `this_thread::sync_wait` denotes a customization point object. For a subexpression `sndr`, let `Sndr` be `decltype((sndr))`. If `sender_in<Sndr, sync-wait-env>` is `false`, the expression `this_thread::sync_wait(sndr)` is ill-formed. Otherwise, it is expression-equivalent to the following, except that `sndr` is evaluated only once:

```
apply_sender(get-domain-early(sndr), sync_wait, sndr)
```

Mandates:

— The type *`sync-wait-result-type`*`<Sndr>` is well-formed.

— `same_as<decltype(e), sync-wait-result-type<Sndr>>` is `true`, where *`e`* is the `apply_sender` expression above.

5. Let *`sync-wait-state`* and *`sync-wait-receiver`* be the following exposition-only class templates:

```
namespace std::this_thread {
  template<class Sndr>
  struct sync-wait-state { // exposition only
    execution::run_loop loop;  // exposition only
    exception_ptr error; // exposition only
    sync-wait-result-type<Sndr> result;  // exposition only
  };

  template<class Sndr>
  struct sync-wait-receiver { // exposition only
    using receiver_concept = execution::receiver_t;
    sync-wait-state<Sndr>* state; // exposition only

    template<class... Args>
    void set_value(Args&&... args) && noexcept;

    template<class Error>
    void set_error(Error&& err) && noexcept;

    void set_stopped() && noexcept;

    sync-wait-env get_env() const noexcept { return {&state->loop}; }
  };
}
```

1.
```
template<class... Args>
void set_value(Args&&... args) && noexcept;
```

   1. *Effects:* Equivalent to:

```
try {
  state->result.emplace(std::forward<Args>(args)...);
} catch (...) {
  state->error = current_exception();
}
state->loop.finish();
```

2.
```
template<class Error>
void set_error(Error&& err) && noexcept;
```

   1. *Effects:* Equivalent to:

```
state->error = AS-EXCEPT-PTR(std::forward<Error>(err)); // see [exec.
state->loop.finish();
```

3. `void set_stopped() && noexcept;`

    1. *Effects:* Equivalent to `state->loop.finish()`.

6. For a subexpression `sndr`, let `Sndr` be `decltype((sndr))`. If `sender_to<Sndr, sync-wait-receiver<Sndr>>` is `false`, the expression `sync_wait.apply_sender(sndr)` is ill-formed; otherwise, it is equivalent to:

```
sync-wait-state<Sndr> state;
auto op = connect(sndr, sync-wait-receiver<Sndr>{&state});
start(op);

state.loop.run();
if (state.error) {
  rethrow_exception(std::move(state.error));
}
return std::move(state.result);
```

7. The behavior of `this_thread::sync_wait(sndr)` is undefined unless:

    1. It blocks the current thread of execution ([defns.block]) with forward progress guarantee delegation ([intro.progress]) until the specified sender completes. *[Note:* The default implementation of `sync_wait` achieves forward progress guarantee delegation by providing a `run_loop` scheduler via the `get_delegation_scheduler` query on the `sync-wait-receiver`'s environment. The `run_loop` is driven by the current thread of execution. — *end note]*

    2. It returns the specified sender's async results as follows:

        1. For a value completion, the result datums are returned in a `tuple` in an engaged `optional` object.

        2. For an error completion, an exception is thrown.

        3. For a stopped completion, a disengaged `optional` object is returned.

8. The name `this_thread::sync_wait_with_variant` denotes a customization point object. For a subexpression `sndr`, let `Sndr` be `decltype(into_variant(sndr))`. If `sender_in<Sndr, sync-wait-env>` is `false`, `this_thread::sync_wait_with_variant(sndr)` is ill-formed. Otherwise, it is expression-equivalent to the following, except `sndr` is evaluated only once:

```
apply_sender(get-domain-early(sndr), sync_wait_with_variant, sndr)
```

Mandates:

— The type `sync-wait-with-variant-result-type<Sndr>` is well-formed.

— `same_as<decltype(e), sync-wait-with-variant-result-type<Sndr>>` is `true`,

where *e* is the `apply_sender` expression above.

9. If *`callable`*`<sync_wait_t, Sndr>` is `false`, the expression `sync_wait_with_variant.apply_sender(sndr)` is ill-formed. Otherwise, it is equivalent to:

```
using result_type = sync-wait-with-variant-result-type<Sndr>;
if (auto opt_value = sync_wait(into_variant(sndr))) {
  return result_type(std::move(get<0>(*opt_value)));
}
return result_type(nullopt);
```

10. The behavior of `this_thread::sync_wait_with_variant(sndr)` is undefined unless:

    1. It blocks the current thread of execution ([defns.block]) with forward progress guarantee delegation ([intro.progress]) until the specified sender completes. *[Note:* The default implementation of `sync_wait_with_variant` achieves forward progress guarantee delegation by relying on the forward progress guarantee delegation provided by `sync_wait`. — *end note]*
    2. It returns the specified sender's async results as follows:
        1. For a value completion, the result datums are returned in an engaged `optional` object that contains a `variant` of `tuple`s.
        2. For an error completion, an exception is thrown.
        3. For a stopped completion, a disengaged `optional` object is returned.

§ 34.10. Sender/receiver utilities **[exec.utils]**

§ 34.10.1. `execution::completion_signatures` [exec.utils.cmplsigs]

1. `completion_signatures` is a type that encodes a set of completion signatures ([async.ops]).
2. [*Example:*

```
struct my_sender {
  using sender_concept = sender_t;
  using completion_signatures =
    execution::completion_signatures<
      set_value_t(),
      set_value_t(int, float),
      set_error_t(exception_ptr),
      set_error_t(error_code),
      set_stopped_t()>;
};

// Declares my_sender to be a sender that can complete by calling
```

```
// one of the following for a receiver expression rcvr:
//    set_value(rcvr)
//    set_value(rcvr, int{...}, float{...})
//    set_error(rcvr, exception_ptr{...})
//    set_error(rcvr, error_code{...})
//    set_stopped(rcvr)
```

-- *end example*]

3. [exec.utils.cmplsigs] makes use of the following exposition-only entities:

```
template<class Fn>
  concept completion-signature = see below;

template<bool>
  struct indirect-meta-apply {
    template<template<class...> class T, class... As>
      using meta-apply = T<As...>; // exposition only
  };

template<class...>
  concept always-true = true; // exposition only
```

1. A type `Fn` satisfies *`completion-signature`* if and only if it is a function type with one of the following forms:

   — `set_value_t(Vs...)`, where *`Vs`* is a pack of object or reference types.

   — `set_error_t(Err)`, where *`Err`* is an object or reference type.

   — `set_stopped_t()`

```
template<class Tag,
         valid-completion-signatures Completions,
         template<class...> class Tuple,
         template<class...> class Variant>
  using gather-signatures = see below;
```

2. Let `Fns` be a pack of the arguments of the `completion_signatures` specialization named by `Completions`, let *`TagFns`* be a pack of the function types in `Fns` whose return types are `Tag`, and let Ts_n be a pack of the function argument types in the n-th type in *`TagFns`*. Then, given two variadic templates *`Tuple`* and *`Variant`*, the type *`gather-signatures`*`<Tag, Completions, Tuple, Variant>` names the type *`META-APPLY`*`(`*`Variant`*`, `*`META-APPLY`*`(`*`Tuple`*`, `Ts_0`...), `*`META-APPLY`*`(`*`Tuple`*`, `Ts_1`...), ... `*`META-APPLY`*`(`*`Tuple`*`, `Ts_{m-1}`...))`, where m is the size of the pack *`TagFns`* and *`META-APPLY`*`(`*`T`*`, `*`As`*`...)` is equivalent to:

```
typename indirect-meta-apply<always-true<As...>>::template meta-apply<T,
```

3. *[Note:* The purpose of `META-APPLY` is to make it valid to use non-variadic templates as `Variant` and `Tuple` arguments to `gather-signatures`. — *end note]*

4.
```
namespace std::execution {
  template<completion-signature... Fns>
    struct completion_signatures {};

  template<class Sndr,
           class Env = empty_env,
           template<class...> class Tuple = decayed-tuple,
           template<class...> class Variant = variant-or-empty>
      requires sender_in<Sndr, Env>
    using value_types_of_t =
        gather-signatures<set_value_t, completion_signatures_of_t<Sndr, Env>,

  template<class Sndr,
           class Env = empty_env,
           template<class...> class Variant = variant-or-empty>
      requires sender_in<Sndr, Env>
    using error_types_of_t =
        gather-signatures<set_error_t, completion_signatures_of_t<Sndr, Env>,

  template<class Sndr, class Env = empty_env>
      requires sender_in<Sndr, Env>
    inline constexpr bool sends_stopped =
        !same_as<
          type-list<>,
          gather-signatures<set_stopped_t, completion_signatures_of_t<Sndr, E
}
```

§ 34.10.2. `execution::transform_completion_signatures` [exec.utils.tfxcmplsigs]

1. `transform_completion_signatures` is an alias template used to transform one set of completion signatures into another. It takes a set of completion signatures and several other template arguments that apply modifications to each completion signature in the set to generate a new specialization of `completion_signatures`.

2. [*Example:*

```
// Given a sender Sndr and an environment Env, adapt the completion
// signatures of Sndr by lvalue-ref qualifying the values, adding an addition
// exception_ptr error completion if its not already there, and leaving the
// other completion signatures alone.
```

```
template<class... Args>
  using my_set_value_t =
    completion_signatures<
      set_value_t(add_lvalue_reference_t<Args>...)>;

using my_completion_signatures =
  transform_completion_signatures<
    completion_signatures_of_t<Sndr, Env>,
    completion_signatures<set_error_t(exception_ptr)>,
    my_set_value_t>;
```

-- *end example*]

3. [exec.utils.tfxcmplsigs] makes use of the following exposition-only entities:

```
template<class... As>
  using default-set-value =
    completion_signatures<set_value_t(As...)>;

template<class Err>
  using default-set-error =
    completion_signatures<set_error_t(Err)>;
```

4.
```
namespace std::execution {
  template<valid-completion-signatures InputSignatures,
           valid-completion-signatures AdditionalSignatures =
               completion_signatures<>,
           template<class...> class SetValue = default-set-value,
           template<class> class SetError = default-set-error,
           valid-completion-signatures SetStopped =
               completion_signatures<set_stopped_t()>>
  using transform_completion_signatures =
    completion_signatures<see below>;
}
```

1. `SetValue` shall name an alias template such that for any pack of types `As`, the type `SetValue<As...>` is either ill-formed or else *`valid-completion-signatures`*`<SetValue<As...>>` is satisfied.
2. `SetError` shall name an alias template such that for any type `Err`, `SetError<Err>` is either ill-formed or else *`valid-completion-signatures`*`<SetError<Err>>` is satisfied.

Then:

3. Let `Vs...` be a pack of the types in the *`type-list`* named by *`gather-signatures`*`<set_value_t, InputSignatures, SetValue, `*`type-list`*`>`.

4. Let `Es...` be a pack of the types in the *`type-list`* named by *`gather-signatures`*`<set_error_t, InputSignatures, type_identity_t,` *`error-list`*`>`, where *`error-list`* is an alias template such that *`error-list`*`<Ts...>` is *`type-list`*`<SetError<Ts>...>`.

5. Let `Ss` name the type `completion_signatures<>` if *`gather-signatures`*`<set_stopped_t, InputSignatures,` *`type-list`*`,` *`type-list`*`>` is an alias for the type *`type-list`*`<>`; otherwise, `SetStopped`.

Then:

6. If any of the above types are ill-formed, then `transform_completion_signatures<InputSignatures, AdditionalSignatures, SetValue, SetError, SetStopped>` is ill-formed.

7. Otherwise, `transform_completion_signatures<InputSignatures, AdditionalSignatures, SetValue, SetError, SetStopped>` is the type `completion_signatures<Sigs...>` where `Sigs...` is the unique set of types in all the template arguments of all the `completion_signatures` specializations in the set `AdditionalSignatures, Vs..., Es..., Ss`.

§ 34.11. Execution contexts **[exec.ctx]**

§ 34.11.1. `execution::run_loop` [exec.run.loop]

1. A `run_loop` is an execution resource on which work can be scheduled. It maintains a thread-safe first-in-first-out queue of work. Its `run()` member function removes elements from the queue and executes them in a loop on the thread of execution that calls `run()`.

2. A `run_loop` instance has an associated *count* that corresponds to the number of work items that are in its queue. Additionally, a `run_loop` instance has an associated *state* that can be one of *starting*, *running*, or *finishing*.

3. Concurrent invocations of the member functions of `run_loop` other than `run` and its destructor do not introduce data races. The member functions `pop_front`, `push_back`, and `finish` execute atomically.

4. *Recommended practice:* Implementations are encouraged to use an intrusive queue of operation states to hold the work units to make scheduling allocation-free.

```
namespace std::execution {
  class run_loop {
    // [exec.run.loop.types] Associated types
    class run-loop-scheduler; // exposition only
    class run-loop-sender; // exposition only
    struct run-loop-opstate-base { // exposition only
      virtual void execute() = 0;  // exposition only
```

```
    run_loop* loop;  // exposition only
    run-loop-opstate-base* next;  // exposition only
  };
  template<class Rcvr>
    using run-loop-opstate = unspecified; // exposition only

  // [exec.run.loop.members] Member functions:
  run-loop-opstate-base* pop-front(); // exposition only
  void push-back(run-loop-opstate-base*); // exposition only

public:
  // [exec.run.loop.ctor] construct/copy/destroy
  run_loop() noexcept;
  run_loop(run_loop&&) = delete;
  ~run_loop();

  // [exec.run.loop.members] Member functions:
  run-loop-scheduler get_scheduler();
  void run();
  void finish();
};
}
```

§ 34.11.1.1. Associated types **[exec.run.loop.types]**

```
class run-loop-scheduler;
```

1. *run-loop-scheduler* is an unspecified type that models `scheduler`.
2. Instances of *run-loop-scheduler* remain valid until the end of the lifetime of the `run_loop` instance from which they were obtained.
3. Two instances of *run-loop-scheduler* compare equal if and only if they were obtained from the same `run_loop` instance.
4. Let *sch* be an expression of type *run-loop-scheduler*. The expression `schedule(sch)` has type *run-loop-sender* and is not potentially-throwing if *sch* is not potentially-throwing.

```
class run-loop-sender;
```

1. *run-loop-sender* is an exposition-only type that satisfies `sender`. For any type `Env`, `completion_signatures_of_t<run-loop-sender, Env>` is:

```
completion_signatures<set_value_t(), set_error_t(exception_ptr), set_stopped_
```

2. An instance of `run-loop-sender` remains valid until the end of the lifetime of its associated `run_loop` instance.

3. Let `sndr` be an expression of type `run-loop-sender`, let `rcvr` be an expression such that `receiver_of<decltype((rcvr)), CS>` is `true` where `CS` is the `completion_signatures` specialization above. Let `C` be either `set_value_t` or `set_stopped_t`. Then:

   — The expression `connect(sndr, rcvr)` has type `run-loop-opstate<decay_t<decltype((rcvr))>>` and is potentially-throwing if and only if `(void(sndr), auto(rcvr))` is potentially-throwing.

   — The expression `get_completion_scheduler<C>(get_env(sndr))` is potentially-throwing if and only if `sndr` is potentially-throwing, has type `run-loop-scheduler`, and compares equal to the `run-loop-scheduler` instance from which `sndr` was obtained.

```
template<class Rcvr>
  struct run-loop-opstate;
```

1. `run-loop-opstate<Rcvr>` inherits privately and unambiguously from `run-loop-opstate-base`.

2. Let `o` be a non-`const` lvalue of type `run-loop-opstate<Rcvr>`, and let `REC(o)` be a non-`const` lvalue reference to an instance of type `Rcvr` that was initialized with the expression `rcvr` passed to the invocation of `connect` that returned `o`. Then:

   — The object to which `REC(o)` refers remains valid for the lifetime of the object to which `o` refers.

   — The type `run-loop-opstate<Rcvr>` overrides `run-loop-opstate-base::execute()` such that `o.execute()` is equivalent to:

```
if (get_stop_token(REC(o)).stop_requested()) {
  set_stopped(std::move(REC(o)));
} else {
  set_value(std::move(REC(o)));
}
```

   — The expression `start(o)` is equivalent to:

```
try {
  o.loop->push-back(addressof(o));
} catch(...) {
  set_error(std::move(REC(o)), current_exception());
}
```

§ 34.11.1.2. Constructor and destructor **[exec.run.loop.ctor]**

```
run_loop() noexcept;
```

1. *Postconditions: count* is `0` and *state* is *starting*.

```
~run_loop();
```

1. *Effects:* If *count* is not `0` or if *state* is *running*, invokes `terminate()`. Otherwise, has no effects.

§ 34.11.1.3. Member functions **[exec.run.loop.members]**

```
run-loop-opstate-base* pop-front();
```

1. *Effects:* Blocks ([defns.block]) until one of the following conditions is `true`:

   — *count* is `0` and *state* is *finishing*, in which case *`pop-front`* returns `nullptr`; or

   — *count* is greater than `0`, in which case an item is removed from the front of the queue, *count* is decremented by `1`, and the removed item is returned.

```
void push-back(run-loop-opstate-base* item);
```

1. *Effects:* Adds `item` to the back of the queue and increments *count* by `1`.
2. *Synchronization:* This operation synchronizes with the *`pop-front`* operation that obtains `item`.

```
run-loop-scheduler get_scheduler();
```

1. *Returns:* An instance of *`run-loop-scheduler`* that can be used to schedule work onto this `run_loop` instance.

```
void run();
```

1. *Precondition: state* is *starting*.
2. *Effects:* Sets the *state* to *running*. Then, equivalent to:

```
while (auto* op = pop-front()) {
  op->execute();
}
```

3. *Remarks:* When *state* changes, it does so without introducing data races.

```
void finish();
```

1. *Effects:* Changes *state* to *finishing*.
2. *Synchronization:* `finish` synchronizes with the *`pop-front`* operation that returns `nullptr`.

§ 34.12. Coroutine utilities [exec.coro.utils]

§ 34.12.1. `execution::as_awaitable` [exec.as.awaitable]

1. `as_awaitable` transforms an object into one that is awaitable within a particular coroutine. [exec.coro.utils] makes use of the following exposition-only entities:

```
namespace std::execution {
  template<class Sndr, class Promise>
    concept awaitable-sender =
      single-sender<Sndr, env_of_t<Promise>> &&
      sender_to<Sndr, awaitable-receiver> && // see below
      requires (Promise& p) {
        { p.unhandled_stopped() } -> convertible_to<coroutine_handle<>>;
      };

  template<class Sndr, class Promise>
    class sender-awaitable;
}
```

2. The type *`sender-awaitable`*`<Sndr, Promise>` is equivalent to:

```
namespace std::execution {
  template<class Sndr, class Promise>
  class sender-awaitable {
    struct unit {};                                          // expositio
    using value-type =                                       // expositio
      single-sender-value-type<Sndr, env_of_t<Promise>>;
    using result-type =                                      // expositio
      conditional_t<is_void_v<value-type>, unit, value-type>;
    struct awaitable-receiver;                               // expositio

    variant<monostate, result-type, exception_ptr> result{}; // expositio
    connect_result_t<Sndr, awaitable-receiver> state;        // expositio

  public:
    sender-awaitable(Sndr&& sndr, Promise& p);
```

```
    static constexpr bool await_ready() noexcept { return false; }
    void await_suspend(coroutine_handle<Promise>) noexcept { start(state)
    value-type await_resume();
  };
}
```

1. *awaitable-receiver* is equivalent to:

```
struct awaitable-receiver {
  using receiver_concept = receiver_t;
  variant<monostate, result-type, exception_ptr>* result-ptr; // expo
  coroutine_handle<Promise> continuation;                     // expo
  // ... see below
};
```

Let `rcvr` be an rvalue expression of type *awaitable-receiver*, let `crcvr` be a `const` lvalue that refers to `rcvr`, let `vs` be a pack of subexpressions, and let `err` be an expression of type `Err`. Then:

1. If `constructible_from<result-type, decltype((vs))...>` is satisfied, the expression `set_value(rcvr, vs...)` is equivalent to:

```
try {
  rcvr.result-ptr->template emplace<1>(vs...);
} catch(...) {
  rcvr.result-ptr->template emplace<2>(current_exception());
}
rcvr.continuation.resume();
```

   Otherwise, `set_value(rcvr, vs...)` is ill-formed.

2. The expression `set_error(rcvr, err)` is equivalent to:

```
rcvr.result-ptr->template emplace<2>(AS-EXCEPT-PTR(err)); // see
rcvr.continuation.resume();
```

3. The expression `set_stopped(rcvr)` is equivalent to:

```
static_cast<coroutine_handle<>>(rcvr.continuation.promise().unhar
```

4. For any expression `tag` whose type satisfies *forwarding-query* and for any pack of subexpressions `as`, `get_env(crcvr).query(tag, as...)` is expression-equivalent to `tag(get_env(as_const(crcvr.continuation.promise())), as...)`.

2. `sender-awaitable(Sndr&& sndr, Promise& p);`

    1. *Effects:* Initializes `state` with `connect(std::forward<Sndr>(sndr), awaitable-receiver{addressof(result), coroutine_handle<Promise>::from_promise(p)})`.

3. `value-type await_resume();`

    1. *Effects:* Equivalent to:

```
if (result.index() == 2)
  rethrow_exception(get<2>(result));
if constexpr (!is_void_v<value-type>)
  return std::forward<value-type>(get<1>(result));
```

2. `as_awaitable` is a customization point object. For subexpressions `expr` and `p` where `p` is an lvalue, `Expr` names the type `decltype((expr))` and `Promise` names the type `decay_t<decltype((p))>`, `as_awaitable(expr, p)` is expression-equivalent to:

    1. `expr.as_awaitable(p)` if that expression is well-formed.

        — *Mandates:* `is-awaitable<A, Promise>` is `true`, where `A` is the type of the expression above.

    2. Otherwise, `(void(p), expr)` if `is-awaitable<Expr, U>` is `true`, where `U` is an unspecified class type that is not `Promise` and that lacks a member named `await_transform`.

        — *Preconditions:* `is-awaitable<Expr, Promise>` is `true` and the expression `co_await expr` in a coroutine with promise type `U` is expression-equivalent to the same expression in a coroutine with promise type `Promise`.

    3. Otherwise, `sender-awaitable{expr, p}` if `awaitable-sender<Expr, Promise>` is `true`.

    4. Otherwise, `(void(p), expr)`.

    except that the evaluations of `expr` and `p` are indeterminately sequenced.

§ 34.12.2. `execution::with_awaitable_senders` [exec.with.awaitable.senders]

1. `with_awaitable_senders`, when used as the base class of a coroutine promise type, makes senders awaitable in that coroutine type.

    In addition, it provides a default implementation of `unhandled_stopped()` such that if a sender completes by calling `set_stopped`, it is treated as if an uncatchable "stopped" exception were thrown from the *await-expression*. *[Note:* The coroutine is never resumed, and the `unhandled_stopped` of the coroutine caller's promise type is called. — *end note]*

```
namespace std::execution {
  template<class-type Promise>
    struct with_awaitable_senders {
      template<class OtherPromise>
        requires (!same_as<OtherPromise, void>)
      void set_continuation(coroutine_handle<OtherPromise> h) noexcept;

      coroutine_handle<> continuation() const noexcept { return continuation;

      coroutine_handle<> unhandled_stopped() noexcept {
        return stopped-handler(continuation.address());
      }

      template<class Value>
      see below await_transform(Value&& value);

    private:
      // exposition only
      [[noreturn]] static coroutine_handle<> default-unhandled-stopped(void*)
        terminate();
      }
      coroutine_handle<> continuation{}; // exposition only
      // exposition only
      coroutine_handle<> (*stopped-handler)(void*) noexcept = &default-unhana
    };
}
```

2.
```
template<class OtherPromise>
  requires (!same_as<OtherPromise, void>)
void set_continuation(coroutine_handle h) noexcept;
```

1. *Effects:* Equivalent to:

```
continuation = h;
if constexpr ( requires(OtherPromise& other) { other.unhandled_stopped(); } )
  stopped-handler = [](void* p) noexcept -> coroutine_handle<> {
    return coroutine_handle<OtherPromise>::from_address(p)
      .promise().unhandled_stopped();
  };
} else {
  stopped-handler = &default-unhandled-stopped;
}
```

3.
```
template<class Value>
call-result-t<as_awaitable_t, Value, Promise&> await_transform(Value&& value)
```

1. *Effects:* Equivalent to:

```
return as_awaitable(std::forward<Value>(value), static_cast<Promise&>(*this))
```

§ Index

§ Terms defined by this specification

associated, in § 33.3.1

asynchronous operation, in § 34.3

asynchronous operation lifetime, in § 34.3

async lifetime, in § 34.3

async result, in § 34.3

attributes, in § 34.3

awaitable, in § 34.9.3

callback function, in § 33.3.3

caller, in § 34.3

child operations, in § 34.3

child sender, in § 34.3

complete, in § 34.3

completion function, in § 34.3

completion operation, in § 34.3

completion scheduler, in § 34.3

completion signature, in § 34.3

completion tag, in § 34.3

connect, in § 34.3

decay-copied from, in § 16

disengaged, in § 33.3.3

disposition, in § 34.3

environment, in § 34.3

error completion, in § 34.3

execution resource, in § 34.3

multi-shot sender, in § 4.7

operation state, in § 34.3

parent operation, in § 34.3

parent sender, in § 34.3

permissible completion, in § 34.9.2

pipeable, in § 4.12

piped, in § 4.12

query, in § 34.2.1

queryable object, in § 34.2.1

query object, in § 34.2.1

receive, in § 34.3

receiver, in § 34.3

schedule-expression, in § 34.3

scheduler, in § 34.3

schedule sender, in § 34.3

send, in § 34.3

sender, in § 34.3

sender adaptor, in § 34.3

sender algorithm, in § 34.3

sender consumer, in § 34.3

sender factory, in § 34.3

single-shot sender, in § 4.7

start, in § 34.3

stoppable callback deregistration, in § 33.3.3

stoppable callback registration, in § 33.3.3

stopped completion, in § 34.3

stop request, in § 33.3.1

stop request operation, in § 33.3.3

stop state, in § 33.3.1

Strictly lazy submission, in § 5.5

value completion, in § 34.3

§ References

§ Informative References

[HPX]

Hartmut Kaiser; et al. *HPX - The C++ Standard Library for Parallelism and Concurrency*. URL: https://doi.org/10.21105/joss.02352

[N4885]

Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 17 March 2021. URL: https://wg21.link/n4885

[P0443R14]

Jared Hoberock, Michael Garland, Chris Kohlhoff, Chris Mysen, H. Carter Edwards, Gordon Brown, D. S. Hollman. *A Unified Executors Proposal for C++*. 15 September 2020. URL: https://wg21.link/p0443r14

[P0981R0]

Richard Smith, Gor Nishanov. *Halo: coroutine Heap Allocation eLision Optimization: the joint response*. 18 March 2018. URL: https://wg21.link/p0981r0

[P1056R1]

Lewis Baker, Gor Nishanov. *Add lazy coroutine (coroutine task) type*. 7 October 2018. URL: https://wg21.link/p1056r1

[P1895R0]

Lewis Baker, Eric Niebler, Kirk Shoop. *tag_invoke: A general pattern for supporting customisable functions*. 8 October 2019. URL: https://wg21.link/p1895r0

[P1897R3]

Lee Howes. *Towards C++23 executors: A proposal for an initial set of algorithms*. 16 May 2020. URL: https://wg21.link/p1897r3

[P2175R0]

Lewis Baker. *Composable cancellation for sender-based async operations*. 15 December 2020. URL: https://wg21.link/p2175r0

[P2855R1]

Ville Voutilainen. *Member customization points for Senders and Receivers*. 22 February 2024. URL: https://wg21.link/p2855r1

[P2999R3]

Eric Niebler. *Sender Algorithm Customization*. 13 December 2023. URL: https://wg21.link/p2999r3

[P3149R3]

Ian Petersen, Ján Ondrušek; Jessica Wong; Kirk Shoop; Lee Howes; Lucian Radu Teodorescu;.

async_scope — Creating scopes for non-sequential concurrency. 22 May 2024. URL: https://wg21.link/p3149r3

[P3175R3]
Eric Niebler. *Reconsidering the std::execution::on algorithm*. 2024-06-24. URL: https://wg21.link/P3175R3

[P3187R1]
Kirk Shoop, Lewis Baker. *remove ensure_started and start_detached from P2300*. 21 March 2024. URL: https://wg21.link/p3187r1

[P3303R1]
Eric Niebler. *Fixing Lazy Sender Algorithm Customization*. 2024-06-24. URL: https://wg21.link/P3303R1